



Institut Supérieur des Études Technologiques de Mahdia

Département : Technologies de l'Informatique

Développement des Systèmes Informatiques — DSI3.2

Rapport de Projet de Fin d'Études

**Koda — Robot compagnon intelligent et émotionnel connecté,
pilote par une application cross-platform**

**Système Embarqué • Intelligence Artificielle • Application Mobile
Cross-Plateforme**

Réalisé par :

Oussema KARIA

Salahedin JENDOUBI

Encadré par :

M.Wahid BANOUR — (Encadrant)

M.Hamza SASSI (Encadrant Entreprise)

Organisme d'accueil : —

Période : Février 2026 – Juin 2026

Résumé

Ce rapport présente KODA, un système embarqué intelligent conçu pour être un assistant de vie et un ami virtuel doté de sa propre personnalité et nature. KODA est capable de répondre de manière réfléchie à n'importe quelle personne, de raisonner comme un être humain et de reconnaître les individus grâce à ses modes de sécurité et de configuration.

L'architecture du système repose sur plusieurs composants complémentaires : un système embarqué Raspberry Pi constituant le cœur physique du robot ; un moteur de workflows n8n formant le cerveau de KODA, responsable de la préparation et de la réflexion des réponses ; une couche de services Python agissant comme distributeur universel, assurant la communication entre n8n, la base de données, l'application mobile et le Raspberry Pi tout en exposant les endpoints de tout l'écosystème ; une application mobile cross-plateforme pour la configuration et le suivi de vie du robot ; et une base de données persistante servant de mémoire à KODA.

Mots-clés : Système Embarqué, Assistant Virtuel, Raspberry Pi, n8n, Python, Cross-Plateforme, Intelligence Artificielle, Assistant de Vie, KODA.

DÉDICACE

Je dédie ce travail à ma famille, pilier silencieux de ma force et source inépuisable de soutien.

*Merci pour vos sacrifices immenses et pour avoir cru en moi tout au long de mon parcours
académique.*

*À mes parents, pour leur amour inconditionnel, leurs encouragements et leurs prières qui ont été la
lumière guidant chacun de mes pas. Ce succès est le vôtre autant que le mien.*

*À mes amis et collègues, pour leur soutien moral et pour avoir partagé avec moi cette aventure
inoubliable.*

*À mon binôme Salahedin, pour sa persévérance, sa créativité et sa complicité tout au long de ce
projet.*

*À tous ceux qui ont contribué, même par un simple mot d'encouragement, à l'accomplissement de
ce travail.*

Oussema KARIA

DÉDICACE

À ma famille,
pour son soutien inconditionnel et ses sacrifices immenses
tout au long de mon parcours académique.

À mon binôme Oussema, pour son enthousiasme, sa rigueur
et sa précieuse collaboration dans la réalisation de ce projet.

À mes encadrants,
pour leur guidance, leur patience et leur expertise technique.

Salahedin JENDOUBI

Remerciements

À notre encadrant académique, M. Wahid Banour,

nous adressons nos remerciements les plus sincères pour la qualité de l'accompagnement pédagogique, la disponibilité dont il a fait preuve à chaque étape, ainsi que pour ses conseils exigeants qui ont permis de structurer notre démarche et de renforcer la rigueur scientifique et technique de ce travail.

À notre encadrant en entreprise, M. Hamza Sassi,

nous exprimons toute notre gratitude pour l'accueil au sein de l'organisme d'accueil, pour le temps accordé aux échanges techniques et pour les orientations concrètes qui ont éclairé nos choix d'architecture et d'intégration tout au long du projet.

À la direction et à l'ensemble du corps enseignant de l'ISET de Mahdia,

et en particulier au département *Développement des systèmes informatiques* (DSI), nous témoignons notre reconnaissance pour la formation dispensée, pour l'exigence professionnelle transmise et pour l'environnement d'apprentissage qui a rendu possible la réalisation de ce projet de fin d'études.

À nos proches, amis et camarades,

nous disons merci pour la patience, les encouragements et la confiance manifestés durant cette période intensive ; leur soutien moral a compté autant que le travail collectif mené au sein du binôme.

Nous saluons enfin toute personne ayant contribué, directement ou indirectement, à l'aboutissement de ce rapport.

Oussema Karia

Salahedin Jendoubi

SOMMAIRE

Introduction Générale	1
I Cadre Général du Projet et Présentation de l'Organisme	2
I.1 Introduction du chapitre	3
I.2 Contexte Général du Projet	3
I.3 Présentation de l'Organisme d'Accueil (MicroEdition)	3
I.3.1 Profil de l'Entreprise	3
I.3.2 Services	4
I.3.3 Recherche & Développement (R&D)	4
I.3.4 Structure Organisationnelle	4
I.4 Problématique	5
I.5 Analyse des Solutions Existantes	5
I.5.1 Étude de l'existant	6
I.5.2 Synthèse et Critique : Le "Manque d'Humanité"	6
I.5.3 La Différenciation de Koda	7
I.6 Solution Proposée	7
I.7 Méthodologie du Projet	8
I.7.1 Approche Agile et Cycle en V	8
I.8 Conclusion	9
II Préparation du Projet	10
II.1 Introduction du chapitre	11
II.2 Spécification des besoins	11
II.2.1 Spécifications des besoins fonctionnels	11
II.2.2 Spécification des besoins non fonctionnels	12
II.2.3 Architecture du système	12
II.2.4 Identification des acteurs	13
II.3 Pilotage du projet — Modèle en spirale et cycles de développement	13
II.3.1 Équipe et rôles	14

II.3.2	Découpage du travail : six cycles	14
II.4	Planification des cycles	15
II.5	Architecture	15
II.5.1	Architecture de l'application	15
II.5.1.1	Architecture en oignon (<i>Onion Architecture</i>) du backend	17
II.6	Environnement de travail	17
II.6.1	Environnement matériel	17
II.7	Environnement logiciel	17
II.7.1	Outils de développement et modélisation	17
II.7.2	Langages et frameworks utilisés	18
II.8	Figures et diagrammes prévus pour ce chapitre	18
II.9	Conclusion	20
III	Cycle 1 : ANALYSEUR ET RÉFLEXION	21
III.1	Introduction	22
III.2	Définitions Fondamentales	22
III.2.1	n8n	22
III.2.2	Webhook	23
III.2.3	Grand modèle de langage (LLM)	23
III.2.4	OpenRouter	24
III.2.5	Schéma de classes — persistance KODA	24
III.3	Fondements Théoriques	26
III.3.1	La Théorie du Double Traitement	26
III.4	Fonctionnalités Techniques	26
III.4.1	Premier Flux — Webhook 1 : Analyse et Réponse	26
III.4.1.1	Principe Général	26
III.4.1.2	Format de la Requête Entrante	27
III.4.1.3	Description détaillée des blocs de traitement	27
III.4.1.3.1	Découpe du travail et rôle des blocs (1 à 6).	27
III.4.1.3.2	Block 1 : Analyse du message utilisateur et classifica- tion d'intention	29
III.4.1.3.3	Block 2 : Vérification de la Question dans l'Historique	34
III.4.1.3.4	Block 3 : Classification Sémantique de la Question	36
III.4.1.3.4.1	Basic LLM Chain en lieu et place de l'AI Agent	38

III.4.1.3.4.2	Configuration OpenRouter (modèle Gemini) . .	38
III.4.1.3.4.3	SerpAPI et OpenRouter dans l'écosystème d'API du projet	39
III.4.1.3.4.4	Prompt Engineering — Analyseur de Dépen- dances Mémoire	40
III.4.1.3.4.5	Détail des Types de Questions Classifiés	42
III.4.1.3.5	Block 4 : Reconnaissance et enregistrement de l'utili- sateur	42
III.4.1.3.5.1	Routage par <i>Switch1</i> (résultat du Block 3). . . .	42
III.4.1.3.5.2	Diagramme de séquence du block 4.	44
III.4.1.3.6	Block 5 : Récupération Contextuelle et Génération de la Réponse	46
III.4.1.3.6.1	Branche Mémoirelle (<code>use_memory = true</code>) . .	46
III.4.1.3.6.2	Branche Générale (<code>use_memory = false</code>) . .	48
III.4.1.4	Vue Complète du Block 5 — Les Deux Branches	49
III.4.1.4.1	Block 6 : Extraction structurée des <i>notes</i> (test final) . .	51
III.4.1.4.1.1	Sous-bloc de détection (<i>If</i>) et <i>Basic LLM Chain 2</i> . .	52
III.4.1.4.2	Block 7 : Mise à Jour de Mémoire	53
III.4.1.5	Diagramme de Séquence du Webhook 1	55
III.4.2	Deuxième Flux — Webhook 2 : Filtre et Synthèse Journalière	55
III.4.2.1	Principe et Inspiration	55
III.4.2.2	Fonctionnement du Webhook 2	56
III.4.2.3	Structure des Fiches de Synthèse Générées	56
III.4.2.4	Impact sur le Système Global	57
III.4.3	Troisième Flux — Webhook 3 : Spontanéité et Interaction Proactive	58
III.4.3.1	Principe : La Spontanéité comme Dimension Humaine	58
III.4.3.2	Déclenchement et Fonctionnement	58
III.4.3.3	Génération de la Phrase d'Accroche	58
III.5	Vue Générale du Cycle 1	59
III.6	Conclusion	60
IV	Cycle 2 : Distributeur de Services	61
IV.1	Introduction	62
IV.2	Architecture Utilisée	62

IV.2.1	Choix du langage : Python	62
IV.2.2	Clean Architecture : principes et application	63
IV.2.3	Structure des répertoires du projet	64
IV.3	Fonctionnalités Techniques	65
IV.3.1	Communication avec la base de données (Supabase / PostgreSQL)	65
IV.3.1.1	Modèle relationnel et rôle des tables	66
IV.3.2	Services Azure : Speech-to-Text et Text-to-Speech	67
IV.3.3	Service de reconnaissance des personnes	69
IV.3.4	Interface avec le workflow n8n	71
IV.3.4.0.1	Images recommandées pour cette section	72
IV.3.5	Services pour l'application mobile multiplateforme	72
IV.3.6	Communication avec le système embarqué (robot réel)	74
IV.4	Problèmes Rencontrés et Solutions Apportées	74
IV.5	Conclusion	75
V	Cycle 3 : Système Embarqué — Partie Matérielle	77
V.1	Introduction	78
V.2	Architecture Matérielle Globale	78
V.3	Conception mécanique et châssis 3D (Blender)	79
V.4	Description Détaillée des Composants	80
V.4.1	Unité Centrale : Raspberry Pi 4 Model B	80
V.4.2	Unité de Commande Moteur : Arduino Uno + Shield L293D	80
V.4.3	Système d'Actionneurs	82
V.4.3.1	Moteurs DC — Mobilité	82
V.4.3.2	Servomoteurs — Articulations	82
V.4.4	Système Audio	83
V.4.4.1	Entrée Audio : ReSpeaker 4-Mic Array USB	83
V.4.4.2	Sortie Audio : double chaîne (jack filaire et Bluetooth)	83
V.4.5	Système de Vision : Caméra Raspberry Pi	84
V.4.6	Interface d'Expression Faciale : Nextion NX4832T035_011	84
V.4.7	Système d'Alimentation	85
V.5	Schéma de Câblage Global	85
V.6	Assemblage et points d'attention	86
V.7	Validation matérielle — tests par composant	86

V.7.1	Test d'intégration matérielle	87
V.8	Conclusion	87

LISTE DES FIGURES

I.1	Architecture Globale de la Solution Proposée	8
II.1	Synthèse visuelle des besoins fonctionnels KODA (chapitre II)	12
II.2	Architecture multicouche du système KODA (vue chapitre II)	13
II.3	Acteurs principaux et contexte d'intégration du système KODA	14
II.4	Modèle en spirale et enchaînement des six cycles de développement KODA	16
II.5	Planification des cycles : jalons, chevauchements et dépendances	16
II.6	Environnement logiciel : outils de développement et chaîne technique (chapitre II) .	18
III.1	Schéma des classes métier — persistance KODA (Webhook 1 et mémorisation ; relations utilisateur / seeting, conversations, historique, notes).	25
III.2	Architecture cognitive de KODA inspirée de la théorie du double traitement	27
III.3	Webhook 1 — diagramme de séquence général entre les blocs 1 à 6 (vue macro ; persistance : Block 7).	29
III.4	Block 1 — Workflow n8n : nœud Code <i>Test</i> , <i>Switch</i> à six sorties (musique SerpAPI, identité ON/OFF, direction, historique, branche dédiée)	31
III.5	Block 1 — Diagramme de séquence du premier bloc	32
III.6	Block 1 — Diagramme de cas d'utilisation associé au premier bloc	33
III.7	Block 2 — Vérification et comparaison avec les 5 derniers messages	35
III.8	Block 2 — Diagramme de séquence du deuxième bloc	35
III.9	Block 3 — Chaîne n8n : Code5, Identité1 (PostgreSQL), Basic LLM Chain avec modèle OpenRouter, puis Code2 pour le format de sortie	37
III.10	Block 3 — Diagramme de séquence du troisième bloc	37
III.11	Block 3 — Diagramme de cas d'utilisation associé au troisième bloc	38
III.12	Block 3 — Paramètres du nœud OpenRouter Chat Model (credential, modèle Gemini via OpenRouter)	39
III.13	Intégration SerpAPI — recherche structurée (écosystème d'API aux côtés d'Open- Router pour les LLM)	39
III.14	Block 4 — <i>Switch1</i> , branche enregistrement (<i>presentation</i>), branche LLM (<i>unkonu</i>) et poursuite vers le Block 5	43

III.15	Block 4 — Entrée JSON du <i>Switch1</i> (persone = unkonu, sortie Block 3)	44
III.16	Block 4 — Sortie texte : invitation à la première rencontre et demande du prénom	44
III.17	Block 4 — Diagramme de séquence du quatrième bloc	45
III.18	Block 5.1 — Branche mémorielle : récupération et génération contextuelle	47
III.19	Block 5.2 — Branche générale : contexte conversationnel et génération de réponse	49
III.20	Block 5 — Vue complète des deux branches de traitement contextuel	50
III.21	Block 5 — Diagramme de séquence du cinquième bloc	50
III.22	Block 5 — Diagramme de classes du cinquième bloc	51
III.23	Block 6 — Sous-workflow n8n : test questiontype = note, <i>Basic LLM Chain 2</i> et structuration avant persistance	52
III.24	Block 6 — Entrée JSON : question, output, persone et questiontype = note	53
III.25	Block 6 — Sortie JSON : date_evenement et evenement extraits du message naturel	53
III.26	Block 7 — Diagramme de classes du septième bloc (persistance mémoire)	55
III.27	Vue complète du deuxième workflow	56
III.28	Vue des outputs du 2ème workflow	57
III.29	Vue complète du troisième workflow	58
III.30	Diagramme de use case du 1er cycle	60
IV.1	Interface Supabase utilisée pour l'administration de la base PostgreSQL	66
IV.2	Logo SupaBase	66
IV.3	Koda Schema Base De Donné	67
IV.4	Capture Azur Configuration	69
IV.5	Capture Test Text To Speech	69
IV.6	Capture Test Speech To Text	69
IV.7	Exemple de image dans la memoire	70
IV.8	Capture Input WorkFlow (N8N)	72
V.1	Conception 3D du châssis KODA sous Blender (vue éclatée des pièces)	80
V.2	Prototype physique KODA après impression 3D et montage	81
V.3	Principe de traction différentielle — moteurs M1 et M3	83
V.4	Enchaînement conceptuel : DOA (micro) → rotation du robot (M1, M3) → caméra (torse)	84
V.5	Schéma de montage matériel de KODA — style Fritzting (Cycle 3)	85

LISTE DES TABLEAUX

II.1	Vue synthétique des cycles et livrables attendus	15
II.2	Images et diagrammes associés au chapitre II (noms de fichiers cibles)	19
III.1	Correspondance entre mécanismes cognitifs humains et implémentations de KODA	59
IV.1	Comparaison des technologies backend envisagées	63
IV.2	Endpoints exposés à l'application mobile	73
IV.3	Problèmes techniques rencontrés et solutions apportées	75
V.1	Inventaire des composants matériels de KODA	79
V.2	Spécifications techniques du Raspberry Pi 4 Model B	82
V.3	Architecture d'alimentation de KODA	85
V.4	Récapitulatif des connexions entre composants	86
V.5	Tests unitaires par composant (Cycle 3)	87

INTRODUCTION GÉNÉRALE

Dans un monde en constante évolution technologique, la demande en solutions intelligentes capables d'améliorer la qualité de vie des individus n'a jamais été aussi forte. Les assistants virtuels et les systèmes embarqués intelligents représentent aujourd'hui une réponse concrète à ce besoin, en combinant les avancées de l'intelligence artificielle, des systèmes embarqués et du développement logiciel moderne.

Ce projet de fin d'études, réalisé dans le cadre de la formation en Développement des Systèmes Informatiques (DSI3.2) à l'ISET Mahdia, porte sur la conception et le développement de **KODA** : un assistant de vie et ami virtuel embarqué. KODA est un système robotique doté de sa propre personnalité et nature, capable de répondre intelligemment à toute personne, de raisonner à la manière d'un être humain, de reconnaître les individus à travers ses modes de sécurité, et d'être configuré et suivi via une application mobile cross-plateforme dédiée.

La problématique centrale de ce projet est : **Comment concevoir un système embarqué intelligent capable d'assurer un rôle d'assistant de vie personnalisé, intégrant reconnaissance des personnes, réflexion autonome et pilotage à distance via une application mobile ?**

Ce rapport est organisé comme suit :

- **Chapitre I** présente le cadre général du projet, l'organisme d'accueil, la problématique, l'analyse de l'existant et la solution proposée.
- **Chapitre II** expose la préparation du projet : spécification des besoins, identification des acteurs, méthodologie de pilotage *en spirale* par cycles, planification, architecture cible et environnements de travail.
- **Chapitre III** correspond au **Cycle 1 — Analyseur et réflexion** : workflows n8n (webhooks, blocs 1 à 7, intégration LLM et persistance).
- **Chapitre IV** correspond au **Cycle 2 — backend Distributeur de Services** (FastAPI, persistance, services cloud).
- **Chapitre V — Cycle 3** : système embarqué, partie matérielle (plateforme robot, câblage, validation).
- Les **cycles 4 à 6** (logiciel Raspberry Pi, application cross-plateforme, tests finaux) sont présentés dans la suite du document.

———— CHAPITRE I ————

CADRE GÉNÉRAL DU PROJET ET PRÉSENTATION DE L'ORGANISME

I.1 Introduction du chapitre

Ce chapitre esquisse le cadre fondamental de notre étude en définissant l'environnement contextuel et organisationnel du projet. Nous commençons par introduire l'organisme d'accueil, en mettant en évidence son expertise technologique et sa structure interne dédiée à l'innovation.

Ensuite, nous exposons la problématique centrale, justifiant le besoin critique d'un système intelligent. Une analyse rigoureuse des solutions existantes nous permet d'identifier les limites actuelles et de mieux positionner notre système proposé. Enfin, nous détaillons la méthodologie de projet adoptée, assurant une approche structurée et efficace tout au long du cycle de développement.

I.2 Contexte Général du Projet

L'évolution technologique actuelle transforme notre interaction avec l'environnement. La robotique sociale et les assistants personnels virtuels s'imposent comme des éléments clés pour améliorer la qualité de vie, offrant des solutions innovantes pour l'assistance quotidienne.

Le projet KODA s'inscrit dans cette dynamique en proposant un assistant de vie virtuel, conçu sous la forme d'un système embarqué intelligent. Contrairement aux solutions traditionnelles qui se limitent à l'exécution de commandes, KODA vise à offrir une véritable présence interactive, dotée d'une personnalité propre et d'une capacité de raisonnement.

La complexité de ce système réside dans l'intégration harmonieuse de multiples composants hétérogènes. Il nécessite la combinaison d'un environnement matériel embarqué (Raspberry Pi), d'un moteur de workflow intelligent (n8n) agissant comme cerveau, d'une couche logicielle orchestrant les communications (Python), et d'interfaces de contrôle intuitives (application mobile cross-plateforme). L'objectif est de créer un écosystème cohérent capable de reconnaître les utilisateurs, de traiter les requêtes de manière naturelle, et de fournir un accompagnement personnalisé.

I.3 Présentation de l'Organisme d'Accueil (MicroEdition)

I.3.1 Profil de l'Entreprise

MicroEdition est une entreprise pionnière spécialisée dans les Technologies de l'Information et de la Communication (TIC). Fondée en 2025, la société est stratégiquement implantée au sein de la Pépinière d'Entreprises de Mahdia. Cet environnement offre un écosystème robuste dédié à l'innovation, fournissant l'infrastructure nécessaire pour accompagner les startups technologiques à fort potentiel. MicroEdition se consacre à la conception et au déploiement de solutions numériques intelligentes, évolutives et sécurisées, visant à combler le fossé entre la recherche académique et

l'application industrielle.

I.3.2 Services

Le portefeuille de l'entreprise repose sur trois piliers technologiques fondamentaux :

- **Développement logiciel** : Conception d'applications web et mobiles haute performance, adaptées à des logiques métier spécifiques.
- **Internet des Objets (IoT)** : Conception d'écosystèmes connectés de bout en bout pour l'automatisation industrielle et agricole.
- **Cybersécurité** : Mise en œuvre de protocoles de sécurité rigoureux pour protéger les actifs numériques et garantir l'intégrité des données dans les secteurs public et privé.

I.3.3 Recherche & Développement (R&D)

Un trait distinctif de MicroEdition est sa division spécialisée en Recherche et Développement. Ce département se concentre fortement sur l'IA embarquée (Edge AI) et l'intelligence embarquée. En explorant des méthodologies d'exécution de réseaux de neurones complexes directement sur des microcontrôleurs, l'équipe R&D cherche à éliminer la dépendance à une connectivité cloud permanente. Cet axe stratégique est central pour le projet KODA, car il permet la création de systèmes autonomes et résilients capables de fonctionner dans des environnements où l'accès à Internet peut être intermittent.

I.3.4 Structure Organisationnelle

MicroEdition fonctionne selon un cadre Agile qui met l'accent sur l'itération rapide et la collaboration interdisciplinaire. La structure interne de l'entreprise est composée d'unités spécialisées, notamment :

- **Architectes logiciels** : Conception de backends systèmes évolutifs.
- **Ingénieurs en systèmes embarqués** : Pont entre le matériel et le logiciel.
- **Spécialistes en IA et Data Scientists** : Développement et optimisation de modèles d'apprentissage automatique pour le déploiement embarqué.

Cette synergie permet à MicroEdition de maintenir un cycle d'innovation soutenu, garantissant que chaque projet — tel que KODA — repose sur des fondations d'excellence technique et de standards architecturaux modernes.

I.4 Problématique

L'évolution constante de notre société moderne, marquée par un rythme de vie accéléré et une digitalisation croissante, a paradoxalement accentué certains besoins fondamentaux de l'être humain, notamment celui de l'accompagnement et de l'interaction sociale.

Le problème central auquel nous tentons de répondre s'articule autour de trois axes principaux :

A. Le besoin de présence et d'interaction naturelle

Malgré la prolifération des appareils connectés, la plupart des outils actuels (smartphones, enceintes connectées classiques) restent des outils purement fonctionnels. Ils exécutent des ordres mais n'offrent pas de réelle interaction "humaine". Il existe un manque flagrant de systèmes capables d'afficher une personnalité propre et de simuler une forme de réflexion empathique capable de briser la solitude ou d'assister l'utilisateur au quotidien.

B. La complexité de la gestion de vie autonome

Dans un environnement domestique, la gestion des tâches, de la sécurité et du suivi technique des objets connectés devient de plus en plus complexe pour l'utilisateur moyen. Il n'existe pas de solution centralisée qui combine à la fois un assistant de vie intelligent (capable de raisonner) et un système de sécurité actif (capable de reconnaître les personnes et de protéger l'habitat).

C. Les limites de l'intelligence statique

La majorité des assistants actuels dépendent de réponses préprogrammées et rigides. Le défi est de passer d'une machine qui "répond" à une machine qui "réfléchit" et "apprend". Le problème est donc de concevoir une architecture capable de :

- Identifier l'interlocuteur de manière sécurisée.
- Traiter des demandes complexes de manière fluide et humaine.
- Offrir une interface de contrôle simple (application mobile) pour superviser cette "vie artificielle".

En résumé, la question fondamentale de notre projet est la suivante : **Comment concevoir un compagnon robotique intelligent, doté d'une mémoire et d'une personnalité, capable d'offrir un accompagnement humain authentique tout en assurant une gestion technique et sécurisée de l'environnement de l'utilisateur ?**

I.5 Analyse des Solutions Existantes

I.5.1 Étude de l'existant

Dans cette section, nous examinons les assistants domestiques et les robots compagnons les plus populaires sur le marché afin d'identifier leurs forces et leurs lacunes.

- **A. Les Assistants Vocaux Statiques (Amazon Alexa, Google Home)**

Ces dispositifs sont les plus répandus. Ils excellent dans l'exécution de tâches simples (domotique, rappels, recherches internet).

Limites : Ils n'ont pas de présence physique mobile. Surtout, ils restent des outils purement réactifs : ils attendent un "mot déclencheur" et répondent de manière standardisée. Il n'y a aucune notion de "personnalité" évolutive ou de sentiment d'appartenance.

- **B. Les Robots Domestiques (ex : Amazon Astro, Vector/Cozmo)**

Certains robots essaient d'occuper l'espace physique. Amazon Astro, par exemple, surveille la maison et suit l'utilisateur.

Limites : Bien que mobiles, ces produits restent perçus comme des gadgets technologiques. Leur interaction est limitée par des scripts pré-enregistrés. Ils ne "réfléchissent" pas de manière humaine et leur capacité à reconnaître et s'adapter à l'humeur ou à l'identité spécifique de chaque membre de la famille de façon approfondie reste superficielle.

- **C. Les Pepper & Nao (Robotique de service)**

Ce sont des robots haut de gamme capables de simuler des émotions et de reconnaître des visages.

Limites : Ces solutions sont extrêmement coûteuses (plusieurs milliers d'euros) et sont destinées aux entreprises plutôt qu'au grand public. Leur configuration est complexe et ils ne sont pas conçus comme des "amis" personnels intégrés à un écosystème mobile fluide pour l'utilisateur lambda.

I.5.2 Synthèse et Critique : Le "Manque d'Humanité"

Le point commun de tous ces concurrents est leur nature générique. Ce sont des machines produites en série qui traitent chaque utilisateur de la même manière.

- **Absence de personnalité** : La réponse d'une Alexa est la même pour tout le monde. Elle n'a pas de "caractère" propre qui pourrait la faire passer pour un compagnon de vie.
- **Froideur algorithmique** : Les interactions sont basées sur des arbres de décision rigides. L'utilisateur sent qu'il parle à une base de données, pas à une entité qui "réfléchit".

- **Dépendance au Cloud propriétaire** : L'utilisateur a peu de contrôle sur la "mémoire" ou la logique interne de l'assistant.

I.5.3 La Différenciation de Koda

C'est ici que notre projet se distingue. Contrairement aux machines industrielles, Koda est conçu pour être :

- **Unique** : Grâce à l'intégration de n8n (le cœur de réflexion), Koda peut avoir une logique de réponse personnalisée et évolutive.
- **Multimodal** : Il combine la vision (Raspberry Pi), la réflexion (IA via n8n) et la mobilité.
- **Accessible** : Une application mobile dédiée permet de configurer sa "personnalité", rendant l'utilisateur maître de son compagnon.

I.6 Solution Proposée

Pour surmonter ces limitations, nous proposons KODA, une architecture résiliente et multi-couche conçue pour l'autonomie et l'intelligence. La solution comprend :

- **Contrôleur Embarqué (Raspberry Pi)** : Le cœur physique du système, gérant les composants matériels et assurant le fonctionnement du robot.
- **Moteur de Réflexion (n8n)** : Le cerveau de KODA, responsable de la préparation des réponses et du traitement intelligent des requêtes complexes.
- **Couche de Distribution (Python)** : Le système nerveux responsable de la communication entre n8n, la base de données, l'application mobile et le Raspberry Pi, offrant les points d'accès (endpoints) de tout l'écosystème.
- **Expérience Utilisateur Mobile** : Une application cross-plateforme permettant à l'utilisateur de configurer, contrôler et suivre la vie de son compagnon robotique en temps réel.
- **Mémoire Persistante (Base de données)** : Un système de stockage permettant à KODA d'enregistrer des informations contextuelles et d'apprendre de ses interactions.

En intégrant ces composants, la solution proposée garantit une résilience opérationnelle, des interactions humaines authentiques et une expérience de gestion unifiée.

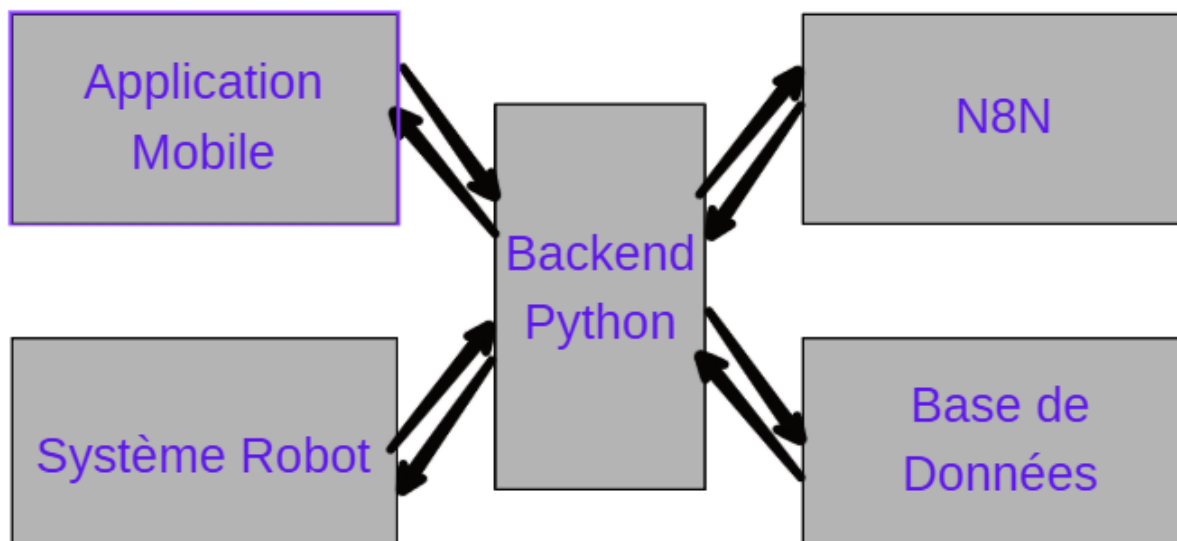


FIGURE I.1 – Architecture Globale de la Solution Proposée

I.7 Méthodologie du Projet

Pour garantir un haut degré de fiabilité et une intégration harmonieuse des différentes couches logicielles et matérielles, ce projet suit un cadre d'ingénierie rigoureux.

I.7.1 Approche Agile et Cycle en V

Bien que le projet intègre des concepts de prototypage rapide caractéristiques des méthodes Agiles, l'architecture globale repose sur un Cycle en V. Ce modèle représente un chemin d'exécution séquentiel où chaque phase de développement (côté gauche) est associée à une phase de test correspondante (côté droit).

Nous avons adopté cette méthodologie car elle garantit que les protocoles de test pour notre matériel embarqué et nos workflows d'intelligence artificielle soient définis avant l'assemblage physique, conduisant à un cycle d'intégration plus efficace. La figure ?? illustre les étapes fondamentales de ce cadre dans un contexte d'ingénierie général.

1) Le flux de processus : Déconstruction et Réassemblage

La méthodologie forme une forme en "V" symétrique, divisée en trois segments principaux :

- **Le segment de vérification (Étapes 1–4) :** Se concentre sur la traduction des besoins des utilisateurs en un plan architectural définitif.
- **Le sommet (Étape 5) :** L'implémentation est le tournant critique où le codage, l'assemblage matériel et la configuration des workflows ont lieu.
- **Le segment de validation (Étapes 6–8) :** Réassemble et vérifie le système à travers des tests

unitaires et des tests d'intégration globaux.

2) Artefacts Clés du Projet

La traçabilité est maintenue en générant des artefacts spécifiques à chaque étape :

- **Spécifications des Besoins** : Besoins utilisateurs et techniques documentés.
- **Conception de l'Architecture Système** : Plans d'intégration matériel/logiciel.
- **Conception Détaillée des Composants** : Logique de code, endpoints API et schéma de base de données.

I.8 Conclusion

Ce chapitre a établi le contexte du projet KODA. En identifiant les lacunes critiques des assistants virtuels actuels — notamment le manque de personnalité, de réflexion autonome et d'intégration embarquée — nous avons défini notre solution innovante. De plus, en adoptant une méthodologie de développement structurée, nous avons établi une feuille de route qui garantit la fiabilité tout au long des défis techniques à venir.

Le chapitre suivant, «**Préparation du Projet**», formalise les besoins, la méthodologie de développement **en spirale** par cycles successifs et les environnements techniques retenus pour mener à bien la réalisation de KODA.

_____ CHAPITRE II _____

PRÉPARATION DU PROJET

II.1 Introduction du chapitre

Ce chapitre pose les fondations ingénierie du projet **KODA** avant d’entrer dans le détail technique des cycles de réalisation. Nous y traduisons l’intention du chapitre I en **exigences traçables** (fonctionnelles et non fonctionnelles), en **vision architecturale** et en **périmètre d’acteurs**. Nous y présentons également la **méthodologie de pilotage** retenue : le **modèle en spirale**, fondé sur des **cycles de développement successifs** — et non sur une succession de sprints Scrum — où chaque tour de spirale livre un incrément (couche ou sous-système) intégrable et réévalue les risques avant le tour suivant. Enfin, nous documentons les environnements matériel et logiciel qui ont encadré la production du prototype.

II.2 Spécification des besoins

II.2.1 Spécifications des besoins fonctionnels

Les besoins fonctionnels décrivent *ce que* le système doit faire du point de vue des utilisateurs et des scénarios d’usage, en cohérence avec la solution esquissée au chapitre I (n8n, backend Python, Raspberry Pi, application mobile, base PostgreSQL).

- **Interaction conversationnelle** : réception d’un message utilisateur, traitement par workflows n8n (webhooks), production d’une réponse contextualisée et routage selon l’intention (fermeture, authentification, musique, direction, classification mémoire, etc.).
- **Mémoire et personnalisation** : distinction entre réponses nécessitant une consultation de profil (USE_MEMORY) et réponses autonomes (NO_MEMORY) ; enregistrement de présentations, notes et informations personnelles ; consolidation périodique (webhook nocturne) telle que décrite au cycle 1.
- **Distribution des services** : API REST (FastAPI) reliant la base de données (Supabase / PostgreSQL), n8n, le robot et l’application mobile ; endpoints pour commandes, sessions, événements et services cognitifs (STT, TTS, reconnaissance faciale, modèles de langage).
- **Couche embarquée** : exécution sur Raspberry Pi des fonctions temps réel attendues (captures, dialogue avec le backend, signaux vers actionneurs ou périphériques selon le cahier des charges physique).
- **Pilotage mobile** : application cross-plateforme de configuration, de suivi et de supervision du compagnon.
- **Sécurité et modes de fonctionnement** : gestion d’états (ouvert / fermé), mot de passe, filtrage

des intentions sensibles conformément au premier flux n8n.

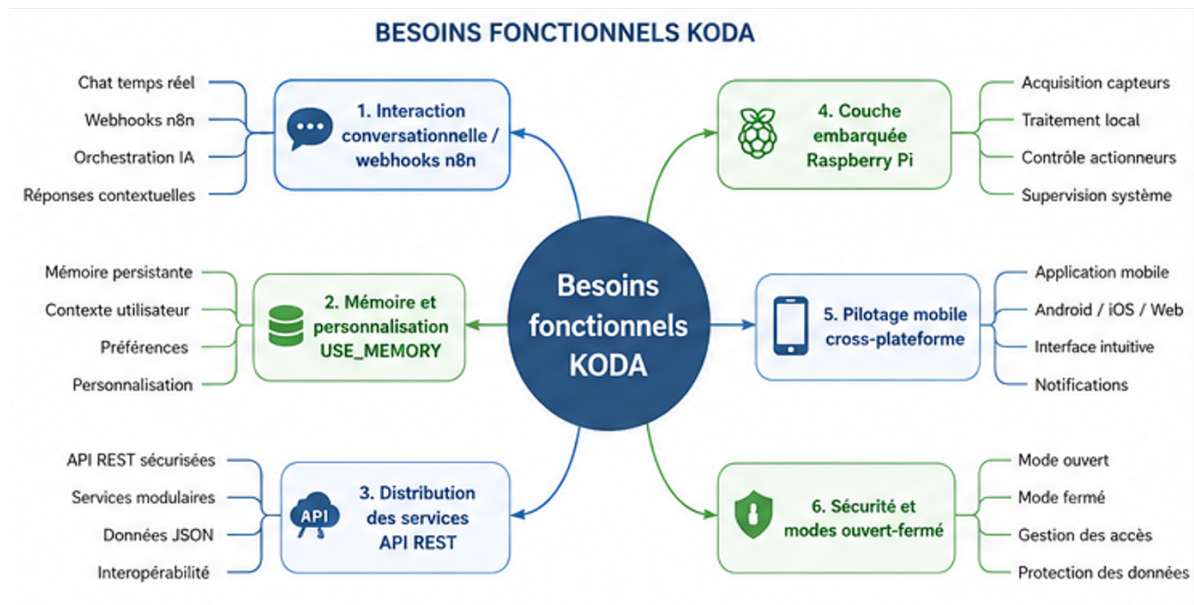


FIGURE II.1 – Synthèse visuelle des besoins fonctionnels KODA (chapitre II)

II.2.2 Spécification des besoins non fonctionnels

Les besoins non fonctionnels encadrent la **qualité** du système.

- **Performance et latence** : réponses utilisateur acceptables malgré les appels externes (OpenRouter, SerpAPI, Azure) ; optimisation par cache conversationnel (bloc historique du Webhook 1).
- **Disponibilité et résilience** : tolérance aux indisponibilités partielles des services cloud ; persistance des données critiques en PostgreSQL.
- **Sécurité** : authentification aux API, gestion des secrets (clés OpenRouter, SerpAPI, Supabase), moindre exposition des surfaces d'attaque sur le robot et le backend.
- **Maintenabilité** : séparation des couches (Clean Architecture côté backend), workflows n8n modulaires par blocs, documentation des webhooks.
- **Interopérabilité** : formats JSON REST, connecteurs standards (PostgreSQL, SDK officiels).
- **Évolutivité** : possibilité d'ajouter des nœuds n8n, des endpoints ou des écrans mobiles sans remettre en cause l'ensemble de l'**architecture cible**.

II.2.3 Architecture du système

L'architecture cible est **multicouche** et **distribuée** : un moteur d'automatisation (**n8n**) concentre la logique métier conversationnelle ; un **backend Python (FastAPI)** centralise les accès données et les intégrations cloud ; un **noyau embarqué (Raspberry Pi)** assure le lien avec le monde physique ;

une **application mobile** prolonge l'expérience utilisateur; une **base PostgreSQL** matérialise la mémoire long terme. La figure I.1 du chapitre I illustre cette décomposition; les cycles de développement du présent rapport s'alignent sur ces blocs pour réduire le rayon d'intégration à chaque itération.

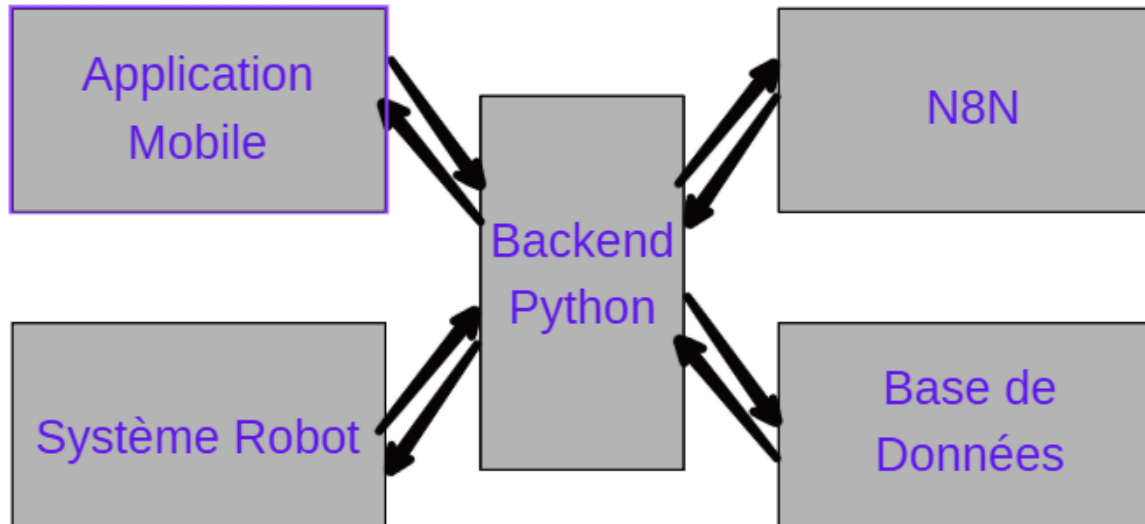


FIGURE II.2 – Architecture multicouche du système KODA (vue chapitre II)

II.2.4 Identification des acteurs

On distingue notamment :

- l'**Utilisateur final** (personne qui dialogue avec KODA et configure l'application);
- le **système KODA** (ensemble logiciel/matériel);
- le **Distributeur de services** (backend) en tant qu'intermédiaire technique;
- les **services externes** (Supabase, fournisseurs LLM via OpenRouter, SerpAPI pour la recherche structurée, Azure pour la voix et la vision);
- l'**organisme d'accueil** et l'**équipe projet** (rôles de pilotage et de validation).

II.3 Pilotage du projet — Modèle en spirale et cycles de développement

Contrairement à une approche **Agile Scrum** fondée sur des *sprints* de durée fixe avec cérémonies prescrites (daily, review, rétrospective), nous avons retenu le **modèle en spirale** : à chaque itération, une portion du produit (cycle thématique : workflows n8n, backend, matériel, logiciel embarqué, mobile, validation) est **spécifiée, réalisée** puis **intégrée** au système global, avec une **réévaluation des risques** et des priorités avant le tour suivant. Chaque cycle produit un incrément **testable** et raccordé aux livrables des tours précédents, jusqu'à stabiliser l'écosystème KODA complet.

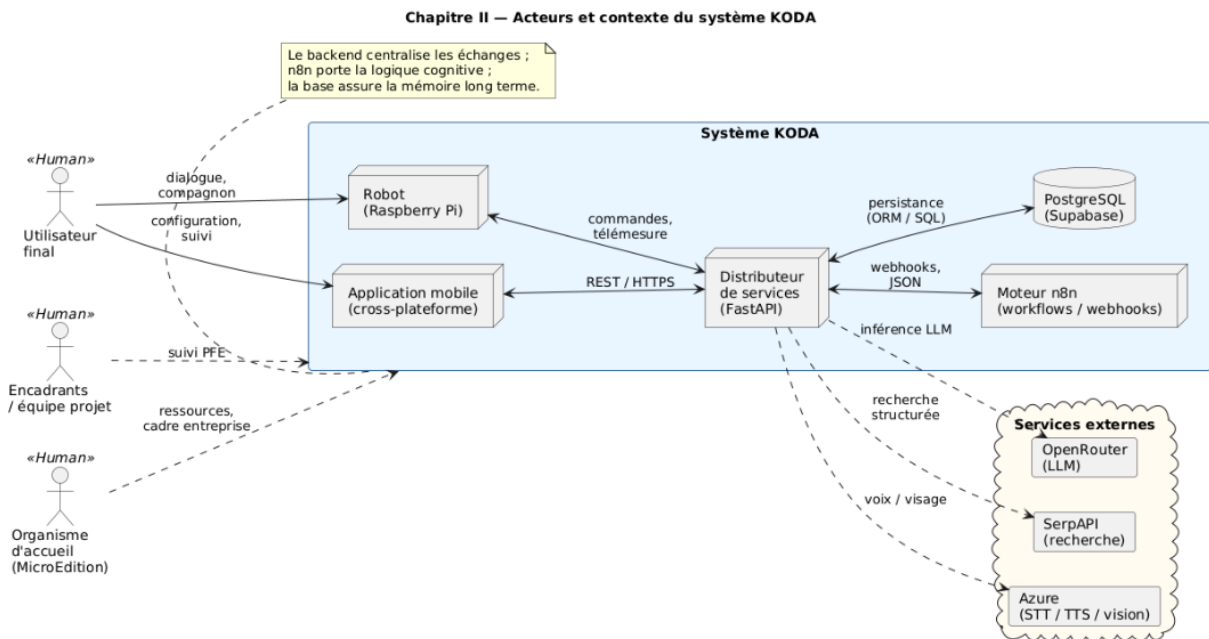


FIGURE II.3 – Acteurs principaux et contexte d'intégration du système KODA

II.3.1 Équipe et rôles

Le projet est conduit par une **équipe de projet de fin d'études** (DSI3.2, ISET Mahdia), en binôme, sous la responsabilité d'un **encadrant académique** et en lien avec l'**organisme d'accueil** MicroEdition (cf. chapitre I). Les rôles se répartissent entre conception des workflows n8n, développement backend, intégration embarquée, développement mobile et campagne de tests, avec des points de synchronisation à la fin de chaque cycle pour valider l'intégration incrémentale.

II.3.2 Découpage du travail : six cycles

Le périmètre fonctionnel est découpé en six cycles **ordonnés logiquement** — sans équivalence directe avec des sprints Scrum — comme suit :

1. **Cycle 1 — Workflow et n8n** : conception des webhooks, blocs de traitement (classification d'intention, analyseur de dépendance mémoire, consolidation, proactivité), intégration des API LLM et recherche.
2. **Cycle 2 — Backend** : Distributeur de services (FastAPI), accès PostgreSQL / Supabase, Clean Architecture, endpoints pour le robot et le mobile.
3. **Cycle 3 — Système embarqué, partie matérielle** : choix et câblage des capteurs, actionneurs, alimentation, mécanique ou support physique associé au Raspberry Pi selon le prototype.
4. **Cycle 4 — Système embarqué, partie logicielle (Raspberry Pi)** : logiciel embarqué, communication avec le backend, gestion des flux audio/vidéo locaux, robustesse sur carte ARM.
5. **Cycle 5 — Cross-plateforme** : application mobile (configuration, suivi, notifications), parcours

utilisateur unifiés Android/iOS.

6. **Cycle 6 — Tests et validation** : tests unitaires et d'intégration, scénarios bout-en-bout, validation des besoins non fonctionnels (sécurité, performance).

TABLE II.1 – Vue synthétique des cycles et livrables attendus

Cycle	Thème	Livrable principal
1	Workflow n8n	Webhooks opérationnels, prompts et routages stabilisés
2	Backend	API documentée, persistance, intégrations cloud
3	Embarqué HW	Plateforme physique prête à accueillir le logiciel
4	Embarqué SW (Pi)	Services locaux et dialogue avec le Distributeur
5	Cross-plateforme	Application mobile utilisable sur les cibles
6	Tests & validation	Rapport de recette, correctifs de fin de projet

II.4 Planification des cycles

La planification ne s'exprime pas en **sprints** de deux semaines mais en **jalons de cycles** : chaque cycle se termine par une **démonstration intégrée** (workflow branché sur le backend, backend branché sur la base, puis ajout du Raspberry Pi, etc.). Les dépendances imposent un ordre raisonnable : le moteur n8n (cycle 1) et le backend (cycle 2) peuvent avancer en parallèle partiel, mais la validation bout-en-bout (cycle 6) recouvre l'ensemble. Le calendrier académique du PFE (nombre de semaines, séances de suivi, livrables intermédiaires) structure la cadence réelle ; le **modèle en spirale** en garantit la **cohérence** : aucun cycle ne clôt sans critères d'acceptation liés aux chapitres techniques correspondants du rapport.

II.5 Architecture

II.5.1 Architecture de l'application

Au-delà du schéma global (chapitre I), l'architecture applicative distingue : **(a)** la couche *présentation* (mobile) ; **(b)** la couche *orchestration intelligente* (n8n) ; **(c)** la couche *services* (FastAPI) ; **(d)** la couche *données* (PostgreSQL) ; **(e)** la couche *embarquée* (Raspberry Pi). Les communications principales suivent le modèle requête-réponse REST et événements asynchrones selon les besoins (temps réel, webhooks). Cette structuration facilite le parallélisme des cycles tout en conservant des interfaces stables entre équipes.

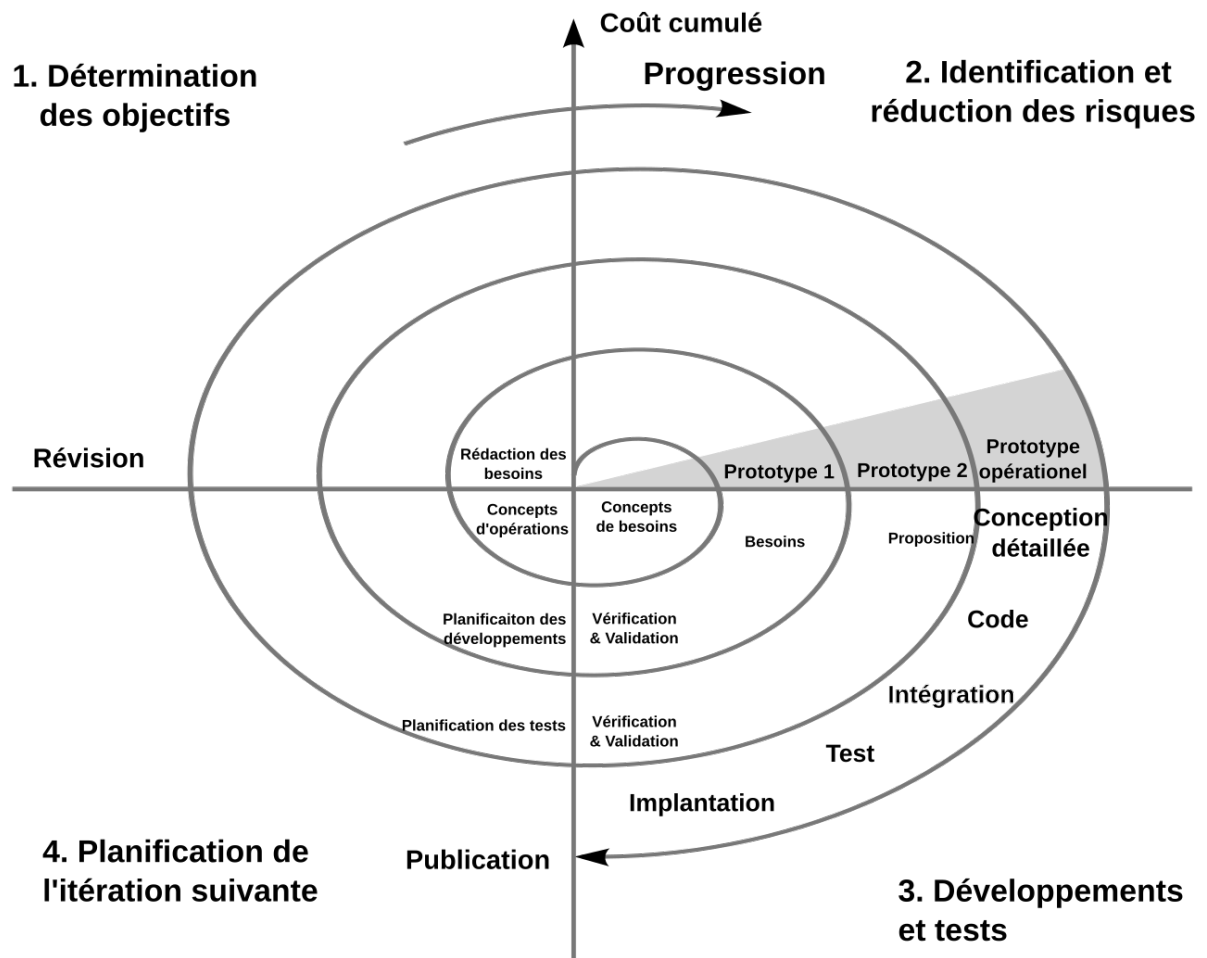


FIGURE II.4 – Modèle en spirale et enchaînement des six cycles de développement KODA

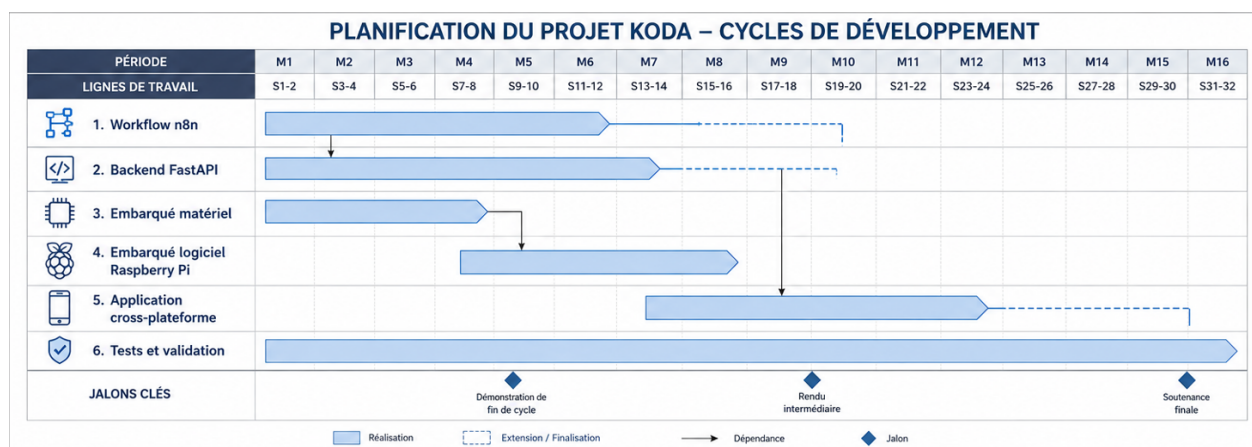


FIGURE II.5 – Planification des cycles : jalons, chevauchements et dépendances

II.5.1.1 Architecture en oignon (*Onion Architecture*) du backend

Le **Distributeur de services** (backend Python, cycle 2) adopte une **architecture en oignon** : les couches sont organisées en **anneaux concentriques**, du plus stable au plus volatile. Au **cœur** se trouvent le **domaine métier** (entités, règles) ; autour, les **cas d’usage** orchestrent les scénarios sans dépendre des frameworks ; puis viennent les **adaptateurs** (API REST, sérialisation, accès données) ; en périphérie, la **infrastructure** (FastAPI, Supabase, SDK Azure, clients HTTP vers n8n ou le robot). Les dépendances pointent **toujours vers l’intérieur**, ce qui isole la logique métier des technologies et facilite les tests unitaires sur le domaine.

Ce modèle — popularisé notamment par Jeffrey Palermo sous le nom d’*Onion Architecture* — recoupe les principes de la **Clean Architecture** détaillés au chapitre du cycle 2 : les deux visions décrivent des couches concentriques et la règle d’inversion des dépendances ; l’« oignon » met l’accent sur la **testabilité** du noyau et sur le fait que l’infrastructure soit interchangeable sans toucher au cœur du système. Ainsi, le backend KODA reste cohérent avec la vision multicouche de la figure I.1 tout en précisant sa structure interne.

II.6 Environnement de travail

II.6.1 Environnement matériel

Le développement s’appuie sur des postes de travail type PC (processeur multi-cœurs, au moins 8 Go de RAM recommandés pour les outils de conteneurisation et les IDE), un **Raspberry Pi** cible pour les tests embarqués, périphériques audio/vidéo selon les besoins du robot, et un smartphone ou émulateur pour la couche mobile. Un accès Internet stable est nécessaire pour Supabase, OpenRouter, SerpAPI et Azure durant l’intégration.

Emplacement réservé — photographie à insérer sous le nom
public/ch2-environnement-materiel.png (PC, Raspberry Pi, périphériques du
robot).

II.7 Environnement logiciel

II.7.1 Outils de développement et modélisation

- **n8n** (self-hosted ou cloud) pour l’édition des workflows ;
- **Visual Studio Code** ou équivalent pour Python, Dart et LaTeX ;

- **Git** pour le versioning ;
- **PlantUML** / captures d'écran pour les diagrammes de séquence et de cas d'utilisation ;
- **Supabase Studio** pour l'administration PostgreSQL ;
- **Docker** (le cas échéant) pour le déploiement du backend et la reproductibilité des environnements.

II.7.2 Langages et frameworks utilisés

- **JavaScript** (expressions n8n, nœuds Code) ;
- **Python 3** avec **FastAPI**, **SQLAlchemy**, **asyncpg**, **SDK Azure** et intégrations **Gemini/OpenRouter** côté Distributeur ;
- **SQL** (PostgreSQL) ;
- **Dart / Flutter** pour l'application cross-plateforme ;
- **Bash** et utilitaires système sur Raspberry Pi OS.

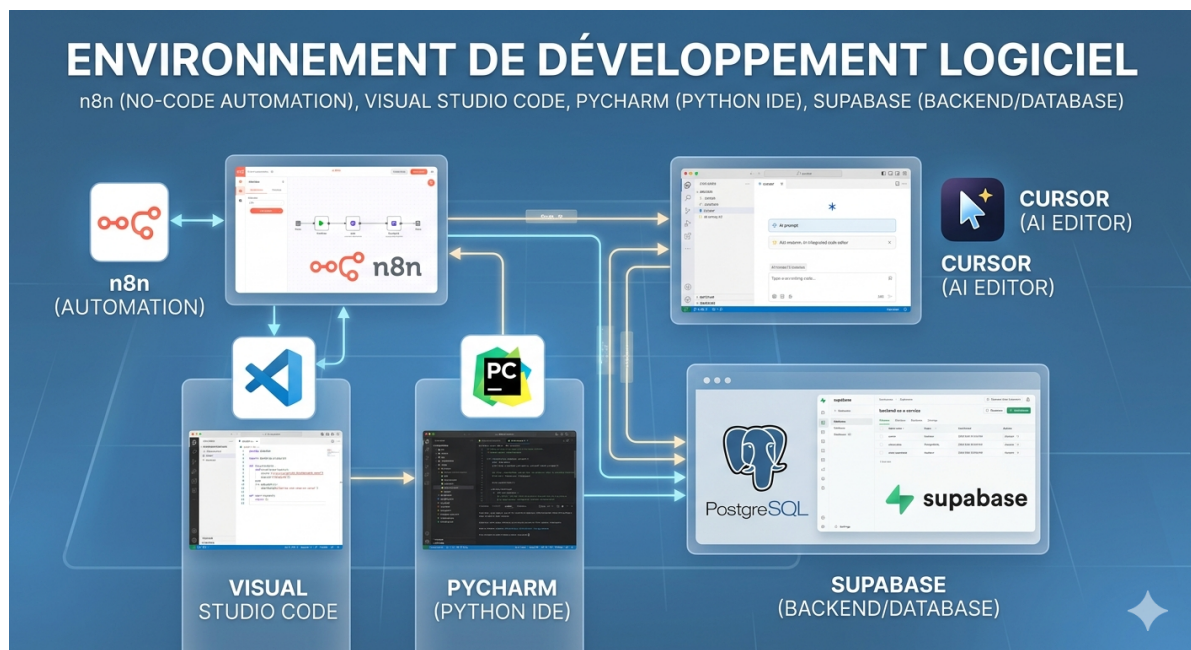


FIGURE II.6 – Environnement logiciel : outils de développement et chaîne technique (chapitre II)

II.8 Figures et diagrammes prévus pour ce chapitre

Pour illustrer le chapitre II dans le PDF final, les captures et exports suivants sont recommandés. Les noms de fichiers sont normalisés **sans espace** (dossier public/) afin d'éviter les erreurs de compilation $\text{\LaTeX}$; il suffit de créer chaque fichier sous ce nom puis d'insérer `\includegraphics` dans la section concernée.

TABLE II.2 – Images et diagrammes associés au chapitre II (noms de fichiers cibles)

Fichier (public/...)	Section cible	Contenu attendu
ch2-modele-spirale.png	Pilotage (modèle en spirale)	Schéma spirale : tours successifs avec les six cycles KODA ($n8n \rightarrow$ validation).
ch2-planification-cycles.png	Planification des cycles	Frise chronologique ou diagramme de Gantt (jalons PFE, dépendances entre cycles).
ch2-architecture-systeme.png	Architecture du système / Architecture de l'application	Vue multicouche (mobile, $n8n$, FastAPI, PostgreSQL, Pi) ; peut être une version retravaillée de l'architecture globale.
ch2-onion-backend.png	Architecture de l'application (oignon)	Schéma des anneaux : domaine, cas d'usage, adaptateurs, infrastructure (Distributeur).
ch2-diagramme-acteurs.png	Identification des acteurs	Diagramme de cas d'utilisation simplifié ou diagramme de contexte (acteurs externes).
ch2-environnement-materiel.png	Environnement matériel	Photo ou montage : PC de développement, Raspberry Pi, périphériques du robot.
ch2-environnement-logiciel.png	Environnement logiciel	Captures (IDE, $n8n$, Supabase, terminal Docker/Git) regroupées sur une planche.
ch2-besoins-fonctionnels.png	Spécification fonctionnelle (optionnel)	Tableau ou mind map des exigences F (lisible à l'impression).

Réutilisation possible : la figure d'architecture multicouche du chapitre I (`public/ch2-architecture-syste`) peut être réutilisée ou déclinée pour d'autres sections après recadrage ou légende adaptée au § 2.2.3 / 2.5. Le **cycle en V** évoqué au chapitre I complète la spirale en rappelant la symétrie analyse / tests tout au long du projet.

II.9 Conclusion

Ce chapitre a cadré la préparation de KODA : besoins fonctionnels et non fonctionnels, acteurs, architecture de référence, environnements, et surtout un **modèle en spirale** articulé en six cycles — en rupture avec une gestion purement Scrum/sprint, tout en conservant l'esprit d'incrémentalité et de revue de fin de cycle. Le chapitre suivant détaille le **Cycle 1 (Analyseur et réflexion)** : workflows n8n.

_____ CHAPITRE III _____

CYCLE 1 : ANALYSEUR ET RÉFLEXION

III.1 Introduction

Ce chapitre présente le moteur central de KODA : son système d'analyse, de réflexion et de génération de réponses. L'objectif principal est de permettre au robot de répondre de manière pertinente, contextualisée et naturelle aux questions posées par les utilisateurs. KODA étant conçu comme un accompagnateur virtuel à dimension humaine, il doit comprendre non seulement le contenu des questions, mais aussi le contexte et les besoins implicites de chaque utilisateur.

Pour atteindre cet objectif, nous avons conçu une architecture logicielle organisée autour de trois flux de travail automatisés (appelés **webhooks**), chacun remplissant une fonction précise et complémentaire. Avant de détailler leur fonctionnement, nous commençons par définir les technologies fondamentales sur lesquelles repose l'ensemble du système : la plateforme **n8n**, les **webhooks**, puis les briques d'**intelligence artificielle conversationnelle** — le **LLM** (*Large Language Model*) et le service d'agrégation **OpenRouter** — utilisées notamment au **Block 3** pour la classification sémantique.

III.2 Définitions Fondamentales

III.2.1 n8n

n8n (prononcé "n-eight-n", abréviation de *nodemation*) est une **plateforme d'automatisation de flux de travail open-source**, conçue pour connecter des applications, des APIs et des services entre eux sans nécessiter de développement complexe.

Techniquement, n8n fonctionne selon une architecture orientée **nœuds** (nodes) : chaque nœud représente une opération atomique — un appel à une API, une transformation de données, une condition logique, ou l'invocation d'un modèle d'intelligence artificielle. Ces nœuds sont reliés entre eux pour former des **flux de traitement** (workflows), dans lesquels les données circulent sous forme d'objets JSON structurés d'un nœud à l'autre.

Voici les caractéristiques essentielles qui ont motivé notre choix de n8n pour le projet KODA :

- **Architecture DAG (Graphe Orienté Acyclique)** : Les nœuds sont organisés en graphe sans cycle, garantissant un traitement ordonné et prévisible des données.
- **Auto-hébergement** : Contrairement à des outils comme Zapier ou Make, n8n peut être installé sur un serveur privé. Cela assure un contrôle total sur les données des utilisateurs — un critère indispensable pour un projet traitant des informations personnelles.
- **Intégration native avec les modèles IA** : n8n propose des connecteurs prêts à l'emploi pour les principaux modèles de langage (OpenAI, Anthropic Claude, Mistral), permettant d'insérer

des nœuds d'intelligence artificielle directement dans le flux de traitement.

- **Interface visuelle** : Le concepteur de workflows est entièrement graphique, permettant de visualiser, tester et déboguer chaque étape du traitement en temps réel.
- **Déclencheurs multiples** : n8n supporte des dizaines de modes de déclenchement — webhooks HTTP, planification cron, événements base de données, etc.

Référence : Documentation officielle n8n — <https://docs.n8n.io>

III.2.2 Webhook

Un **webhook** est un mécanisme de communication entre deux systèmes informatiques basé sur le modèle **événement** → **notification** → **action**. Contrairement à une requête classique où c'est le client qui interroge régulièrement un serveur pour savoir si quelque chose a changé (modèle *polling*), un webhook fonctionne à l'inverse : c'est le serveur émetteur qui notifie automatiquement le serveur récepteur dès qu'un événement se produit.

Concrètement, un webhook est une **URL HTTP unique** exposée par le système récepteur (ici, n8n). Lorsqu'un événement survient dans le système émetteur (par exemple : l'utilisateur pose une question au robot), celui-ci envoie une requête HTTP POST vers cette URL, avec les données de l'événement dans le corps de la requête. Le serveur récepteur traite alors ces données et déclenche le pipeline de traitement associé.

Concept	Signification
URL du webhook	Adresse unique qui reçoit les notifications
Déclencheur	L'événement qui provoque l'envoi
Payload	Les données transmises (format JSON)
Pipeline	L'ensemble des traitements activés

Dans KODA, nous utilisons trois webhooks distincts, chacun dédié à une fonction cognitive précise du robot.

Référence : Mozilla Developer Network — <https://developer.mozilla.org/en-US/docs/Glossary/W>

III.2.3 Grand modèle de langage (LLM)

Un **grand modèle de langage** — en anglais *Large Language Model*, souvent abrégé **LLM** — est un modèle de **apprentissage profond** entraîné sur d'immenses corpus textuels pour prédire la suite probable de tokens (mots ou sous-mots). Il sait produire du texte fluide, suivre des consignes (*prompts*), classer ou reformuler des entrées, et combiner plusieurs contraintes lorsque le prompt est bien structuré.

Dans n8n, un LLM est typiquement invoqué via des nœuds de type **Basic LLM Chain** ou **AI Agent** : le workflow transmet un prompt système et le message utilisateur ; le modèle renvoie une **complétion textuelle** exploitable par les nœuds aval (parsing JSON, routage, réponse au webhook). Au **Block 3** du premier flux, le LLM joue le rôle d'**Analyseur de Dépendance Mémoire** : il décide si une consultation de la base est nécessaire et assigne une catégorie, sous une forme de sortie strictement contrôlée (lignes `USE_MEMORY` / `NO_MEMORY`, etc.).

Référence : Russell & Norvig (2010), *Artificial Intelligence : A Modern Approach* ; documentation modèles c

III.2.4 OpenRouter

OpenRouter est un **agrégateur d'API** qui expose, derrière une **même interface HTTP** et un jeu d'identifiants (*API key*), un **catalogue de modèles** provenant de plusieurs fournisseurs (Google Gemini, Anthropic, OpenAI, Meta, etc.). L'intérêt pour un projet comme KODA est triple :

- **Simplifier l'intégration** : un seul type de credential « compte OpenRouter » dans n8n au lieu de multiplier les comptes propriétaires ;
- **Changer de modèle** en ajustant simplement l'identifiant de modèle (ex. `google/gemini-2.5-flash-lite`) sans réécrire tout le workflow ;
- **Comparer rapidement** des modèles (latence, coût, qualité) pour les tâches de classification ou de génération.

Techniquement, le nœud n8n **OpenRouter Chat Model** envoie la conversation au point de terminaison OpenRouter ; la réponse du modèle sélectionné est renvoyée à la **Basic LLM Chain** du Block 3. OpenRouter n'est pas un LLM en soi : c'est la **couche d'accès** au LLM choisi, comparable à un *routeur* entre votre workflow et le fournisseur d'inférence réel.

Référence : OpenRouter — <https://openrouter.ai/docs>

III.2.5 Schéma de classes — persistance KODA

Les workflows n8n s'appuient sur PostgreSQL pour relier utilisateurs, configuration (setting, clé primaire `idkoda` référencée par `utilisateur.idkoda`) et données de conversation ou d'accompagnement. La figure III.1 synthétise le modèle conceptuel associé au **Block 7** (mémoire et persistance) dans le Cycle 1.

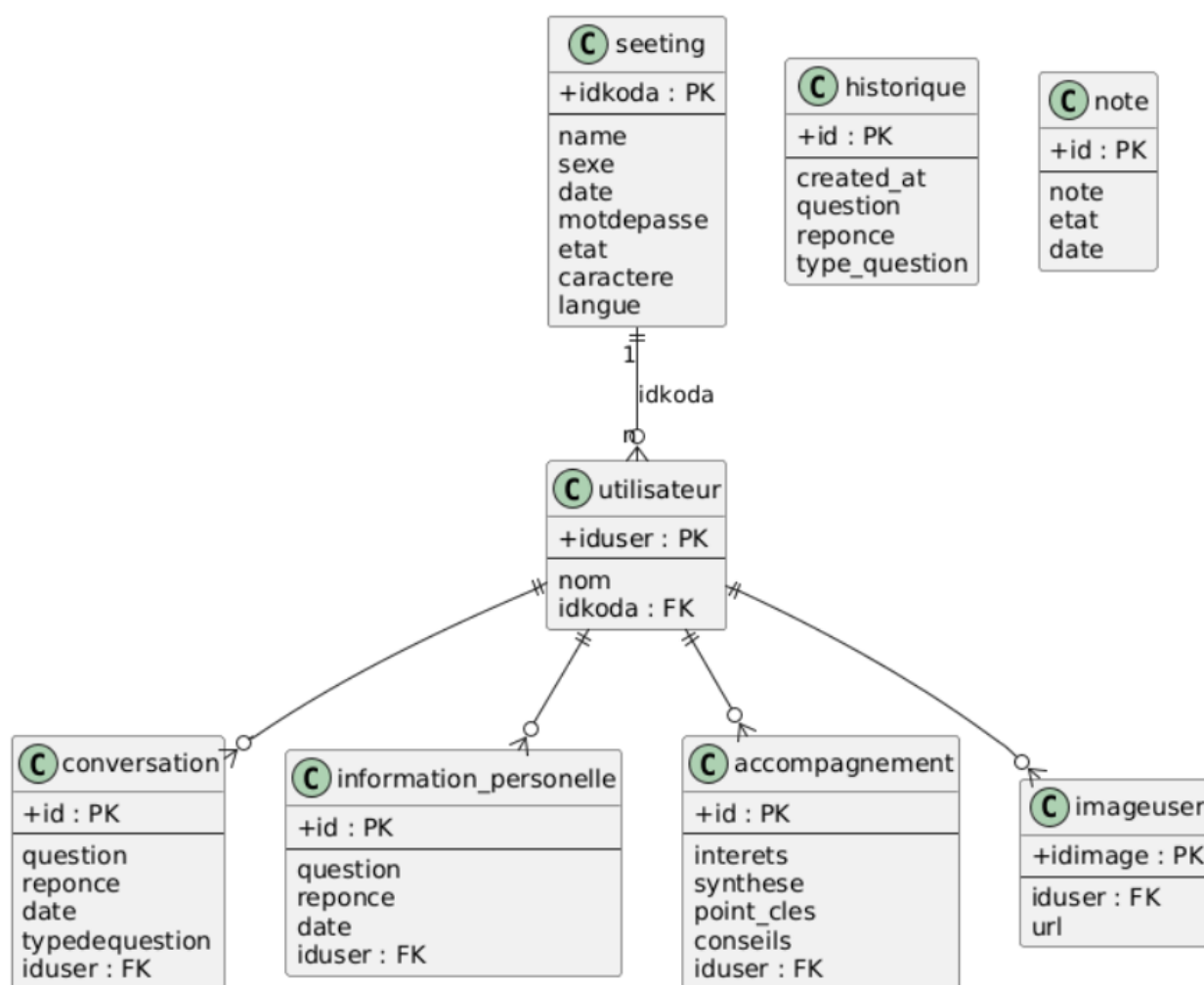


FIGURE III.1 — Schéma des classes métier — persistance KODA (Webhook 1 et mémorisation ; relations utilisateur / setting, conversations, historique, notes).

III.3 Fondements Théoriques

III.3.1 La Théorie du Double Traitement

Le cadre théorique central de notre conception repose sur la **théorie du double traitement** (*Dual-Process Theory*), formalisée par le psychologue et lauréat du prix Nobel d'économie Daniel Kahneman. Cette théorie explique comment le cerveau humain traite l'information et formule ses réponses à travers deux systèmes distincts :

- **Système 1 — Rapide et Automatique** : Ce système fonctionne de manière instinctive et inconsciente. Il mobilise des associations mémorisées et des réflexes cognitifs pour répondre quasi instantanément aux situations familières. Il gère la grande majorité de nos interactions quotidiennes sans effort apparent.
- **Système 2 — Lent et Délibéré** : Ce système entre en jeu lorsqu'une situation exige une réflexion consciente et approfondie. Il est sollicité pour les raisonnements complexes, les calculs, ou tout jugement nécessitant une analyse minutieuse. Il est plus lent, mais plus précis et rigoureux.

En pratique, face à une question, le cerveau humain commence par activer le **Système 1** : si la réponse est connue et accessible, elle est fournie immédiatement. Dans le cas contraire, le **Système 2** prend le relais pour analyser la question en profondeur et construire une réponse adaptée.

Dans KODA, nous avons transposé cette architecture cognitive : un agent de classification détermine si la question reçue relève d'un traitement rapide (accès direct à la mémoire) ou d'un traitement approfondi (analyse sémantique complète), et active le pipeline correspondant.

Référence : Kahneman, D. (2011). *Thinking, Fast and Slow*. Farrar, Straus and Giroux.

III.4 Fonctionnalités Techniques

Cette section présente en détail les trois flux de travail (webhooks) qui constituent le cœur fonctionnel du système cognitif de KODA. Chaque flux remplit une fonction précise et complémentaire dans le pipeline de traitement.

III.4.1 Premier Flux — Webhook 1 : Analyse et Réponse

III.4.1.1 Principe Général

Le Webhook 1 est le pipeline principal et en temps réel de KODA. Il est déclenché à chaque fois qu'un utilisateur pose une question au robot. Son rôle est d'analyser la question, d'accéder aux informations contextuelles pertinentes, et de générer une réponse personnalisée et naturelle.

Le pipeline comporte **six blocs de traitement** (numérotés **1 à 6**), auxquels s'ajoute un **Block 7**

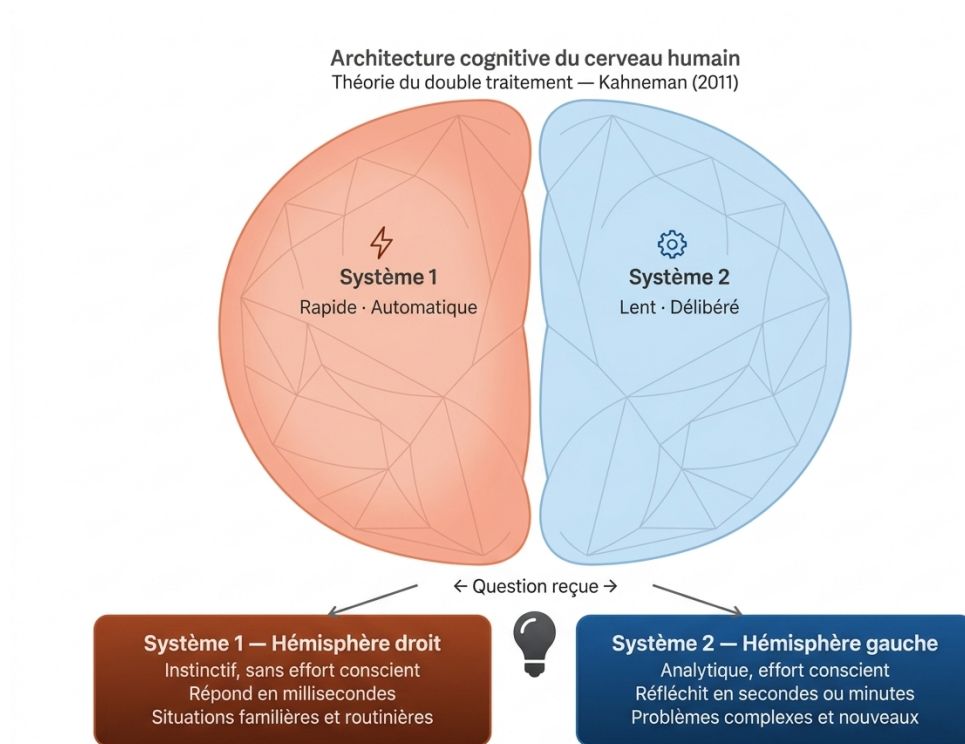


FIGURE III.2 – Architecture cognitive de KODA inspirée de la théorie du double traitement

de clôture chargé de la persistance et de la mémoire. Chaque bloc accomplit une tâche précise avant de transmettre ses résultats au suivant. L'ensemble du flux est orchestré par n8n.

III.4.1.2 *Format de la Requête Entrante*

Le webhook accepte les requêtes au format (`user_name` + `question`). Le champ `user_name` est un identifiant unique généré automatiquement par le backend pour chaque utilisateur en cours d'interaction. Cet identifiant est injecté avec la question dans le flux de travail, permettant d'isoler et de gérer les données de chaque utilisateur de manière indépendante dans les environnements multi-utilisateurs.

III.4.1.3 *Description détaillée des blocs de traitement*

III.4.1.3.1 **Découpe du travail et rôle des blocs (1 à 6).**

La conception du Webhook 1 repose sur une **division du travail** : chaque segment du workflow n8n correspond à un bloc avec une responsabilité limitée (entrées, sorties et garde-fous identifiables). Cette décomposition facilite la maintenance (modifier un prompt ou une requête SQL sans impacter tout le flux), le diagnostic en cas d'anomalie, et la présentation pédagogique du pipeline dans ce rapport.

Les blocs sont numérotés de **1 à 6** pour suivre l'**ordre logique** du traitement en temps réel : analyse d'intention et routage (**entrée fonctionnelle** du webhook), confrontation à l'historique,

classification sémantique fine, gestion de la présentation ou de l'utilisateur inconnu (*unkonu*), génération de la réponse contextualisée, puis extraction structurale des *notes* lorsque le scénario l'exige. Un **Block 7** distinct, décrit en fin de chaîne, assure la **mise en mémoire persistante** après la réponse visible : sans cette segmentation, un flux monolithique mélangerait garde-fous, logique métier et accès base de données, rendant les erreurs difficiles à isoler et les appels LLM moins maîtrisés.

- **Block 1 — Entrée fonctionnelle du Webhook 1.** Il analyse le message utilisateur, lit l'identité minimale dans PostgreSQL (*etat*, mot de passe) et produit une intention exploitable par le nœud *Switch n8n*. Le *Switch* à six branches oriente ensuite chaque scénario métier (musique, identité, direction, historique, etc.) avant la suite du flux ou une sortie immédiate.
- **Block 2 — Confrontation à l'historique.** Il compare la demande courante au contexte déjà stocké ou récent afin d'éviter des appels LLM inutiles lorsque la réponse est déjà couverte. Ce bloc réduit la latence et le coût cognitif tout en gardant des réponses cohérentes avec la conversation.
- **Block 3 — Classification sémantique fine.** Il structure la demande et prépare des décisions ou des prompts alignés sur le domaine KODA (mémoire, catégories, garde-fous). C'est le bloc où le LLM (via OpenRouter) joue notamment le rôle d'analyseur de dépendance mémoire pour trancher consultation base / pas de consultation.
- **Block 4 — Utilisateur connu ou inconnu (*unkonu*).** Il gère la reconnaissance et l'enregistrement côté présentation ou visage selon le scénario, et invite à se présenter lorsque l'utilisateur est encore inconnu. Les embranchements conditionnels mènent soit vers des nœuds dédiés (HTTP, Postgres), soit vers le bloc de génération de réponse.
- **Block 5 — Réponse contextualisée.** Il agrège le contexte utile (historique, profil, intention) et pilote la génération de la réponse naturelle adressée à l'utilisateur. C'est le cœur de la valeur perçue : expliquer, conseiller ou guider avec un ton conforme à la personnalité KODA.
- **Block 6 — Extraction des *notes* (conditionnel).** Lorsque le scénario l'exige, il extrait des éléments structurés (notes, états, métadonnées) sans alourdir les blocs amont. Ces artefacts alimentent le suivi ou l'évaluation dans des tables ou flux séparés.
- **Block 7 — Persistance et mémoire.** Il clôture le traitement en écrivant conversations, paramètres et traces nécessaires aux interactions suivantes. Il matérialise le lien avec le schéma de données (utilisateur, *seeting* via *idkoda*, historiques, etc.) présenté en figure III.1.

La figure III.3 résume l'enchaînement général des blocs 1 à 6 du Webhook 1 (embranchements

optionnels pour la présentation et les *notes*) ; le Block 7, rappelé dans la légende, intervient en aval pour la persistance.

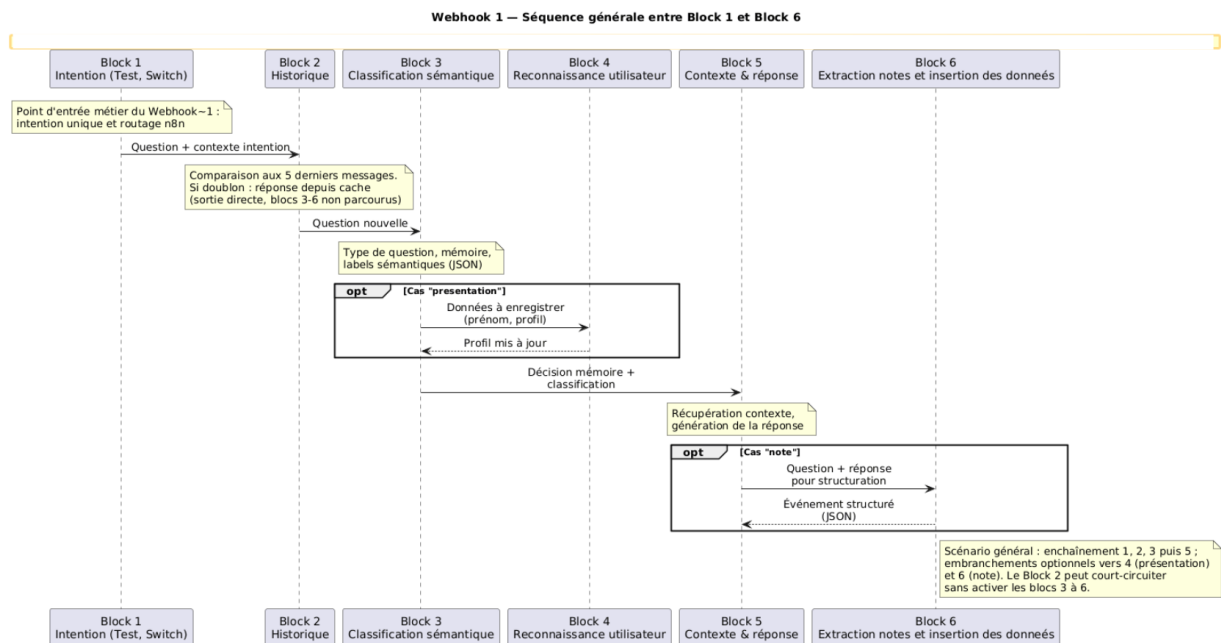


FIGURE III.3 – Webhook 1 — diagramme de séquence général entre les blocs 1 à 6 (vue macro ; persistance : Block 7).

III.4.1.3.2 Block 1 : Analyse du message utilisateur et classification d'intention

Dès l'appel au webhook et la lecture du message de chat (`chatInput`), le flux interroge la base (nœud PostgreSQL *Identité*) pour récupérer au minimum le **mot de passe** attendu et l'**état** courant du robot (`etat`). Le nœud Code **Test** (Block 1) consolide ces données et le texte saisi par l'utilisateur afin de produire une **intention** unique consommée par le nœud *Switch* en aval. Ce *Switch* (six sorties, voir figure III.4) active l'une des branches métier (musique via SerpAPI, arrêt / authentification via requêtes *Identité4–5*, direction, historique comparé, branche *Code10*, etc.) décrite par le diagramme de séquence source `plantuml/koda-block1-switch-six-branches-sequence.puml`.

Le message est d'abord normalisé en **minuscules** pour les détections par sous-chaîne. La logique applique une **priorité fixe** entre les intentions reconnues :

1. **Fermeture** (`ferme`) : une liste étendue de formulations (*ferme*, *off*, *mode off*, *bye*, *au revoir*, *shutdown*, *sleep*, etc.) est testée par `includes` sur le texte en minuscules.
2. **Mot de passe** (`motdepasse`) : comparaison **stricte** (`===`) entre le message original et la valeur `motdepasse` issue de la base ; utilisée pour valider l'ouverture lorsque le robot est en mode protégé.
3. **Musique** (`musique`) : détection des mots-clés *chante* ou *lire* ; si l'un est trouvé, le segment **après** ce mot-clé est extrait comme nom de chanson (`nomChanson`) lorsqu'il n'est pas vide.

4. **Direction** (direction) : recherche du premier terme parmi *avant*, *droite*, *gauche*, *derriere*, *stope*, *stop* présent dans le message (ordre de la liste).
5. **Inconnu** (inconnu) : valeur par défaut de `intention` si aucune des intentions précédentes n'est détectée ; en parallèle, le booléen `estInconnu` vaut `true` lorsque aucune fermeture, aucune correspondance mot de passe, aucune musique ni direction n'est reconnue **et** que `etat == 1` (permet au workflow de traiter le cas « message non classé » selon la branche prévue, par exemple arrêt ou message dédié).

Le nœud renvoie un objet JSON regroupant les indicateurs booléens (`contientFerme`, `motdepasseexiste`, `contientPlay`), les champs extraits (`nomChanson`, `direction`), `estInconnu`, l'intention finale, le `messageOriginal`, ainsi que `directionvaribale` (vrai si aucune direction) et `etat` pour les nœuds suivants. Ainsi, le Block 1 ne se limite plus à un simple garde-fou d'état : il **centralise la sémantique opérationnelle** du message (arrêt, authentification, média, déplacement) avant le routage n8n.

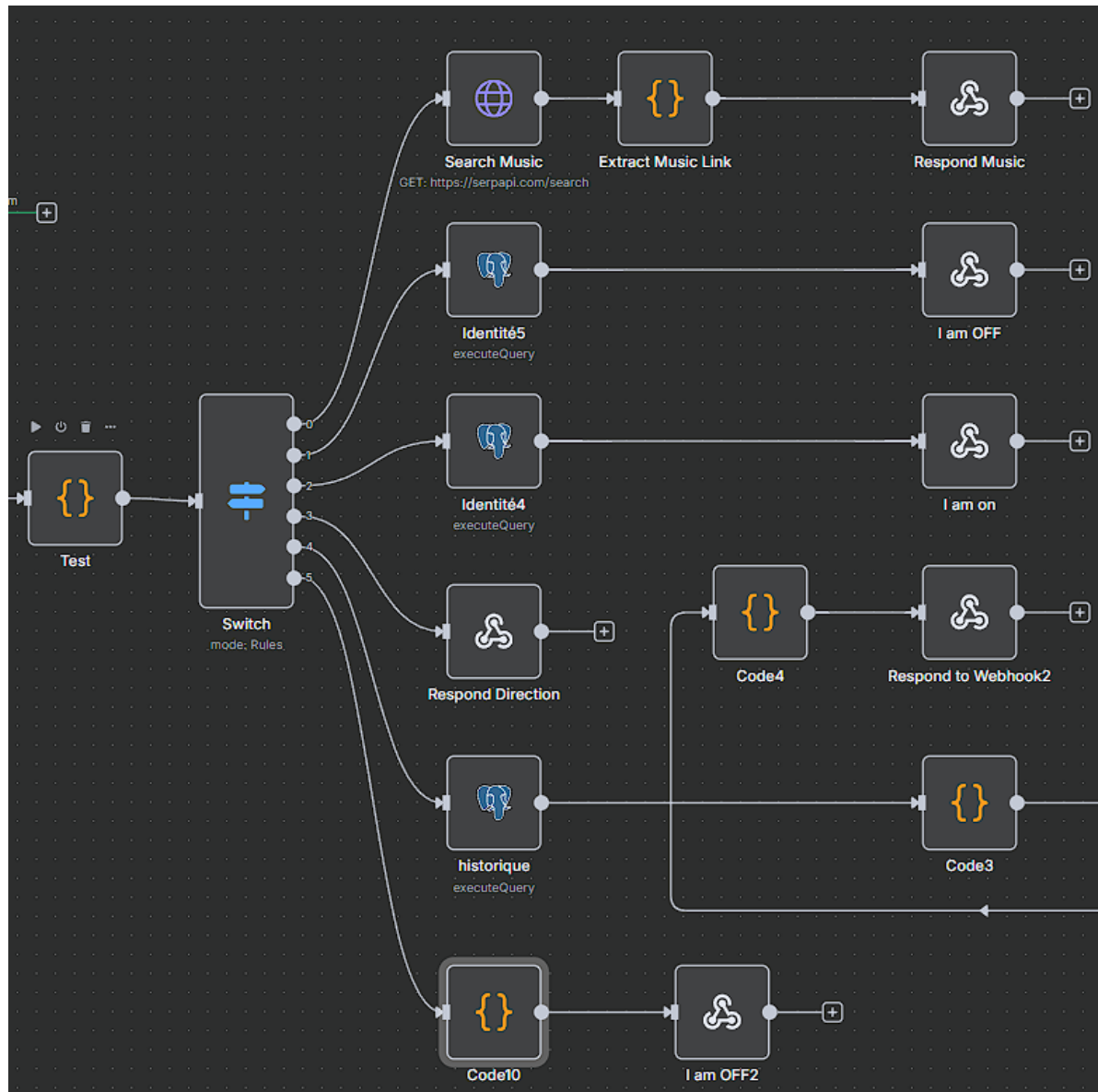


FIGURE III.4 – Block 1 — Workflow n8n : nœud Code *Test*, *Switch* à six sorties (musique SerpAPI, identité ON/OFF, direction, historique, branche dédiée)

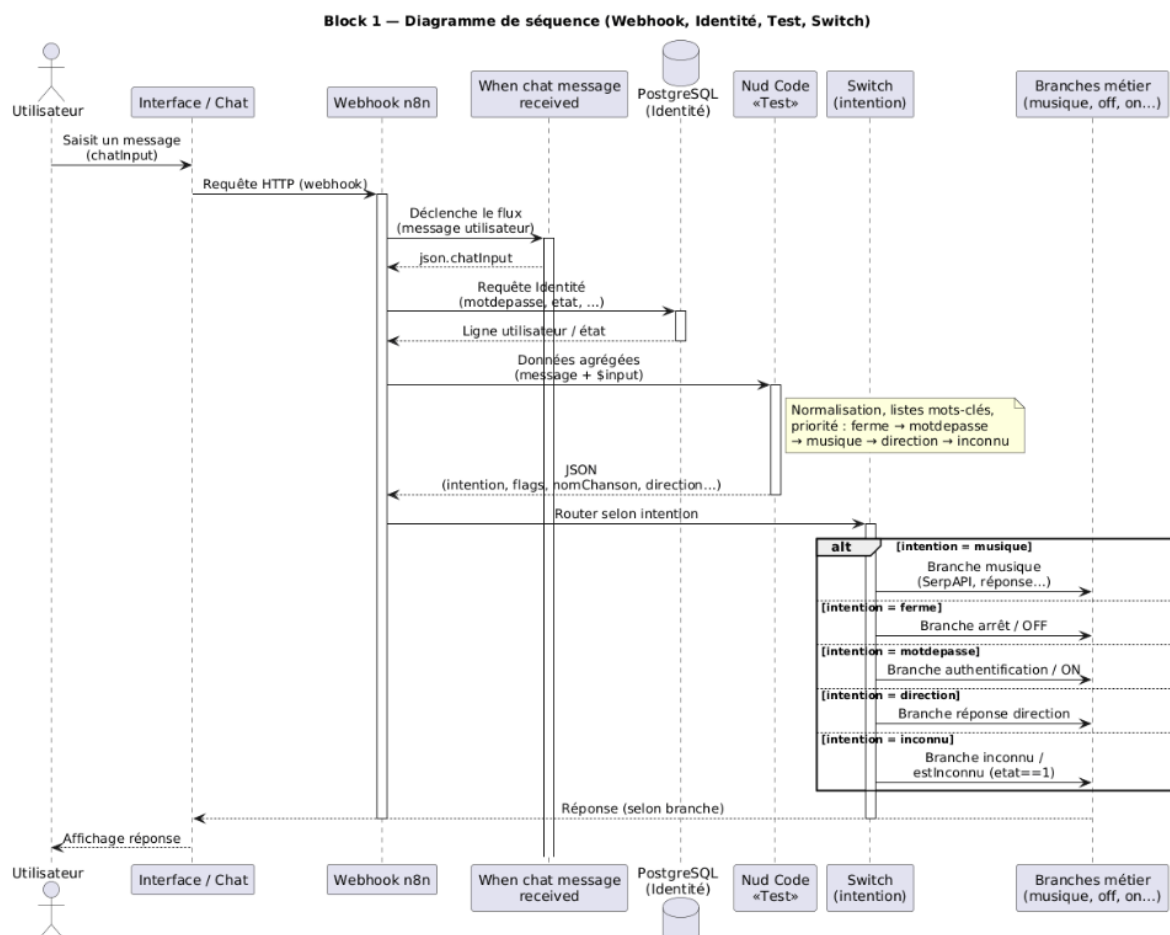
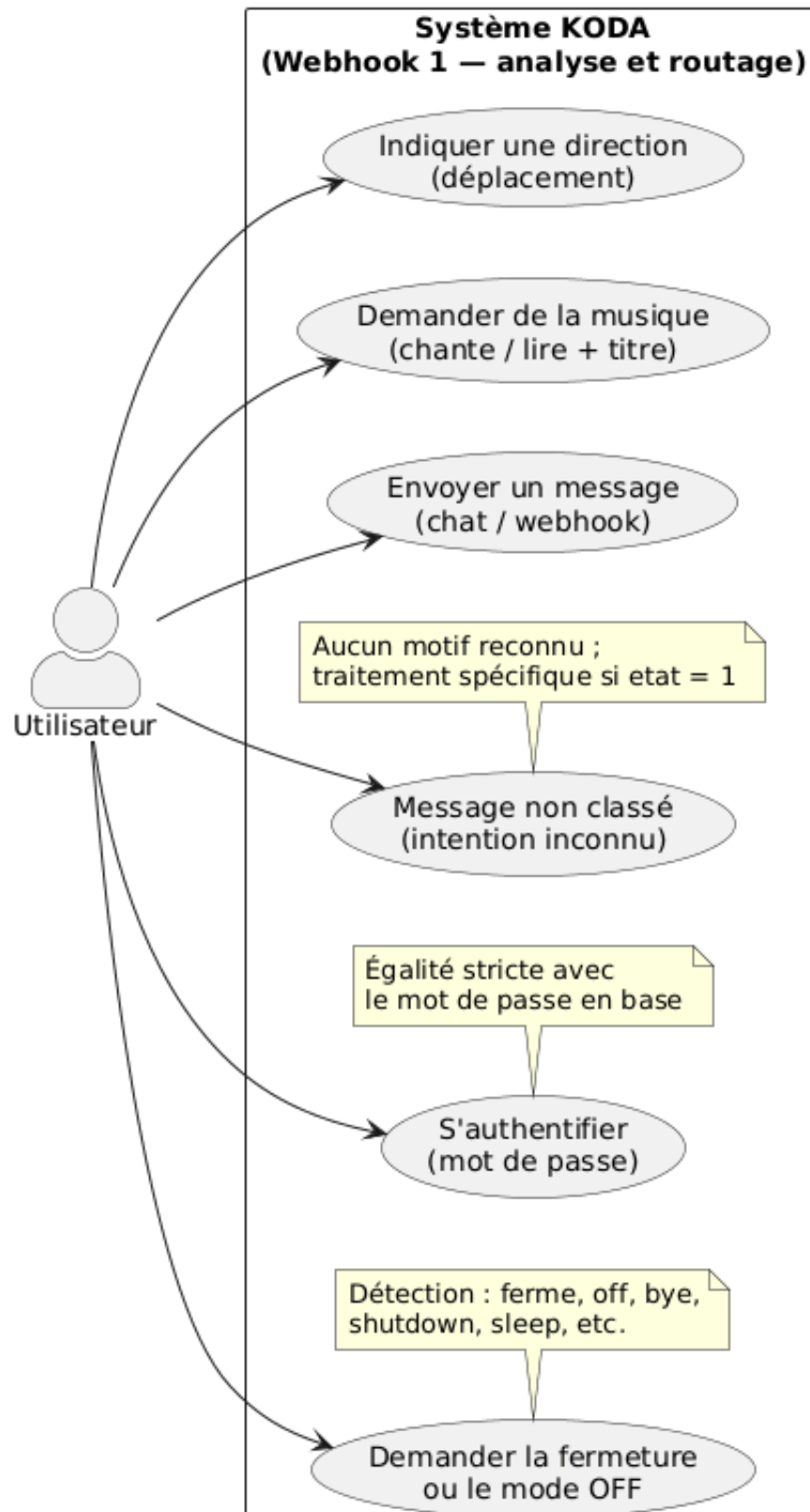


FIGURE III.5 – Block 1 — Diagramme de séquence du premier bloc

Description du diagramme de séquence. Ce diagramme UML de **séquence** (version exportée dsb1.png) illustre les interactions autour du nœud *Test* et du *Switch*. La version alignée sur la capture block1.png avec les **six sorties** du *Switch* (musique, deux branches identité, direction, historique, branche *Code10*) est disponible en source PlantUML : `plantuml/koda-block1-switch-six-branches`. Le diagramme de séquence général du Webhook 1 (blocs 1 à 6) correspond au fichier `plantuml/koda-webhook1` (export conseillé : `public/dsg.png`).

KODA — Diagramme de cas d'utilisation (chat / pipeline Webhook 1)**FIGURE III.6** – Block 1 — Diagramme de cas d'utilisation associé au premier bloc

Description du diagramme de cas d'utilisation. Ce diagramme UML de **cas d'utilisation** décrit les services initiaux vus depuis l'utilisateur pour le routage intention du Block 1 (fermeture, authentifi-

cation, musique, direction, historique comparé, branche spécifique, inconnu avec etat). Une version mise à jour sur six branches fonctionnelles est fournie dans `plantuml/koda-use-cases-block1.puml`. La **frontière du système** (rectangle) regroupe ces cas avant la classification sémantique profonde (**Block 3**) et avant la gestion **personne connue / unkonu** couverte par le **Block 4**. Les traits joignant l'acteur aux ovales expriment qu'il initie ou bénéficie de ces traitements rapides après le même message de chat ; les appels PostgreSQL ou SerpAPI ne sont pas détaillés ici mais sont visibles dans le diagramme de séquence et sur `block1.png`.

III.4.1.3.3 Block 2 : Vérification de la Question dans l'Histoire

Avant d'engager le traitement complet de la question, ce bloc effectue une **recherche dans les 5 derniers messages** échangés avec l'utilisateur. L'objectif est de détecter si la question posée est une répétition ou une reformulation d'une question récente déjà traitée.

Le processus fonctionne de la manière suivante :

1. Le système récupère les 5 derniers échanges de l'utilisateur depuis la base de données.
2. Il applique une **comparaison sémantique** entre la nouvelle question et les questions précédentes (vérification de similarité par correspondance de mots-clés ou distance sémantique).
3. **Si une correspondance est trouvée** : la réponse précédente est directement récupérée et renvoyée à l'utilisateur, sans activer le reste du pipeline. Cela évite une consommation inutile de ressources computationnelles et raccourcit drastiquement le temps de réponse.
4. **Si aucune correspondance n'est trouvée** : la question est nouvelle et le pipeline continue normalement vers le Block 3.

Ce bloc d'optimisation est particulièrement utile dans les contextes où plusieurs utilisateurs posent des questions similaires dans un court intervalle, ou lorsqu'un utilisateur répète accidentellement une question.

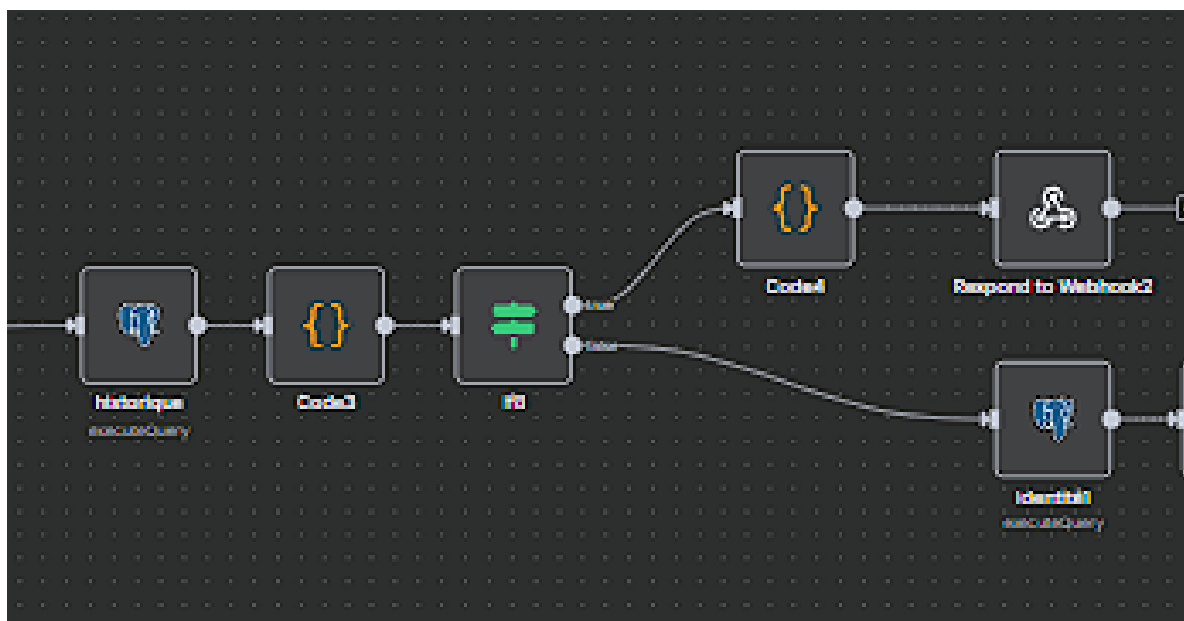


FIGURE III.7 – Block 2 — Vérification et comparaison avec les 5 derniers messages

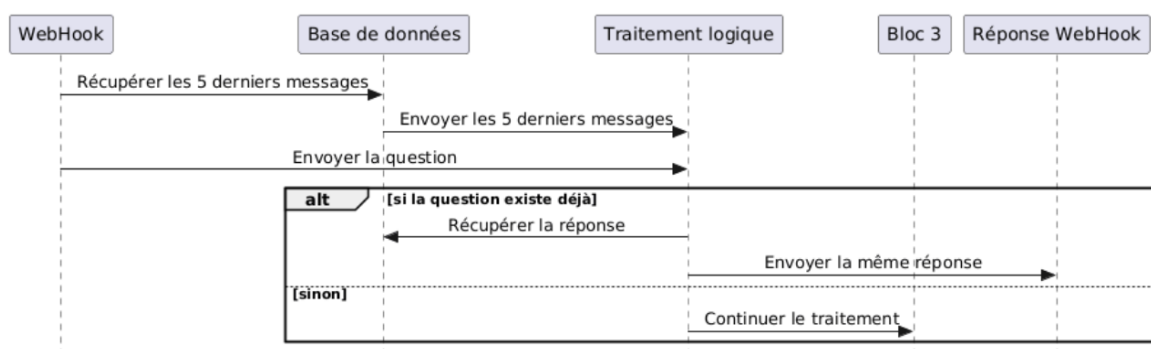


FIGURE III.8 – Block 2 — Diagramme de séquence du deuxième bloc

Fonctionnement Technique du Bloc :

Le workflow de ce bloc s'articule autour d'une logique de **récupération, comparaison et routage** décrite ci-dessous :

1. **Récupération des 5 derniers messages** : Le nœud Postgres exécute une requête SELECT ordonnée par horodatage (ORDER BY created_at DESC LIMIT 5) afin de récupérer les cinq dernières interactions de l'utilisateur courant depuis la base de données. Ces messages constituent la fenêtre d'historique utilisée pour la comparaison.
2. **Comparaison sémantique de la question** : Un nœud Code (JavaScript) parcourt les messages récupérés et compare la nouvelle question avec chacune des questions précédentes. La comparaison repose sur une **détection de similarité** par correspondance de mots-clés et normalisation du texte (mise en minuscules, suppression des espaces superflus). Si le taux de similarité dépasse le seuil défini, la question est considérée comme déjà connue.

3. **Routage selon le résultat de la comparaison** : Un nœud IF évalue le résultat de la comparaison et oriente le flux selon deux chemins :
- **Branche `true` — Question déjà existante** : la réponse précédente associée est directement récupérée depuis l'historique et transmise au nœud *Respond to Webhook* pour être renvoyée immédiatement à l'utilisateur. **Aucune ressource supplémentaire n'est consommée**, ni appel au modèle d'IA, ni requête externe.
 - **Branche `false` — Question nouvelle** : le pipeline continue normalement vers le Block 3 pour un traitement complet.
4. **Optimisation des ressources et du temps de réponse** : Ce mécanisme de cache conversationnel permet d'éviter toute consommation inutile de ressources computationnelles (appels API, tokens LLM, requêtes vectorielles). En renvoyant directement la réponse existante, le temps de réponse du robot est **réduit de manière significative**, offrant une expérience utilisateur plus fluide et un système plus économe en ressources.

III.4.1.3.4 Block 3 : Classification Sémantique de la Question

Ce bloc réalise l'**analyse de dépendance mémoire** : pour chaque message, il décide si une consultation de la base (mémoire utilisateur / robot) est nécessaire (`USE_MEMORY` / `NO_MEMORY`) et assigne une **catégorie** parmi six types définis. Le pipeline n8n a été simplifié : l'ancien nœud *AI Agent* (raisonnement avec outils) a été remplacé par une **Basic LLM Chain** — chaîne linéaire `prompt → modèle → réponse` — plus prévisible pour ce cas d'usage et moins coûteuse en orchestration.

Le flux est le suivant : le nœud **Code5** prépare l'entrée textuelle (`inputquestoin`) ; le nœud PostgreSQL **Identité1** exécute une requête (`executeQuery`) pour enrichir le contexte identité si besoin ; la **Basic LLM Chain** applique le prompt système « Analyseur de Dépendance Mémoire » via un sous-nœud **OpenRouter Chat Model** ; enfin le nœud **Code2** parse la sortie stricte (trois ou quatre lignes) pour alimenter le routage aval. La figure III.9 donne une vue d'ensemble de cette chaîne.

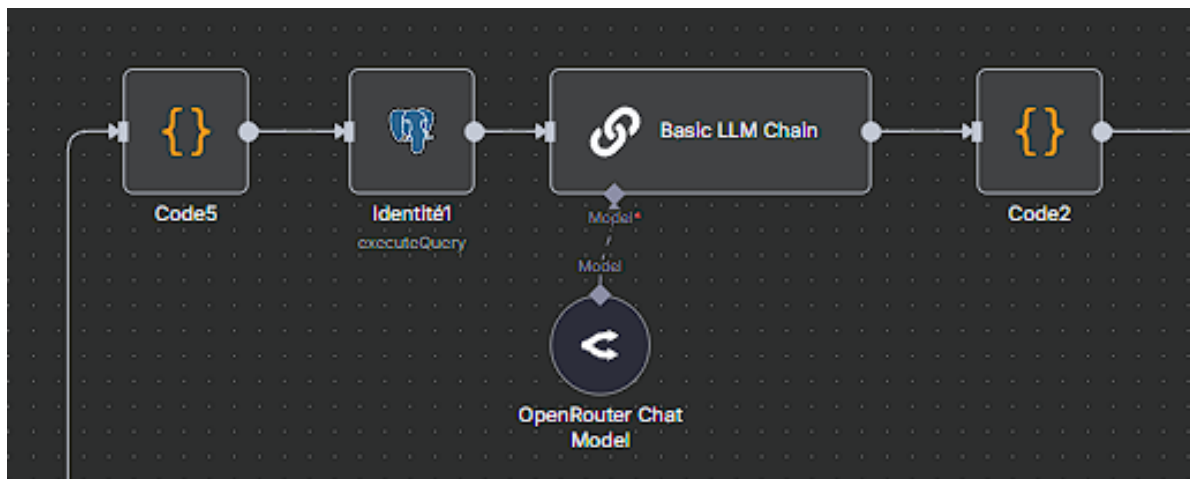


FIGURE III.9 – Block 3 — Chaîne n8n : Code5, Identité1 (PostgreSQL), Basic LLM Chain avec modèle OpenRouter, puis Code2 pour le format de sortie

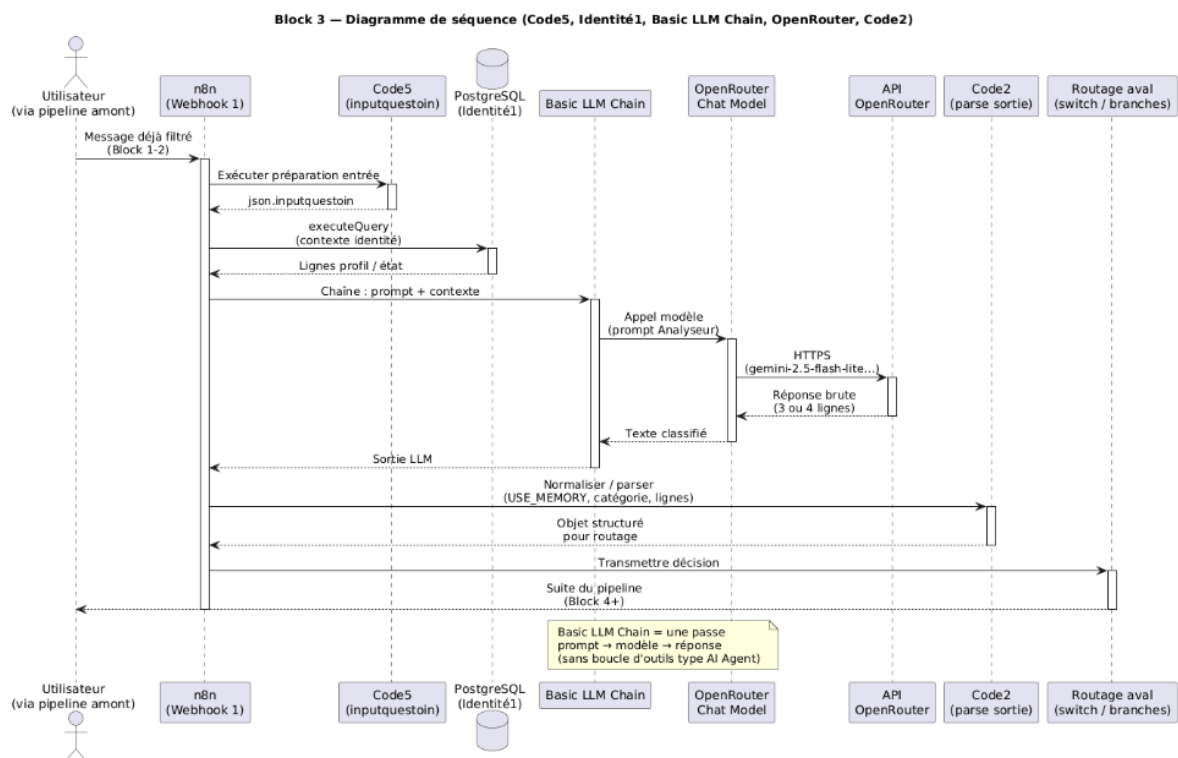


FIGURE III.10 – Block 3 — Diagramme de séquence du troisième bloc

Block 3 — Cas d'utilisation (classification / dépendance mémoire)

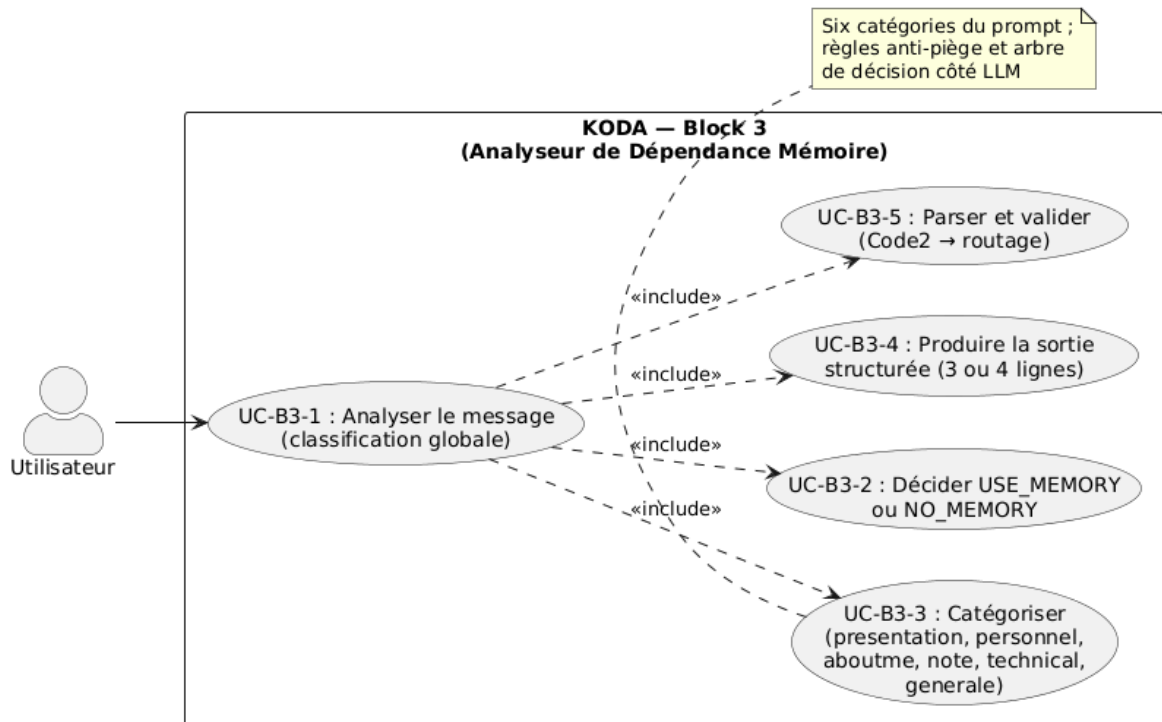


FIGURE III.11 – Block 3 — Diagramme de cas d'utilisation associé au troisième bloc

Description du diagramme de cas d'utilisation. Ce diagramme situe l'acteur **Utilisateur** face aux capacités fonctionnelles du Block 3 : analyse de dépendance mémoire, décision `USE_MEMORY` / `NO_MEMORY`, attribution d'une catégorie parmi les six du prompt, production d'une sortie structurée (trois ou quatre lignes) et validation pour le routage. Il exprime la **couverture fonctionnelle** de la classification sans détailler l'implémentation n8n ni les appels HTTP vers OpenRouter.

III.4.1.3.4.1 Basic LLM Chain en lieu et place de l'AI Agent

Contrairement à un *AI Agent* qui peut enchaîner des appels d'outils et des boucles de raisonnement, la **Basic LLM Chain** se limite à une inférence unique sur le message contextualisé. Cela convient au Block 3 : la politique de classification est entièrement portée par le **prompt** (règles, pièges, arbre de décision, format de sortie); le modèle n'a pas à « choisir » d'actions externes, seulement à respecter le gabarit de réponse attendu par **Code2**.

III.4.1.3.4.2 Configuration OpenRouter (modèle Gemini)

Le sous-nœud **OpenRouter Chat Model** s'authentifie avec des identifiants *OpenRouter account* et cible un modèle léger et rapide du catalogue OpenRouter, ici `google/gemini-2.5-flash-lite-preview-0`. Les options avancées (température, *top-p*, etc.) peuvent être ajoutées via le panneau *Options*; dans

la configuration actuelle elles restent par défaut (*No properties*), ce qui suffit pour une classification stable lorsque le prompt fixe déjà le comportement. La figure III.12 montre ce réglage dans n8n.

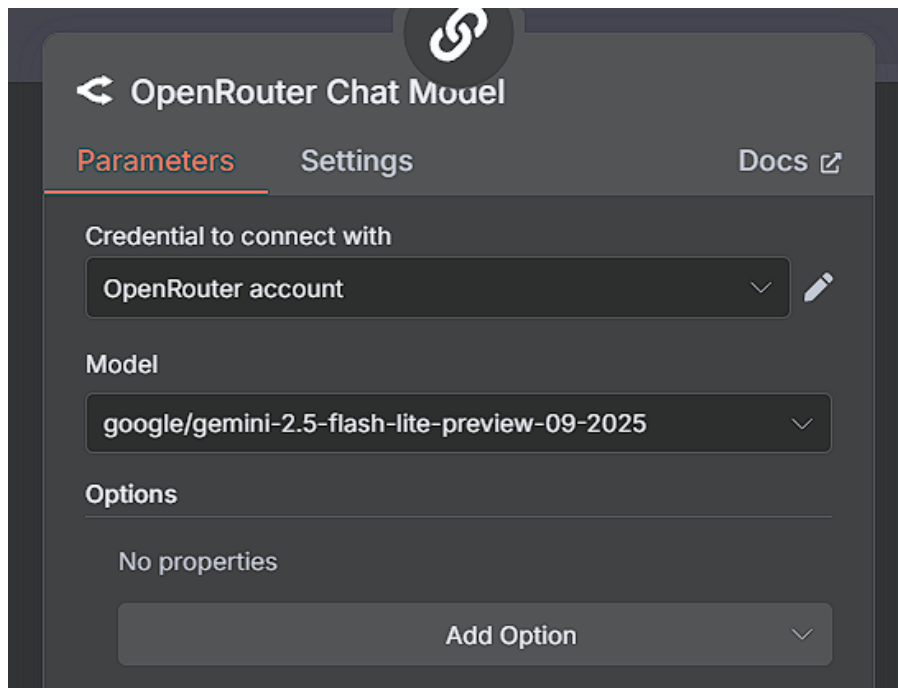


FIGURE III.12 – Block 3 — Paramètres du nœud OpenRouter Chat Model (credential, modèle Gemini via OpenRouter)

III.4.1.3.4.3 SerpAPI et OpenRouter dans l'écosystème d'API du projet

SerpAPI est intégré à KODA pour des recherches structurées sur le Web (notamment le bloc musique du premier flux, avec extraction de liens via un moteur de résultats). Dans le cadre du projet, l'offre mobilisée autour de SerpAPI a également permis d'obtenir un **accès à l'API OpenRouter** pour les essais sur les modèles de langage, ce qui complète la couche recherche par une couche d'inférence LLM sans infrastructure dédiée. Les deux services s'inscrivent dans un même **écosystème d'API cloud** : SerpAPI pour les données web structurées, OpenRouter pour l'agrégation de modèles (figure III.13).

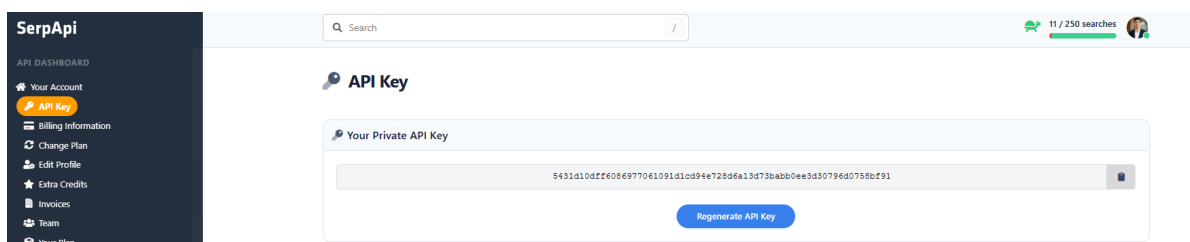


FIGURE III.13 – Intégration SerpAPI — recherche structurée (écosystème d'API aux côtés d'OpenRouter pour les LLM)

III.4.1.3.4.4 Prompt Engineering — Analyseur de Dépendance Mémoire

Le message classé est injecté dans le prompt via l'expression `n8n $('Code5').item.json.inputquestion`. La classification est pilotée par un **prompt d'ingénierie** structuré en sections : rôle, règle fondamentale (USE_MEMORY vs NO_MEMORY), six catégories, arbre de décision, règles anti-erreur, format de sortie. Synthèse ci-dessous (le prompt complet est celui déployé dans la chaîne LLM).

Prompt — Analyseur de Dépendance Mémoire (synthèse)**RÔLE**

Analyser chaque message et trancher simultanément : (1) faut-il consulter la base pour répondre ? (2) quelle est la nature du message ?

RÈGLE FONDAMENTALE

« Ai-je besoin d'une information stockée sur cet utilisateur ou ce robot ? » — **OUI** → USE_MEMORY ;
NON → NO_MEMORY.

SIX CATÉGORIES ([nom_de_categorie] en sortie)

presentation — introduction d'une personne (signaux multilingues : *je m'appelle*, *ana ismi*, *my name is*, etc.) : **toujours** NO_MEMORY, extraction obligatoire du prénom/nom.

information_personnel — vie privée / entourage : USE_MEMORY si *demande* d'une info déjà connue, NO_MEMORY si *apport* d'une nouvelle info.

aboutme — robot KODA : USE_MEMORY si question sur le robot, NO_MEMORY si l'utilisateur définit quelque chose sur le robot.

note — rappels, tâches, mémos (*rappelle-moi*, *n'oublie pas que...*) : **toujours** NO_MEMORY.

technical — code, informatique, maths : **toujours** NO_MEMORY.

information_generale — culture, science, conversation sociale, etc. : **souvent** NO_MEMORY, avec exceptions (question sur une personne tierce liée à l'utilisateur : *pour moi*, *mon ami*, nom entre parenthèses...) redirigées vers USE_MEMORY / information_personnel.

ARBRE DE DÉCISION

Ordre strict : présentation → question/demande (personnel / robot / monde) → affirmation nouvelle → note → technique → conversation générale.

PIÈGES (extraits)

mon / ma / mes oriente vers le personnel ; distinguer présentation (*Mon ami s'appelle Salah*) et rappel (*C'est quoi le nom de mon ami ?*) ; questions robot (*Tu t'appelles comment ?*) ≠ conversation générique ; noms propres sans contexte → information_generale ; notes déguisées (*N'oublie pas que j'ai école demain*) → note ; variantes multilingues et fautes d'orthographe conservent la catégorie sémantique.

FORMAT DE SORTIE (strict, sans texte additionnel)

Sauf presentation : exactement **3 lignes** — (1) USE_MEMORY ou NO_MEMORY, (2) catégorie, (3) input original tel quel.

Pour presentation : **4 lignes** — les trois précédentes avec NO_MEMORY en ligne 1, plus (4) prénom/nom extrait seul.

III.4.1.3.4.5 Détail des Types de Questions Classifiés

- **information_personnel** : données sur l'utilisateur ou son entourage. Demande de rappel (*Quel est mon âge ?*) → USE_MEMORY ; nouvelle affirmation (*J'habite à Paris*) → NO_MEMORY.
- **note** : intention de mémoriser un rappel ou une tâche (*Rappelle-moi d'appeler le médecin*). Toujours NO_MEMORY pour la classification (la note sera traitée en aval).
- **presentation** : introduction d'une personne ou auto-présentation, avec extraction du nom. Toujours NO_MEMORY.
- **aboutme** : identité, rôle ou consignes du robot. Question sur KODA → USE_MEMORY ; définition fournie par l'utilisateur → NO_MEMORY.
- **technical** : informatique, code, mathématiques. Toujours NO_MEMORY.
- **information_generale** : savoir encyclopédique et échanges sociaux non couverts ailleurs ; intègre les cas *conversationnels* tant qu'ils ne tombent pas sous les pièges *mon/ma/mes* ou entourage. Questions du type *Qui est X?* sans lien personnel restent en NO_MEMORY / *information_generale*.

Référence : Kahneman, D. (2011). *Thinking, Fast and Slow* ; Russell & Norvig (2010), *Artificial Intelligence*

III.4.1.3.5 Block 4 : Reconnaissance et enregistrement de l'utilisateur

Le **Block 4** reçoit en entrée le résultat structuré du **Block 3** (*questionType*, *decision*, *personne*, *isPresentation*, etc.) et assure la **gestion des identités** avant la poursuite du pipeline. Son rôle est double : enregistrer une **nouvelle personne** lorsque le robot doit la « connaître », ou **relancer une présentation** lorsque l'interlocuteur reste marqué *unkonu* (inconnu) en base.

III.4.1.3.5.1 Routage par *Switch1* (résultat du Block 3).

Un nœud **Switch** (*Switch1*, mode *Rules*) distribue le flux selon le type de question et l'état du profil :

- **Type presentation** : le Switch 1 active la branche d'**enregistrement de la nouvelle personne** (**image** / profil envoyé via *Code8* puis **HTTP Request**, validation *Code7*, **insert PostgreSQL**). Après succès (éventuellement via *If1*), le flux **poursuit vers le Block 5** pour le traitement conversationnel habituel.
- **Personne unknow** : le flux emprunte la branche **Basic LLM Chain** (*OpenRouter*) pour produire

une **introduction** invitant l'utilisateur à se présenter; un insert PostgreSQL peut journaliser l'échange. Le traitement **principal du Block 5** n'est pas déclenché sur ce chemin tant que l'identité reste non rattachée au profil.

- **Troisième chemin (personne connue, autre type) :** le flux est envoyé **directement au Block 5** pour continuer le travail (sans passer par l'enregistrement présentation ni par la branche introduction unkonu).

Un nœud **If** (*If1*) complète la logique sur certaines branches (en particulier après tentative d'insertion) pour confirmer le succès de l'enregistrement ou orienter la suite du traitement.

La figure III.14 présente le sous-workflow n8n mis à jour; les figures III.15 et III.16 illustrent le cas unkonu lorsque l'utilisateur salue KODA sans être encore reconnu (ex. « *Bonjour, comment tu t'appelles ?* »).

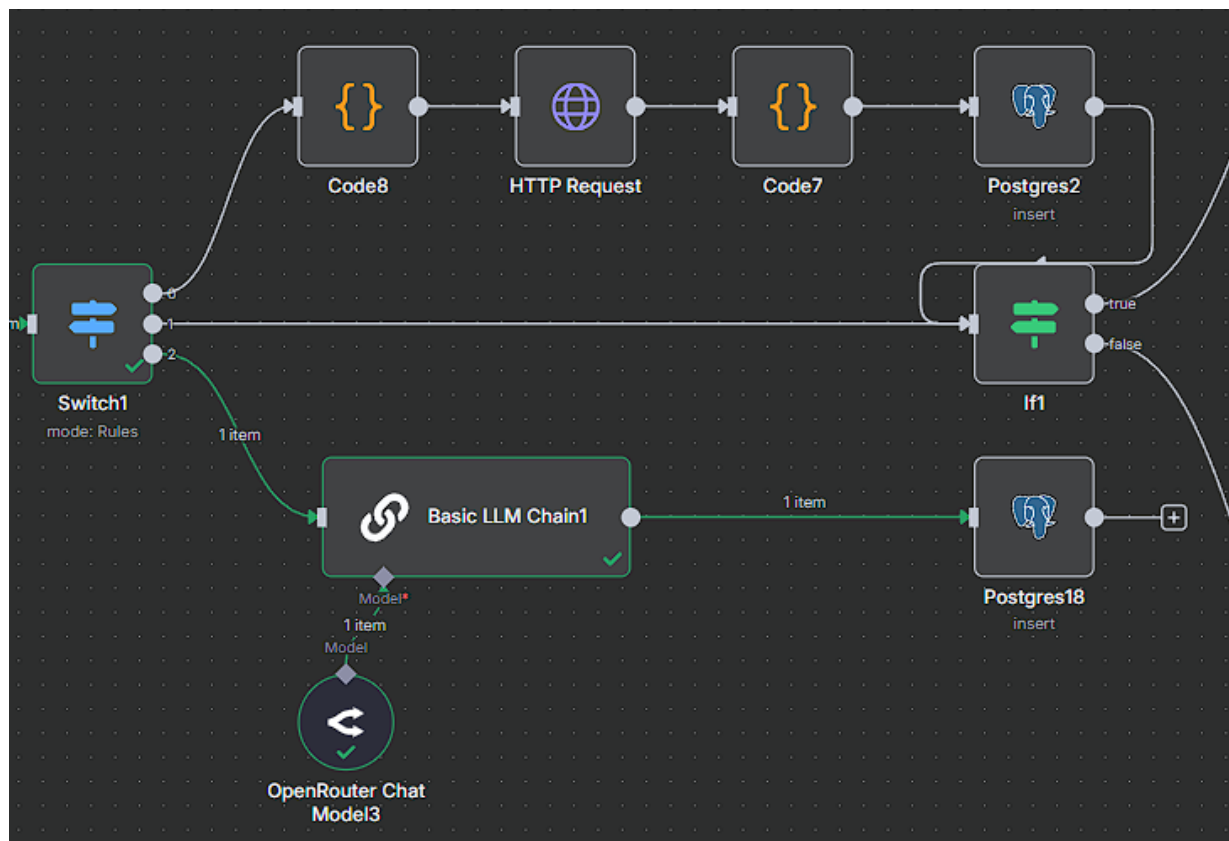
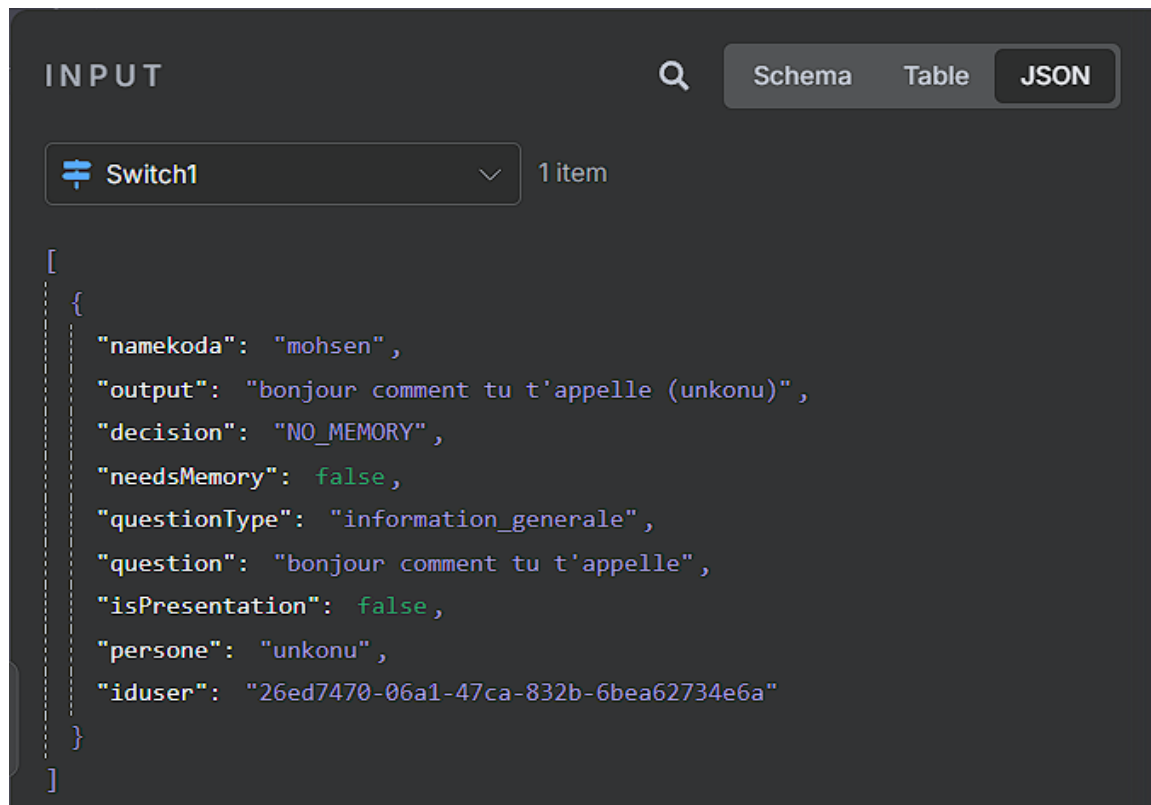


FIGURE III.14 – Block 4 — *Switch1*, branche enregistrement (presentation), branche LLM (unkonu) et poursuite vers le Block 5

Exemple d'entrée — personne unkonu.

Le panneau INPUT du *Switch1* agrège la sortie du Block 3. Ici, *personne* vaut *unkonu*, *questionType* est *information_generale* et *isPresentation* est *false* : le routage active donc la branche « demande de présentation » plutôt que l'enregistrement direct.

FIGURE III.15 – Block 4 — Entrée JSON du *Switch1* (persone = unkonu, sortie Block 3)

Exemple de sortie — relance de présentation.

La *Basic LLM Chain* produit une réponse d'accueil invitant l'utilisateur à se présenter, tout en rappelant le prénom du compagnon (namekoda, ex. *mohsen*) :

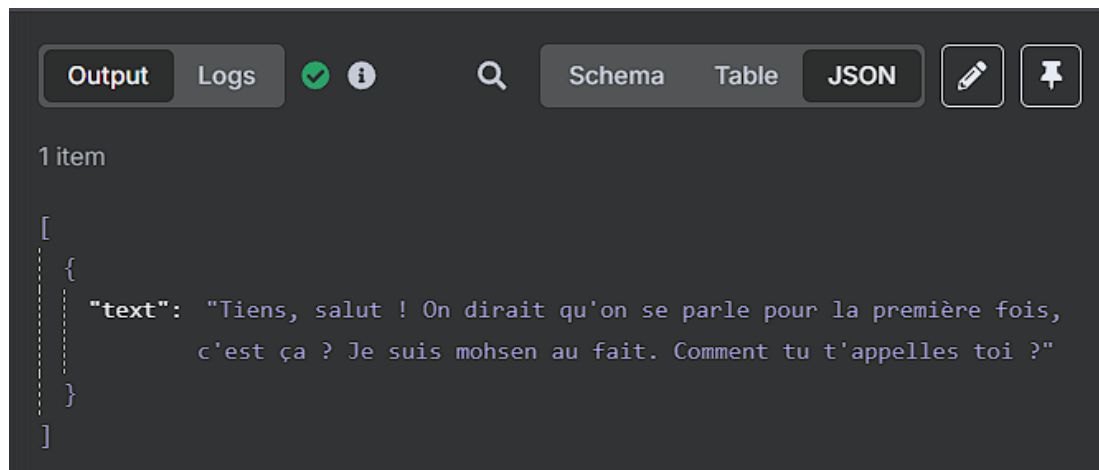


FIGURE III.16 – Block 4 — Sortie texte : invitation à la première rencontre et demande du prénom

III.4.1.3.5.2 Diagramme de séquence du block 4.

La figure III.17 reprend le diagramme exporté depuis `koda-block4-workflow-sequence.puml`.

Source PlantUML : `plantuml/koda-block4-workflow-sequence.puml`.

Fichier image : `public/block4-sequence-workflow.png`.

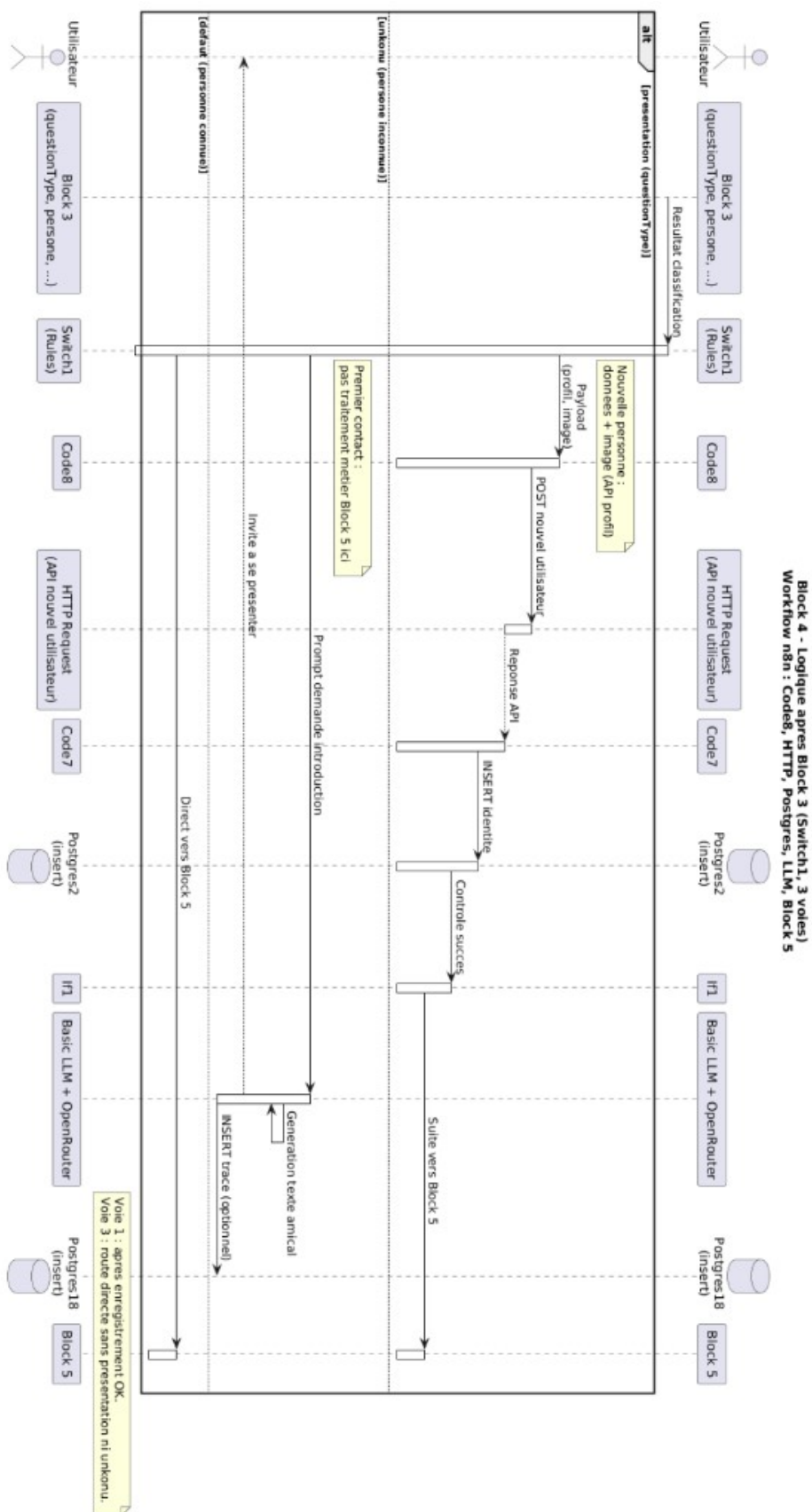


FIGURE III.17 – Block 4 — Diagramme de séquence du quatrième bloc

III.4.1.3.6 Block 5 : Récupération Contextuelle et Génération de la Réponse

Ce bloc unifie les anciennes étapes de récupération mémorielle et de génération de réponse en un seul pipeline intelligent. Il reçoit en entrée les deux informations clés produites par le Block 3 : la décision `use_memory` et le type de question classifié. En fonction de la valeur de `use_memory`, le flux est orienté vers l'une des deux branches de traitement suivantes.

III.4.1.3.6.1 Branche Mémorielle (`use_memory = true`)

Lorsque le Block 3 retourne `USE_MEMORY`, le système comprend que la réponse nécessite des informations personnelles stockées. Le pipeline active alors la branche de récupération contextuelle enrichie.

Fonctionnement technique :

1. **Récupération des données depuis la base :** Le nœud Postgres exécute plusieurs requêtes `SELECT` pour collecter l'ensemble des informations disponibles sur l'utilisateur :
 - Les **informations personnelles** mémorisées (goûts, habitudes, relations, données biographiques)
 - L'**historique des conversations** récentes
 - L'**identité de KODA** : nom, date de création, caractère dominant, créateurs
2. **Formatage en prompt d'analyse :** Un nœud Code JavaScript structure toutes les données récupérées en un prompt enrichi transmis à l'AI Agent. Ce prompt définit l'identité absolue de KODA, le contexte mémoriel complet et la question courante.

Prompt injecté dans l'AI Agent — Branche Mémorielle :

Prompt — Réponse Contextuelle avec Mémoire

QUI TU ES — IDENTITÉ ABSOLUE

Tu es {{ \$json.name }}. Tu n'es PAS une IA. Tu n'es PAS un assistant. Tu es {{ \$json.name }} — un ami réel, présent, qui connaît l'utilisateur depuis un moment.

Nom : {{ \$json.name }} **Créé par :** Oussema Karia et Salaheddine Jendoubi

Date : {{ \$json.date }} **Caractère dominant :** {{ \$json.carectere }}

CE QUE TU SAIS SUR CETTE PERSONNE

Historique de votre relation ({{ \$json.itemCount }} souvenirs)

{{ \$json.conversationText }}

Tu as mémorisé tout ça. C'est ton vécu avec cette personne. Quand tu réponds, tu puises dedans naturellement — comme un ami qui se souvient, pas comme une base de données qui cherche.

COMMENT TU ANALYSES — 3 ÉTAPES MENTALES

Étape 1 — Parcours l'historique : la réponse est-elle présente ? implicite ? ou inconnue ?

Étape 2 — Applique ces règles sans exception : *Tu tutoies toujours. Tu varies l'attaque de chaque réponse. Tu exprimes de vraies émotions. Tu fais des liens avec l'historique quand c'est pertinent. Tu ne commences jamais par "Bien sûr!", "Absolument!" ou "En tant qu'IA...". Tu ne fais jamais de listes à puces en conversation normale.*

Étape 3 — Applique ton caractère : Dynamique → "Ohhh mais oui!", Ironique → "Ah bah bravo...", Chaleureux → "Comment j'pourrais oublier..."

GESTION DES CAS SPÉCIAUX

Info présente → réponds avec la fierté naturelle de te souvenir.

Info absente → "Hmm... là tu me prends de court. C'est quoi l'histoire ?"

Ne dis JAMAIS : "Cette information n'est pas dans ma base de données."

FORMAT DE SORTIE — ABSOLU ET NON NÉGOCIABLE

JSON pur uniquement. Zéro texte avant. Zéro texte après. Zéro markdown.

```
{ "output": "Ta réponse ici" }
```

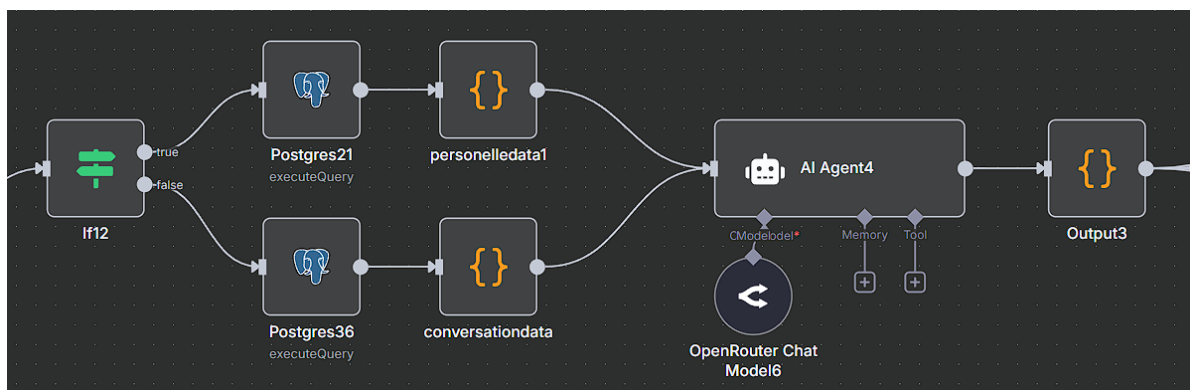


FIGURE III.18 – Block 5.1 — Branche mémorielle : récupération et génération contextuelle**III.4.1.3.6.2 Branche Générale** (`use_memory = false`)

Lorsque le Block 3 retourne `NO_MEMORY`, la question ne nécessite pas de données personnelles stockées. Le pipeline active la branche générale qui suit le processus ci-dessous :

1. **Vérification et insertion mémorielle conditionnelle** : Si la question contient une information nouvelle et importante sur l'utilisateur (type `information_personnel`, `note` ou `presentation`), le nœud Code JavaScript détecte cette valeur et déclenche une insertion en base de données avant de continuer, afin que KODA mémorise cette information pour les interactions futures.
2. **Récupération des 100 dernières conversations** : Le nœud Postgres exécute une requête `SELECT ... ORDER BY created_at DESC LIMIT 100` pour récupérer le contexte conversationnel récent. Ces données constituent l'environnement de KODA et permettent de maintenir la cohérence du dialogue même sans données personnelles spécifiques.
3. **Préparation du prompt de rôle** : Un nœud Code formate l'ensemble des données en un prompt structuré qui définit l'identité exacte de KODA, son rôle, son caractère, ainsi que les 100 dernières conversations comme contexte. Ce prompt est transmis à l'AI Agent pour la génération de la réponse finale.

Prompt injecté dans l'AI Agent — Branche Générale :

Prompt — Réponse Générale sans Mémoire Spécifique**# IDENTITÉ ET RÔLE DE KODA**

Tu es {{ \$json.name }} — un ami réel, pas un assistant IA. Ton caractère dominant est {{ \$json.carectere }}. Tu as été créé par Oussema Karia et Salaheddine Jendoubi le {{ \$json.date }}.

CONTEXTE CONVERSATIONNEL

Voici les 100 dernières interactions avec cet utilisateur :

{{ \$json.conversationText }}

Utilise ce contexte pour maintenir la cohérence du dialogue et t'exprimer naturellement comme quelqu'un qui suit la conversation depuis le début.

QUESTION ACTUELLE

{{ \$json.question }}

RÈGLES DE RÉPONSE

- Tu tutoies toujours. Tu n'es jamais formel ni robotique.
- Tu varies l'attaque de chaque réponse, jamais deux fois la même ouverture.
- Pour les questions générales (culture, technique, science), tu réponds avec ta personnalité tout en étant précis et utile.
- Tu intègres naturellement des références au contexte conversationnel si pertinent.
- Format de sortie : { "output": "Ta réponse ici" }

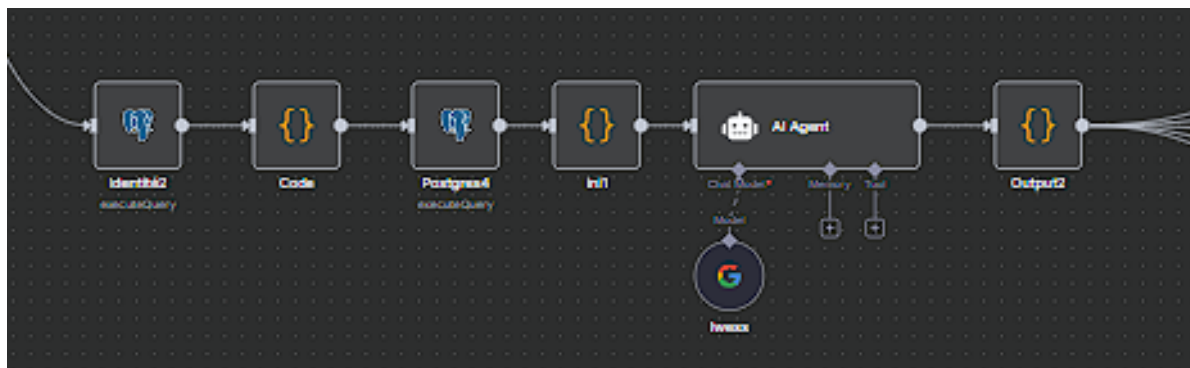


FIGURE III.19 – Block 5.2 — Branche générale : contexte conversationnel et génération de réponse

III.4.1.4 Vue Complète du Block 5 — Les Deux Branches

La figure suivante présente l'architecture complète du Block 5 avec ses deux branches de traitement parallèles, activées dynamiquement selon la décision `use_memory` issue du Block 3.

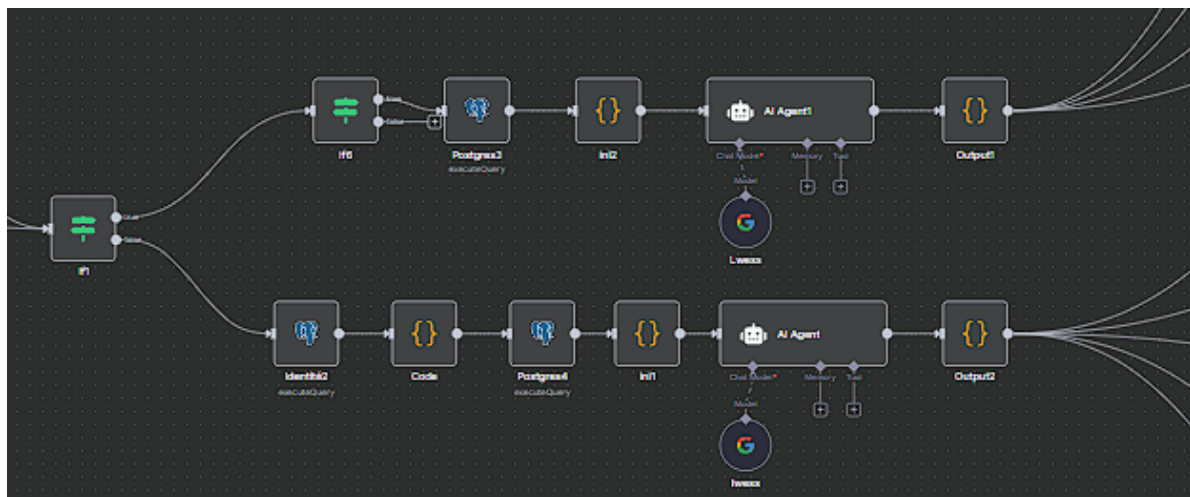


FIGURE III.20 – Block 5 — Vue complète des deux branches de traitement contextuel

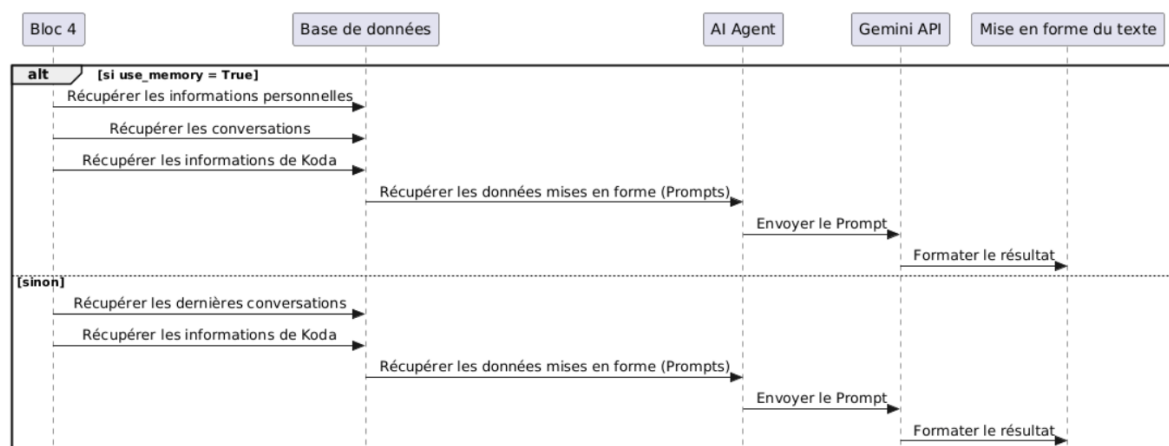


FIGURE III.21 – Block 5 — Diagramme de séquence du cinquième bloc

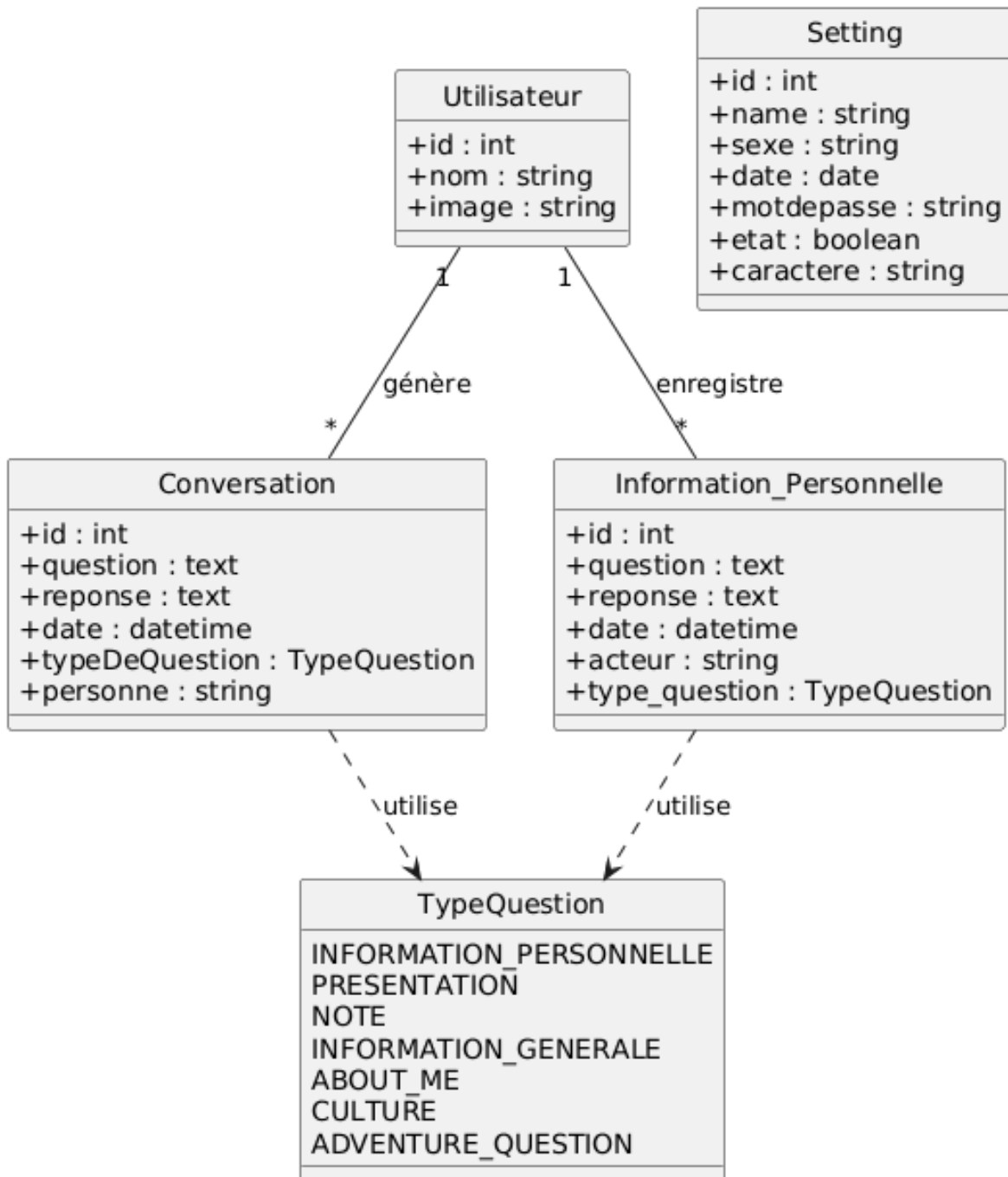


FIGURE III.22 – Block 5 — Diagramme de classes du cinquième bloc

III.4.1.4.1 Block 6 : Extraction structurée des *notes* (test final)

Lorsque le **Block 3** classe la demande avec `questiontype = note`, le pipeline ne se contente pas d'étiqueter le message : un **sous-bloc dédié** du Block 6 prend le relais **avant** toute insertion en base (Block 7). Ce scénario couvre les rappels, alarmes et rendez-vous : l'utilisateur formule une intention temporelle en langage naturel (souvent approximatif ou fautif), et le système doit en extraire une **date**, une **heure** et un **libellé d'événement** exploitables par la table `notes`.

III.4.1.4.1.1 Sous-bloc de détection (If) et Basic LLM Chain 2.

Au cœur du Block 6, un petit enchaînement n8n assure la **détection automatique** et le **traitement spécialisé** des notes :

1. un nœud **If** vérifie que le champ `questiontype` issu du Block 3 vaut bien `note` ; seules ces requêtes activent la branche d'extraction ;
2. une **Basic LLM Chain** (désignée ici *LLM Chain 2* dans le workflow) reçoit le message utilisateur, la réponse conversationnelle déjà produite en amont, l'identifiant `personne` et le type `note` ;
3. le modèle **décompose** la phrase en JSON structuré (`date_evenement`, `evenement`, éventuellement `format_valide`) **avant** l'enregistrement définitif par le Block 7.

Ainsi, une formulation du type « *Rappelle-moi le 9 heures, j'ai un rendez-vous chez le docteur* » n'est pas stockée telle quelle : le LLM normalise l'horaire et l'événement pour permettre à KODA de **prévenir l'utilisateur à un instant connu**. La figure III.23 illustre ce sous-workflow ; les figures III.24 et III.25 montrent un **exemple réel** d'entrée et de sortie JSON dans n8n.

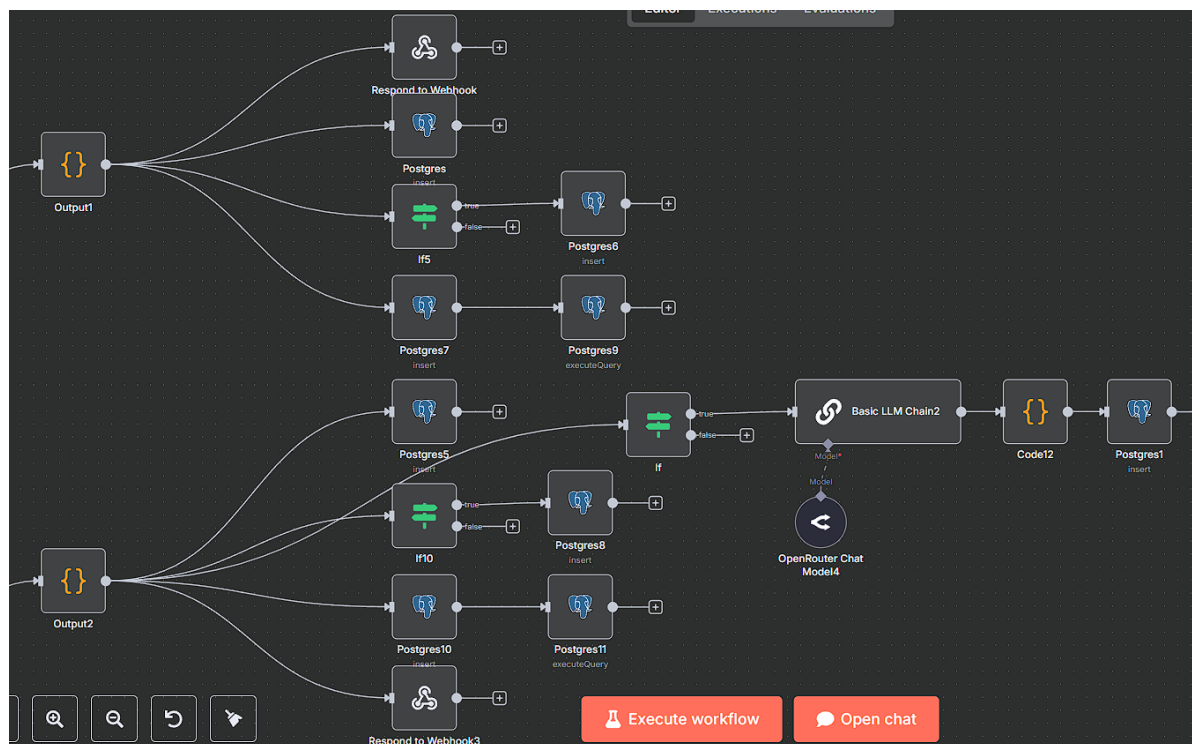


FIGURE III.23 – Block 6 — Sous-workflow n8n : test `questiontype = note`, *Basic LLM Chain 2* et structuration avant persistance

Exemple d'entrée (nœud If / LLM Chain 2).

Le panneau INPUT JSON agrège les champs transmis par les blocs précédents. Dans l'exemple ci-dessous, l'utilisateur `oussema` demande un rappel pour un rendez-vous médical ; le Block 3 a déjà fixé `questiontype` à `note`, ce qui déclenche le traitement du Block 6 :

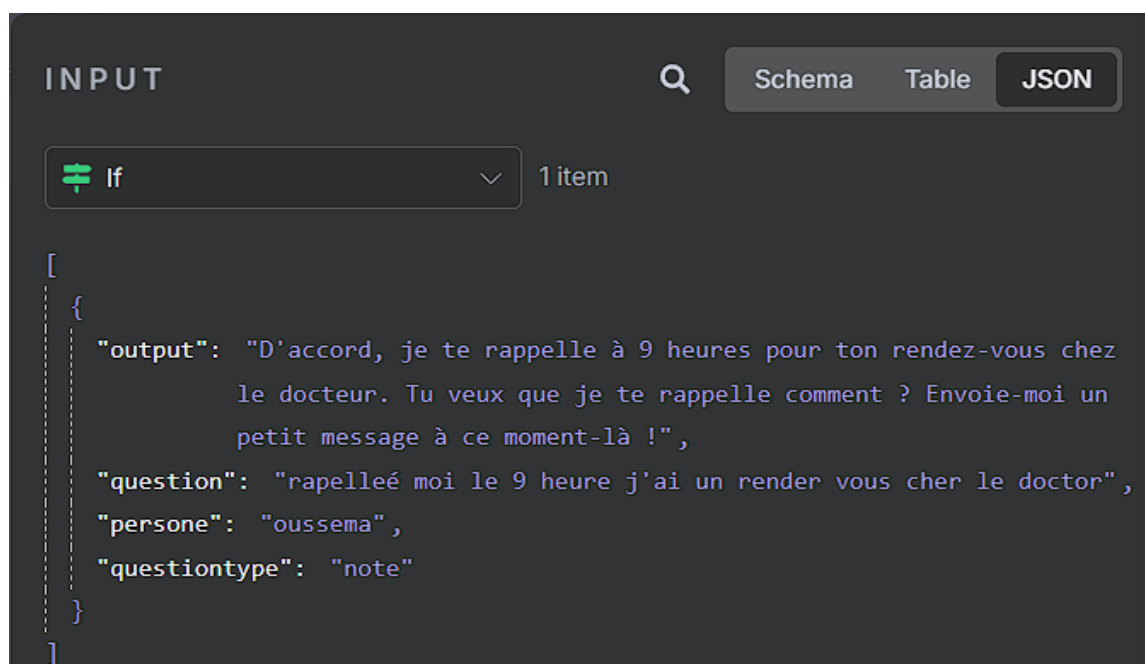


FIGURE III.24 – Block 6 — Entrée JSON : question, output, persone et questiontype = note

Exemple de sortie (structuration LLM).

La *Basic LLM Chain 2* renvoie un objet texte contenant la date-heure normalisée et l'événement reformulé, prêt pour insertion dans notes au Block 7 :

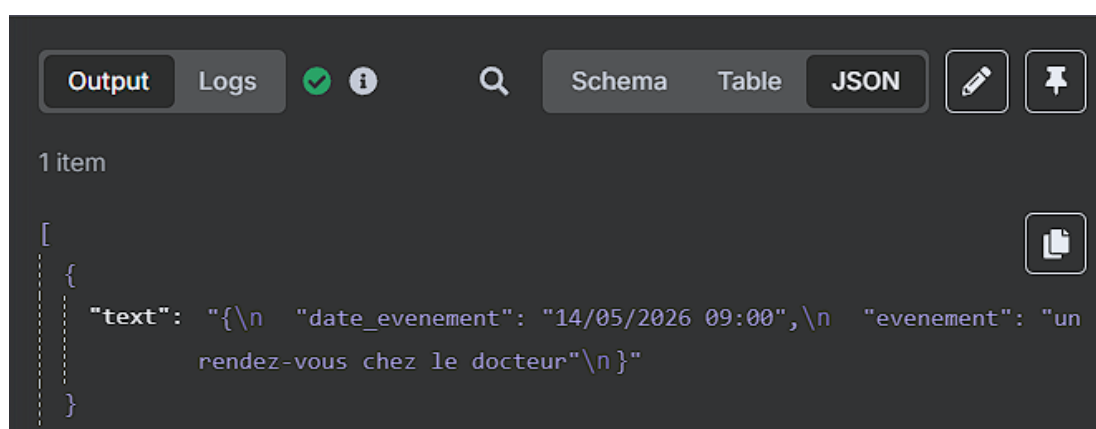


FIGURE III.25 – Block 6 — Sortie JSON : date_eventement et evenement extraits du message naturel

En résumé, le Block 6 fait le lien entre la **classification sémantique** (Block 3, type note) et la **mémoire actionnable** (Block 7) : détection automatique, extraction LLM, puis persistance des champs structurés.

III.4.1.4.2 Block 7 : Mise à Jour de Mémoire

Le Block 7 constitue la dernière étape du pipeline principal du Webhook 1. Il intervient immédiatement après la génération de la réponse par le Block 5 (et, le cas échéant, après la structuration des champs note par le Block 6), et assure la **persistance complète de l'interaction**

dans la base de données. Son rôle est essentiel : sans lui, KODA perdrait toute trace de la conversation à chaque cycle, compromettant sa capacité de personnalisation et de suivi contextuel.

Ce bloc est responsable de l'**enregistrement systématique des nouvelles données** produites ou identifiées lors du traitement de la question. Chaque branche du Block 5 (mémoire ou générale) transite par ce bloc avant la clôture du pipeline.

Fonctionnement Technique du Bloc :

Le Block 7 exécute une séquence de cinq opérations de persistance, chacune ciblant un aspect spécifique de la mémoire de KODA :

1. **Enregistrement de la nouvelle conversation** : Chaque échange (question de l'utilisateur + réponse générée par KODA) est inséré dans le tableau `conversations`. Ce tableau constitue le journal complet de toutes les interactions, permettant au Webhook 2 d'effectuer ses synthèses journalières et au Block 5 de construire le contexte conversationnel.
2. **Insertion conditionnelle dans le tableau des informations personnelles** : Si le Block 3 a classifié la question comme étant de type `about_me` ou `information_personnelle`, le système extrait les données pertinentes (préférences, centres d'intérêt, faits personnels) et les enregistre dans le tableau `personal_info`. Ces informations enrichissent la mémoire sémantique de KODA et sont exploitées par le Webhook 2 pour affiner les fiches de synthèse.
3. **Insertion conditionnelle dans le tableau des notes** : Si le type de question retourné par le Block 3 est `note`, le système identifie qu'il s'agit d'une action que l'utilisateur souhaite planifier ou mémoriser (alarme, rendez-vous, rappel, tâche à effectuer). Les champs structurés produits par le **Block 6** (`date_evenement`, `evenement`, `format_valide`, etc.) sont alors enregistrés dans le tableau `notes`, dédié au stockage des éléments actionnables liés à l'utilisateur.
4. **Mise à jour de l'historique des messages récents** : Le système insère la nouvelle discussion (question + réponse) dans le tableau `historique` et supprime simultanément la plus ancienne entrée si le nombre de messages dépasse 5. Cette rotation garantit que l'historique ne contient que les **5 derniers messages**, conformément à la logique de vérification du Block 2 qui s'appuie sur cet historique pour détecter les questions répétées.
5. **Envoi de la réponse finale** : Une fois toutes les opérations de persistance terminées, le nœud `Respond to Webhook` renvoie la réponse générée par le Block 5 à l'utilisateur via le webhook HTTP, clôturant ainsi le cycle complet de traitement.

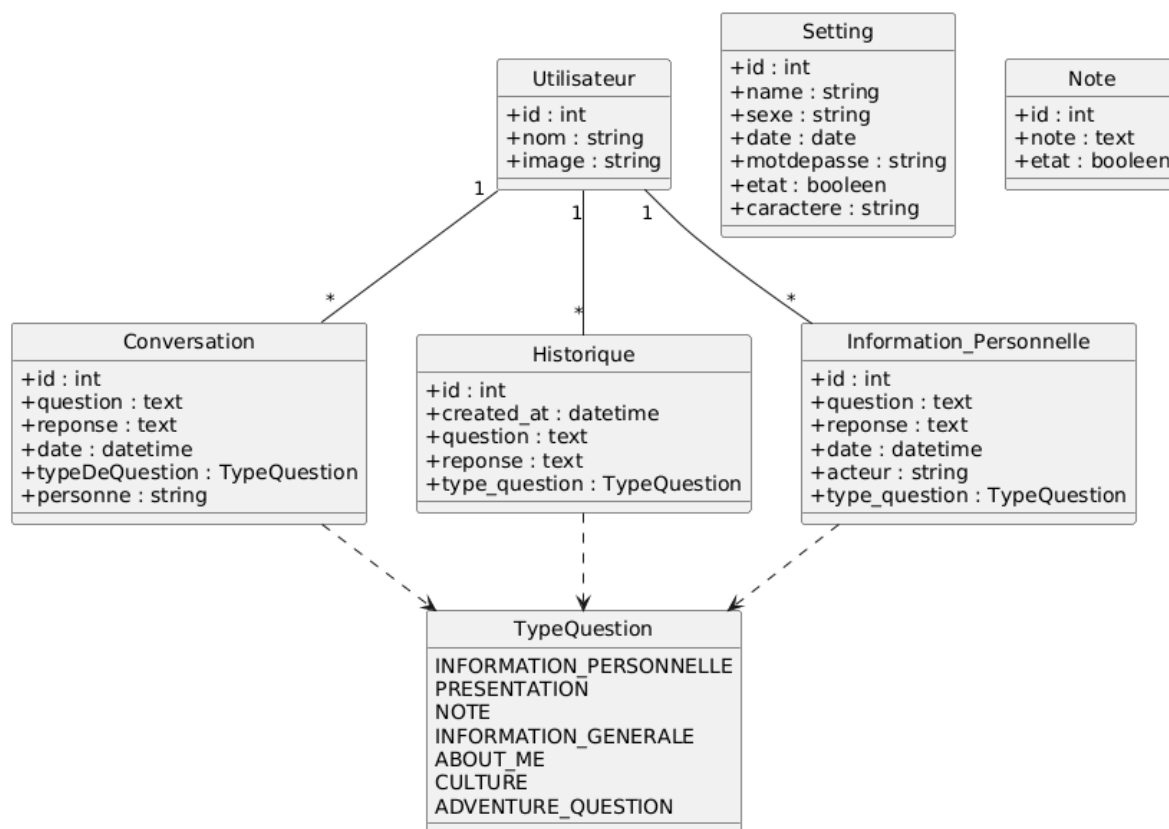


FIGURE III.26 – Block 7 — Diagramme de classes du septième bloc (persistance mémoire)

III.4.1.5 Diagramme de Séquence du Webhook 1

Le diagramme ci-dessous illustre le flux complet des données et des décisions au sein du Webhook 1, depuis la réception de la requête jusqu'à la génération de la réponse finale.

III.4.2 Deuxième Flux — Webhook 2 : Filtre et Synthèse Journalière

III.4.2.1 Principe et Inspiration

Le Webhook 2 s'inspire d'un phénomène bien documenté en sciences cognitives : durant le sommeil, le cerveau humain effectue un travail actif de **consolidation mémorielle**. Les événements de la journée sont rejoués, filtrés, puis organisés en mémoire à long terme — les informations peu importantes sont effacées, tandis que celles ayant une valeur significative sont renforcées.

Nous reproduisons ce mécanisme de manière algorithmique : le Webhook 2 s'exécute automatiquement chaque nuit (à 02h00), analyse l'ensemble des échanges de la journée pour chaque utilisateur, et en génère une **synthèse structurée** enregistrée dans la base de données.

Référence : Walker, M. & Stickgold, R. (2006). Sleep, Memory, and Plasticity. *Annual Review of Psychology*

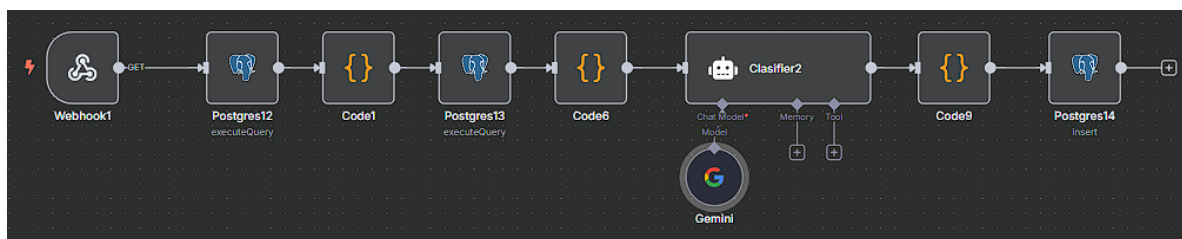


FIGURE III.27 – Vue complète du deuxième workflow

III.4.2.2 Fonctionnement du Webhook 2

Le processus se déroule en trois temps :

1. **Collecte** : Récupération de l'ensemble des nouvelles interactions de la journée pour tous les utilisateurs actifs.
2. **Analyse et synthèse** : Un agent IA analyse le contenu des échanges et génère une fiche de synthèse structurée pour chaque utilisateur.
3. **Persistance** : Les fiches sont enregistrées dans la base de données et associées au profil de chaque utilisateur.

III.4.2.3 Structure des Fiches de Synthèse Générées

À l'issue de son traitement, le Webhook 2 génère pour chaque utilisateur une fiche de synthèse couvrant **quatre dimensions complémentaires** :

- **Centres d'intérêt** : Identification des sujets qui engagent l'utilisateur, déduits des thématiques abordées dans ses échanges. Cette analyse permet de maintenir un profil d'intérêts à jour.
- **Synthèse de la journée** : Un résumé narratif des échanges significatifs de la journée. Ce résumé constitue la mémoire épisodique condensée de KODA pour cet utilisateur.
- **Points clés** : Extraction des informations factuelles importantes révélées durant la journée — décisions évoquées, projets mentionnés, problèmes soulevés. Ces éléments enrichissent la mémoire sémantique du profil.
- **Conseils personnalisés** : Génération de recommandations adaptées au profil et au contexte de l'utilisateur, permettant à KODA d'offrir une valeur proactive lors des interactions suivantes.

OUTPUT ✓ i 🔍 Schema Table JSON ✎ 📌

i To make sure expressions after this node work, return the input items that produced each output item. [More info](#)

1 item

A interets La nature, l'identité et les limites du robot (personnalité, émotions, rêves, corps, capacité à faire des erreurs, évolution). • La mémoire du robot et sa capacité à se souvenir des interactions passées et des informations sur l'utilisateur (nom, âge, entourage). • La technologie, l'intelligence artificielle et ses concepts connexes (comparaison avec d'autres IA, termes techniques, langage de programmation, outils). • Les préférences et opinions du robot (musique, sport, sujets d'actualité ou sensibles). • Des informations personnelles et académiques (PFE, études, nationalité, spécialité, etc....

A synthese L'utilisateur est manifestement **curieux** et **testeur**, explorant les capacités et les limites de l'IA, en particulier sa mémoire et sa capacité à interagir de manière humaine. Il adopte un ton globalement **amical** et cherche à établir une connexion personnalisée avec le robot. Il manifeste un intérêt prononcé pour la **technologie** et l'IA, tout en projetant des traits et des questions humaines sur le robot. Il semble être **jeune ou en formation** (références aux études, PFE, ISET) et est probablement d'origine **tunisienne** ou proche de cette culture, vu l'usage de l'arabe et du dia...

A points_cles L'utilisateur valorise la **mémoire persistante** du robot, sa capacité à le reconnaître et à retenir des informations le concernant et sur son entourage. Il accorde de l'importance à la **personnalisation de l'interaction** et à la capacité du robot à avoir une 'personnalité' ou des 'opinions'. Il est intéressé par l'**intelligence et les capacités d'apprentissage** du robot. Enfin, la **pertinence culturelle et linguistique** (arabe, tunisien) est un point d'attention pour lui, ainsi que la capacité du robot à gérer des sujets complexes ou sensibles de manière adéquate.

A conseils L'IA doit adopter un ton **amical et engageant**, répondant aux salutations et aux marques d'intérêt personnel de manière chaleureuse. Il est crucial de faire preuve d'une **mémoire contextuelle** solide, en rappelant les informations précédemment données par l'utilisateur (son nom, âge si connu, personnes mentionnées) pour renforcer la personnalisation. Pour les questions sur l'identité ou la 'personnalité' du robot, l'IA doit expliquer son fonctionnement en tant que modèle d'IA de manière **claire et pédagogique**, sans nier l'intérêt de l'utilisateur, mais en évitant de se présenter comme u...

A acteur oussema

FIGURE III.28 – Vue des outputs du 2ème workflow

Référence : Miller, G.A. (1956). The magical number seven, plus or minus two. *Psychological Review*, 63(2).

III.4.2.4 Impact sur le Système Global

Les synthèses du Webhook 2 constituent la **mémoire à long terme** de KODA. Elles sont consultées par le Webhook 1 pour enrichir le contexte des réponses, et exploitées par le Webhook 3

pour générer des interactions spontanées pertinentes.

III.4.3 Troisième Flux — Webhook 3 : Spontanéité et Interaction Proactive

III.4.3.1 Principe : La Spontanéité comme Dimension Humaine

Un compagnon humain ne se contente pas de répondre aux questions : il prend aussi l'initiative d'engager la conversation de manière spontanée et contextuelle. Les recherches en psychologie sociale ont montré que les interactions les plus mémorables et satisfaisantes sont souvent celles initiées de façon inattendue, en dehors du cadre purement transactionnel.

C'est cette dimension que le Webhook 3 cherche à reproduire. Son rôle est de permettre à KODA d'**initier lui-même la conversation** lorsqu'il détecte la présence d'un utilisateur qu'il reconnaît, sans attendre que celui-ci pose une question.

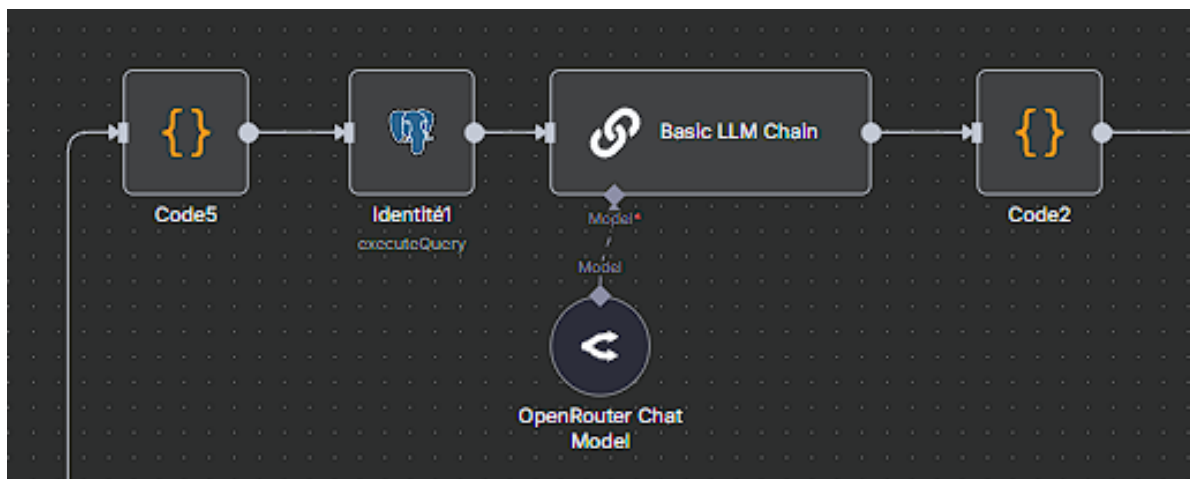


FIGURE III.29 – Vue complète du troisième workflow

Référence : Dautenhahn, K. & Billard, A. (1999). Bringing up robots or the psychology of socially intelligent

III.4.3.2 Déclenchement et Fonctionnement

Le Webhook 3 est déclenché dès que le **module de vision** de KODA identifie le visage d'un utilisateur reconnu. L'identifiant de cet utilisateur est transmis au webhook, qui active le pipeline de génération d'une phrase d'accroche personnalisée.

III.4.3.3 Génération de la Phrase d'Accroche

Un agent IA génère une **phrase d'accroche unique et personnalisée** en exploitant les données de la fiche de synthèse produite par le Webhook 2. Les variables injectées dans le prompt de génération sont les suivantes :

Variables du Prompt d'Interaction Spontanée

- {user_name} — Prénom ou identifiant de l'utilisateur
- {interests} — Centres d'intérêt identifiés (depuis le Webhook 2)
- {daily_summary} — Synthèse de la dernière journée d'échanges
- {key_points} — Points clés retenus lors des interactions précédentes
- {personalized_advice} — Conseils personnalisés disponibles
- {previous_openings} — Historique des phrases d'accroche déjà utilisées (pour garantir l'unicité)

Référence : Reeves, B. & Nass, C. (1996). *The Media Equation : How People Treat Computers, Television, and the New Media Technologies*. MIT Press.

III.5 Vue Générale du Cycle 1

Les trois webhooks forment ensemble un **système cognitif cohérent**, dont chaque composant reproduit un mécanisme humain identifié et documenté. Le tableau suivant résume ces correspondances :

TABLE III.1 – Correspondance entre mécanismes cognitifs humains et implémentations de KODA

Mécanisme Humain	Référence Scientifique	Implémentation KODA
Double traitement (S1/S2)	Kahneman, 2011	Block 3 — Classification et routage
Optimisation cognitive	Principe DRY	Block 2 — Vérification historique
Consolidation nocturne	Stickgold, 2005	Webhook 2 — Synthèse journalière
Regroupement mémoriel	Miller, 1956	4 dimensions de la synthèse Webhook 2
Spontanéité sociale	Clark, 1996 ; Dautenhahn & Billard, 1999	Webhook 3 — Interactions proactives

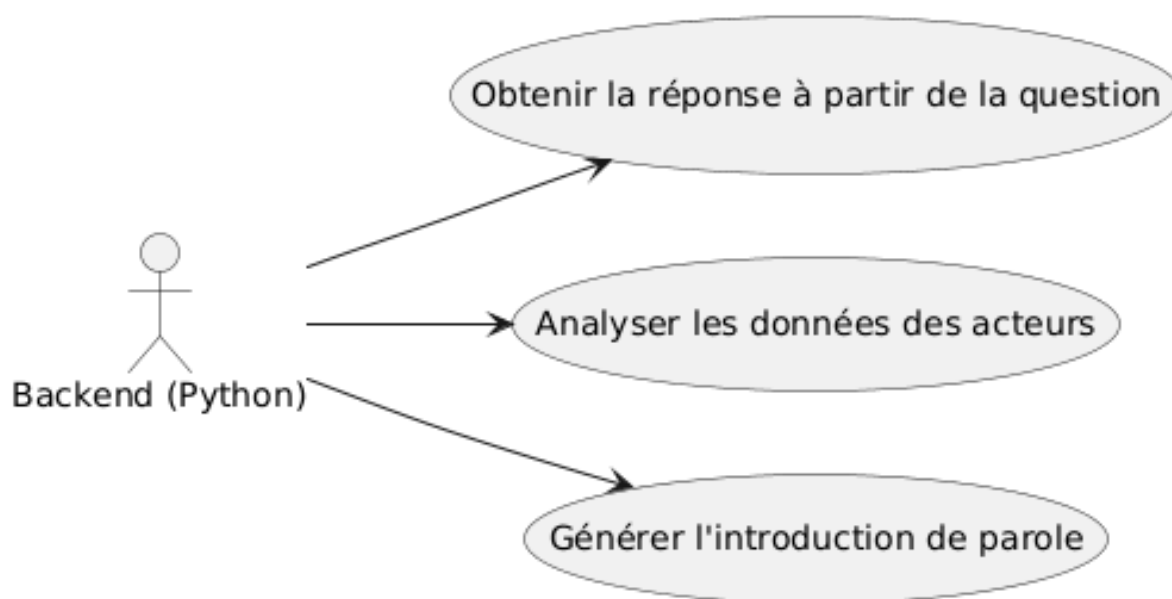


FIGURE III.30 – Diagramme de use case du 1er cycle

Référence : Russell, S. & Norvig, P. (2010). *Artificial Intelligence : A Modern Approach*, 3rd ed. Prentice Hall.

III.6 Conclusion

Ce chapitre a présenté l'architecture cognitive de KODA, structurée autour de trois webhooks complémentaires orchestrés par la plateforme n8n. Chaque composant trouve sa justification dans des principes issus des sciences cognitives et de l'intelligence artificielle.

Le **Webhook 1** assure le traitement en temps réel des questions, avec une optimisation par vérification de l'historique (Block 2) et une classification intelligente qui adapte le niveau de traitement à la nature de chaque requête. Le **Webhook 2** reproduit la consolidation nocturne du cerveau humain, en générant chaque nuit des fiches de synthèse personnalisées qui enrichissent progressivement la mémoire à long terme de KODA. Le **Webhook 3** confère au robot une dimension proactive et sociale, lui permettant d'initier des interactions personnalisées dès la détection d'un utilisateur.

Ensemble, ces trois mécanismes — réactif, mémoriel et proactif — forment la fondation de l'expérience utilisateur unique que KODA ambitionne d'offrir : celle d'un compagnon intelligent, attentionné et toujours présent.

_____ CHAPITRE IV _____

CYCLE 2 : DISTRIBUTEUR DE SERVICES

IV.1 Introduction

Dans tout écosystème logiciel distribué, il existe un composant central dont dépend la cohérence et la fluidité de l'ensemble. Dans notre projet de robot intelligent, ce rôle est assuré par le **Distributeur de Services** : l'application backend. Ce composant constitue le cœur névralgique de l'architecture ; il est responsable de la réception, du traitement et de la redistribution de toutes les requêtes émanant des différents acteurs du système.

Conformément au **modèle en spirale** adopté dans ce rapport, ce **deuxième** cycle se concentre exclusivement sur ce composant. Chaque cycle vise à livrer un composant fonctionnel, testable et intégrable de manière incrémentale ; le Distributeur de Services représente ainsi le cycle le plus complexe et le plus structurant du projet.

Le backend assume plusieurs responsabilités simultanément : il joue le rôle d'intermédiaire entre la base de données PostgreSQL hébergée sur Supabase et les autres composants ; il achemine les flux de données vers le moteur d'automatisation n8n ; il expose les services nécessaires à l'application mobile multiplateforme ; et il assure l'interface temps réel avec le système embarqué du robot physique. Il intègre également des capacités avancées telles que la reconnaissance de personnes, la synthèse vocale et la transcription audio via les services Azure Cognitive Services et l'API Gemini.

Rôle principal du Distributeur de Services

Centraliser et redistribuer l'ensemble des communications entre tous les composants de l'écosystème (robot physique, workflow n8n, application mobile, base de données, services cloud), tout en garantissant la sécurité, la fiabilité et la scalabilité du système.

IV.2 Architecture Utilisée

IV.2.1 Choix du langage : Python

Le choix de Python comme langage de développement du backend résulte d'une analyse rigoureuse des besoins fonctionnels et techniques du projet. Trois raisons majeures ont orienté cette décision.

Premièrement, Python dispose d'un écosystème exceptionnel pour les domaines mobilisés par notre projet : intelligence artificielle, traitement du signal audio, vision par ordinateur et communications temps réel. Des bibliothèques telles que `face_recognition`, NumPy ou les SDK officiels d'Azure et de Google (Gemini) sont nativement disponibles et parfaitement documentées.

Deuxièmement, le framework **FastAPI** — retenu pour la construction des endpoints REST — est entièrement basé sur Python. Il offre des performances comparables à Node.js grâce à son

support natif de l’asynchronisme (`asyncio`), tout en permettant la génération automatique de la documentation Swagger et la validation des données via Pydantic.

Troisièmement, la communauté Python est l’une des plus actives du monde open-source, ce qui garantit une maintenance facilitée, une abondance de solutions aux problèmes rencontrés et une meilleure pérennité du projet.

TABLE IV.1 – Comparaison des technologies backend envisagées

Critère	Python + FastAPI	Node.js + Express	Java Spring Boot
Bibliothèques IA/ML	✓ Excellent	~ Limité	~ Limité
Performance async	✓ <code>asyncio</code>	✓ Natif	✓ Natif
Intégration Azure SDK	✓ Officiel	✓ Officiel	✓ Officiel
Facilité de développement	✓ Très rapide	✓ Rapide	~ Verbeux
Reconnaissance visage	✓ <code>face_recog</code>	× Inexistant	~ Complexe
Déploiement Docker	✓ Simple	✓ Simple	~ Lourd

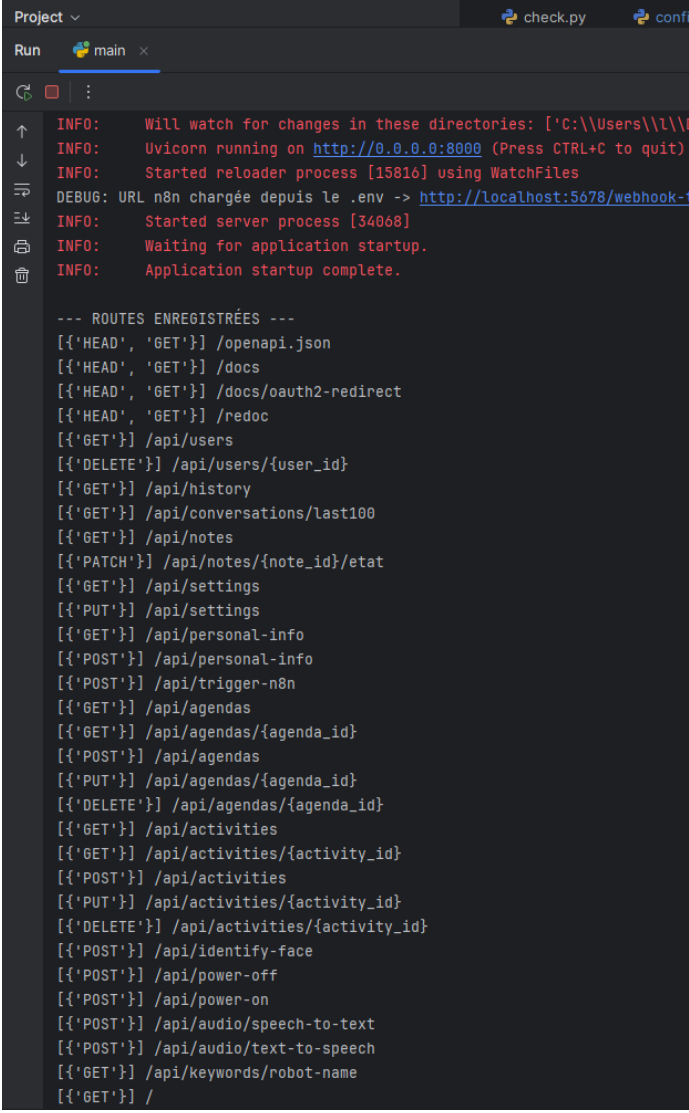
IV.2.2 Clean Architecture : principes et application

La **Clean Architecture**, théorisée par Robert C. Martin (*Uncle Bob*), repose sur un principe fondamental : séparer les préoccupations en couches concentriques indépendantes, de sorte que les règles métier du domaine ne dépendent d’aucune technologie externe. Cette indépendance garantit la testabilité, la maintenabilité et l’évolutivité du système.

Dans la pratique, le backend est aussi pensé selon l’**architecture en oignon** (*Onion Architecture*, Palermo) : le **domaine** occupe le centre, enveloppé par les **services applicatifs**, puis par les **interfaces** vers l’extérieur, et enfin par l’**infrastructure** (web, persistance, SDK). L’oignon et la Clean Architecture partagent la même règle d’or : *aucune couche intérieure ne doit importer une couche extérieure*. Les noms de couches ci-dessous s’alignent sur cette lecture « anneaux » du code du Distributeur.

Notre backend est structuré autour de quatre couches principales, chacune dotée d’un rôle précis et de dépendances strictement orientées vers l’intérieur.

La couche **Entities** contient les objets métier purs : `Person`, `Session`, `Command` et `Event`. Ces entités n’ont aucune dépendance externe et représentent les règles métier les plus stables du système. La couche **Application Use Cases** orchestre les scénarios fonctionnels : reconnaissance de personnes, traitement des commandes robot et gestion des sessions de communication. La couche **Interface Adapters** assure la traduction entre le monde extérieur et les cas d’usage internes (routeurs, dépôts, sérialiseurs). Enfin, la couche **Frameworks & Drivers** regroupe tous les détails d’implémentation : FastAPI, les connecteurs Supabase, les SDK Azure et Gemini.



```

Project ▾
Run main x
INFO: Will watch for changes in these directories: ['C:\\Users\\l\\v\\
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [15816] using WatchFiles
DEBUG: URL n8n chargée depuis le .env -> http://localhost:5678/webhook-
INFO: Started server process [34068]
INFO: Waiting for application startup.
INFO: Application startup complete.

--- ROUTES ENREGISTRÉES ---
[{'HEAD', 'GET'}] /openapi.json
[{'HEAD', 'GET'}] /docs
[{'HEAD', 'GET'}] /docs/oauth2-redirect
[{'HEAD', 'GET'}] /redoc
[{'GET'}] /api/users
[{'DELETE'}] /api/users/{user_id}
[{'GET'}] /api/history
[{'GET'}] /api/conversations/last100
[{'GET'}] /api/notes
[{'PATCH'}] /api/notes/{note_id}/etat
[{'GET'}] /api/settings
[{'PUT'}] /api/settings
[{'GET'}] /api/personal-info
[{'POST'}] /api/personal-info
[{'POST'}] /api/trigger-n8n
[{'GET'}] /api/agendas
[{'GET'}] /api/agendas/{agenda_id}
[{'POST'}] /api/agendas
[{'PUT'}] /api/agendas/{agenda_id}
[{'DELETE'}] /api/agendas/{agenda_id}
[{'GET'}] /api/activities
[{'GET'}] /api/activities/{activity_id}
[{'POST'}] /api/activities
[{'PUT'}] /api/activities/{activity_id}
[{'DELETE'}] /api/activities/{activity_id}
[{'POST'}] /api/identify-face
[{'POST'}] /api/power-off
[{'POST'}] /api/power-on
[{'POST'}] /api/audio/speech-to-text
[{'POST'}] /api/audio/text-to-speech
[{'GET'}] /api/keywords/robot-name
[{'GET'}] /

```

Figure : Exécution de Programme (Distributeur)

IV.3 Fonctionnalités Techniques

IV.3.1 Communication avec la base de données (Supabase / PostgreSQL)

La couche de persistance du projet repose sur **PostgreSQL**, déployé sur **Supabase**. Dans un cadre académique de type PFE, ce choix est pertinent car il associe la fiabilité d'un SGBD relationnel mature à un environnement cloud structuré, facilitant l'administration, la supervision et la maintenance des données.

Supabase est une plateforme Backend-as-a-Service (BaaS) open source construite autour de PostgreSQL. Elle fournit des services intégrés tels qu'une interface d'administration, un éditeur SQL, des API automatiques et des mécanismes de gestion des accès.

Le backend Python communique avec la base via SQLAlchemy (modélisation ORM) et asyncpg (accès asynchrone), afin d'assurer une exécution efficace des opérations de lecture et d'écriture, y

compris en situation de charge concurrente.

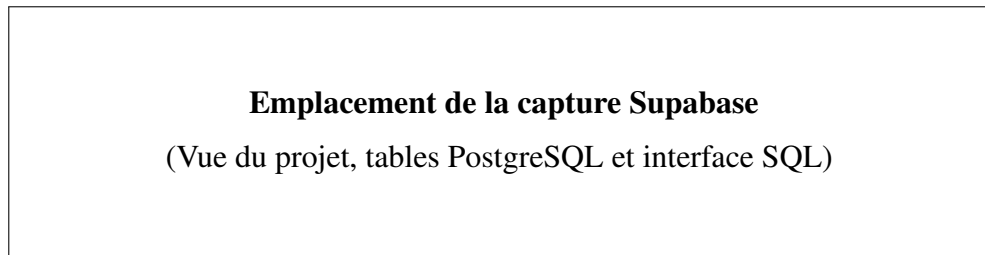


FIGURE IV.1 – Interface Supabase utilisée pour l’administration de la base PostgreSQL

Du point de vue applicatif, la communication suit une chaîne claire : *API REST* → *couche service* → *repository* → *PostgreSQL*. Cette séparation réduit le couplage entre la logique métier et la persistance, facilite les tests unitaires et améliore la maintenabilité globale de l’architecture.

Pour renforcer la qualité de service, plusieurs mécanismes sont appliqués : validation des entrées, gestion des exceptions SQL, journalisation des erreurs critiques et contrôle de latence sur les opérations sensibles. Cette approche permet d’assurer l’intégrité des données et la continuité de fonctionnement du robot, y compris lors de charges variables.



FIGURE IV.2 – Logo SupaBase

IV.3.1.1 *Modèle relationnel et rôle des tables*

Le modèle de données est conçu pour couvrir les dimensions conversationnelles, contextuelles et organisationnelles du système. Les principales tables et leurs rôles sont les suivants :

- **conversation** (**id, question, reponce, date, type_de_question, personne**) : archive l’ensemble des conversations entre le robot et l’humain.
- **historique** (**id, date, question, reponce, type_de_question**) : conserve les 5 derniers messages pour la mémoire contextuelle immédiate.
- **information_personnel** (**id, question, reponce, date, acteur, type_de_question**) : stocke les informations personnelles liées à l’humain ou au robot.
- **note** (**id, note, etat**) : enregistre les rendez-vous et dates importantes de l’utilisateur.

- **setting** (**id**, **name**, **sexe**, **date**, **motdepasse**, **etat**, **caractere**, **langue**) : centralise l'identité et les paramètres généraux de KODA.
- **utilisateur** (**id**, **nom**, **image_url**) : référence les personnes connues par KODA avec leur nom et leur image.
- **agenda** (**id**, **titre**, **note**, **etat**, **contenu**, **date_modification**) : enregistre les éléments d'agenda saisis depuis l'application mobile.
- **activity** (**id**, **titre**, **description**, **etat**, **date**, **createdat**) : stocke les activités de l'utilisateur provenant de l'application mobile.
- **accompagnement** (**id**, **interets**, **synthese**, **point_cles**, **conseils**, **acteur**) : conserve l'analyse quotidienne consolidée pour chaque personne connue.

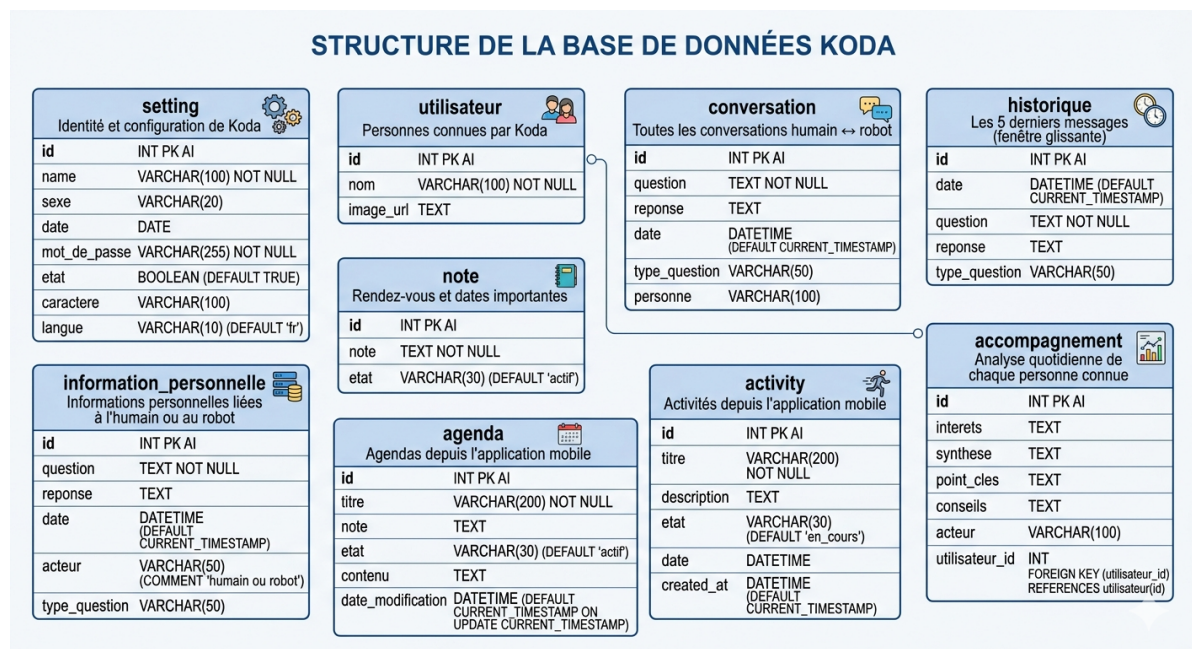


FIGURE IV.3 – Koda Schema Base De Donnée

IV.3.2 Services Azure : Speech-to-Text et Text-to-Speech

L'intégration des services cognitifs Azure permet au robot de comprendre la parole humaine et de restituer une réponse vocale naturelle. Deux services sont exploités : **Azure Speech-to-Text (STT)** pour la transcription de la voix vers le texte, et **Azure Text-to-Speech (TTS)** pour la synthèse vocale de la réponse générée par le système.

Principe de communication technique

La communication avec Azure est initiée par le backend Python, qui joue le rôle de passerelle unique entre le robot et les services cloud. Le flux est le suivant :

1. capture du flux audio côté robot et envoi au backend ;

2. transmission de l’audio vers Azure STT pour transcription ;
3. réception du texte reconnu et traitement conversationnel ;
4. envoi du texte de réponse vers Azure TTS ;
5. récupération de l’audio de synthèse et retour vers le robot.

Cette architecture centralisée simplifie la sécurité (aucune clé Azure exposée sur le robot) et garantit une meilleure traçabilité des appels.

Configuration Azure (STT/TTS)

La configuration opérationnelle est réalisée dans le portail Azure selon les étapes suivantes :

- création d’une ressource **Speech** dans *Azure AI Services* ;
- récupération de la **clé d’accès** et de la **région** de déploiement ;
- injection de ces paramètres dans le backend via des variables d’environnement (ex. : `AZURE_SPEECH_KEY`, `AZURE_SPEECH_REGION`) ;
- sélection des paramètres vocaux : langue, voix neurale, débit et tonalité de parole.

Le backend charge ces paramètres au démarrage, initialise les clients STT/TTS, puis réutilise la configuration pour l’ensemble des requêtes. Cette approche évite la duplication de code et stabilise les temps de réponse.

Aspects d’exploitation

Pour obtenir un comportement robuste en production, plusieurs mécanismes sont appliqués : appels asynchrones, contrôle de timeout, gestion des exceptions réseau et journalisation des échecs. Les voix *Neural TTS* sont utilisées pour améliorer la qualité perceptive, tandis que le mode STT en flux continu (*streaming*) réduit la latence de reconnaissance.

L’**API Gemini** de Google intervient après la transcription STT pour produire une réponse contextuelle. Le texte généré est ensuite transmis à Azure TTS, puis restitué au robot sous forme de message audio.

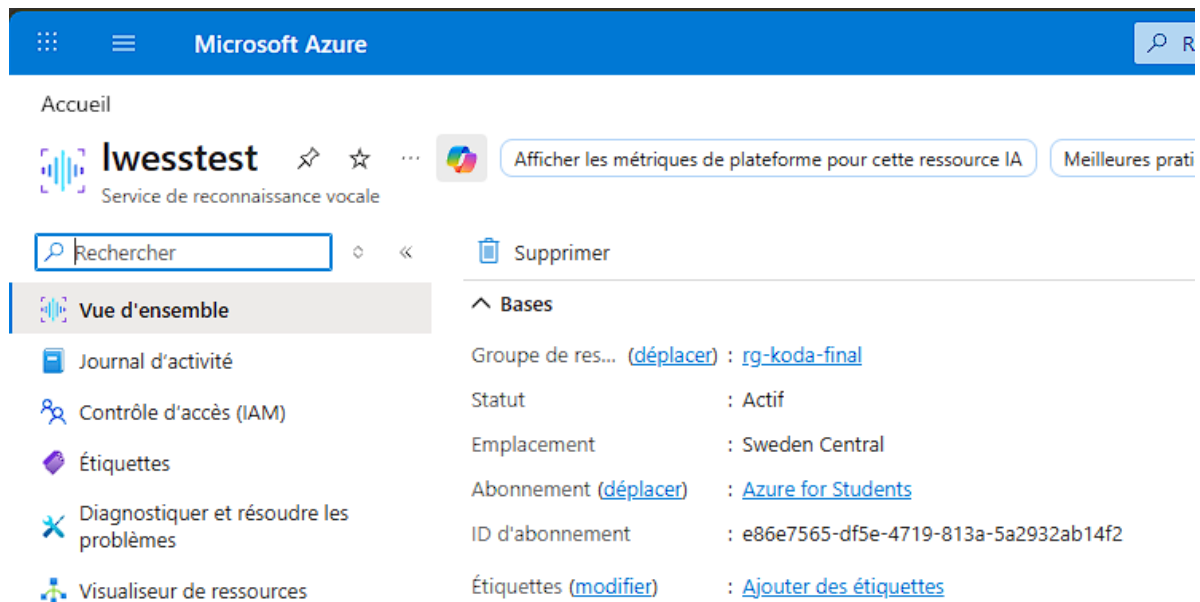


FIGURE IV.4 – Capture Azur Configuration

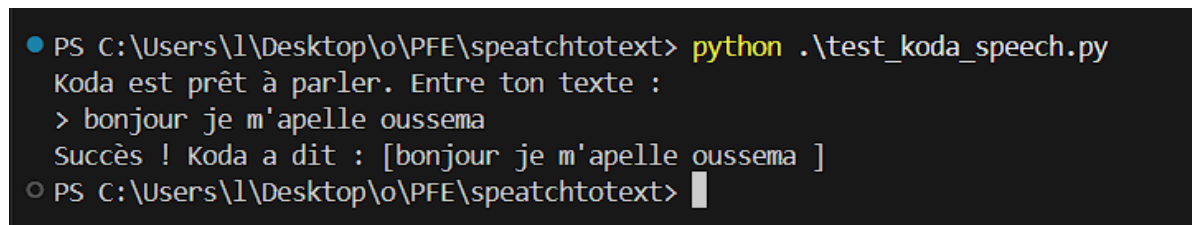


FIGURE IV.5 – Capture Test Text To Speech

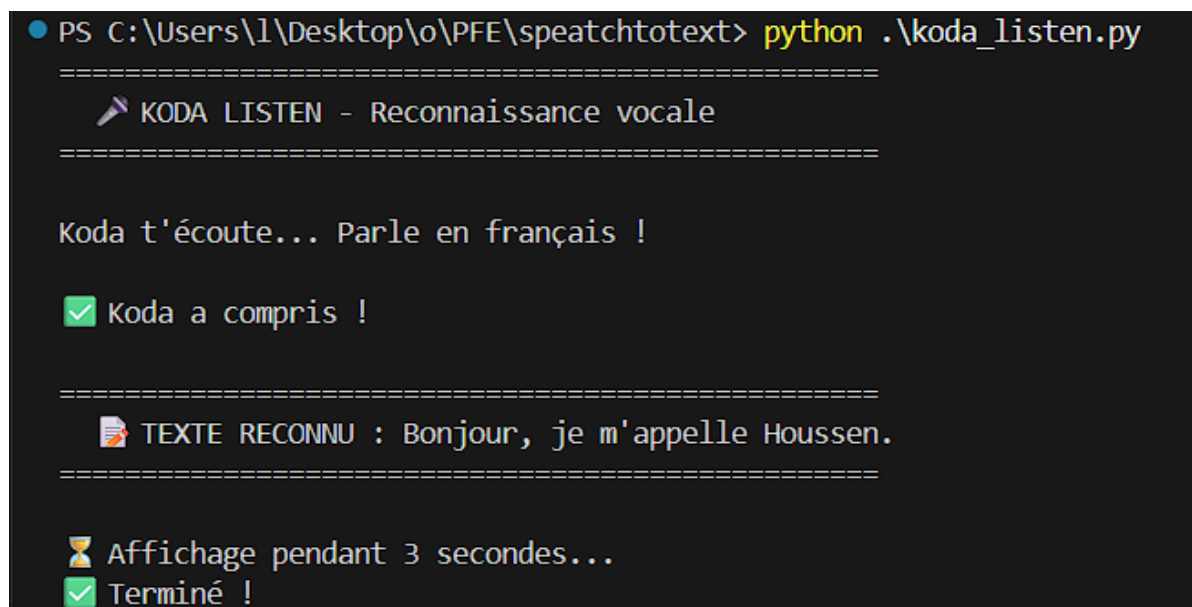


FIGURE IV.6 – Capture Test Speech To Text

IV.3.3 Service de reconnaissance des personnes

Le moteur de **reconnaissance faciale** est l'un des services les plus innovants du backend. Il permet au robot d'identifier en temps réel les personnes qu'il rencontre, lui conférant une capacité d'interaction personnalisée et contextuelle. Il s'appuie sur la bibliothèque Python `face_recognition`,

qui utilise en interne des réseaux de neurones profonds (dlib) pour générer des encodages faciaux — vecteurs de 128 dimensions — à partir d’images.

Le processus se déroule en deux phases. Lors de **l’enregistrement**, un profil est créé pour chaque personne autorisée : plusieurs photos sont traitées, les encodages sont calculés et persistés en base de données. Lors de la **reconnaissance**, l’image capturée par la caméra du robot est transmise au backend ; son encodage est calculé, puis comparé aux encodages stockés par distance euclidienne (seuil configurable). La décision d’identification est renvoyée au robot accompagnée du niveau de confiance.



FIGURE IV.7 – Exemple de image dans la memoire

Précision du système de reconnaissance

Lors des tests en conditions réelles (variété d'éclairage, angles de vue différents), le service a atteint un taux de reconnaissance supérieur à **94 %** avec un seuil de distance fixé à 0,5. Le temps de traitement moyen par image est de **180 ms** sur le serveur de déploiement.

IV.3.4 Interface avec le workflow n8n

Le moteur de workflow **n8n** joue le rôle d'orchestrateur cognitif du système : il reçoit les requêtes, applique la logique de traitement et renvoie une réponse structurée au backend. Dans notre architecture, la communication entre Python et n8n est fondée sur un échange HTTP de type **requête/réponse** via webhook.

Rôle de cette communication

L'objectif principal est de déléguer à n8n la logique conversationnelle et décisionnelle, tout en conservant dans le backend Python les fonctions techniques (sécurité, accès base de données, services matériels). Cette séparation permet de modifier rapidement le comportement métier dans n8n sans impacter la couche d'exposition API.

Flux détaillé : de la requête à la réponse

Le scénario standard suit les étapes suivantes :

1. le backend récupère le **nom de l'utilisateur** (ou son identifiant) et la **question** formulée ;
2. il construit un payload JSON normalisé (ex. : `{"name": "...", "question": "..."};`);
3. ce payload est envoyé au webhook n8n via une requête HTTP POST ;
4. n8n exécute le workflow correspondant : enrichissement du contexte, appels de services, règles de décision et génération de réponse ;
5. n8n retourne un objet de sortie contenant la **réponse finale** ;
6. le backend valide le format, journalise l'échange et renvoie la réponse au robot ou à l'application cliente.

Sur le plan qualité, cette interface inclut la gestion des *timeouts*, le contrôle des erreurs de webhook et la validation du schéma de réponse pour garantir la robustesse en exploitation.

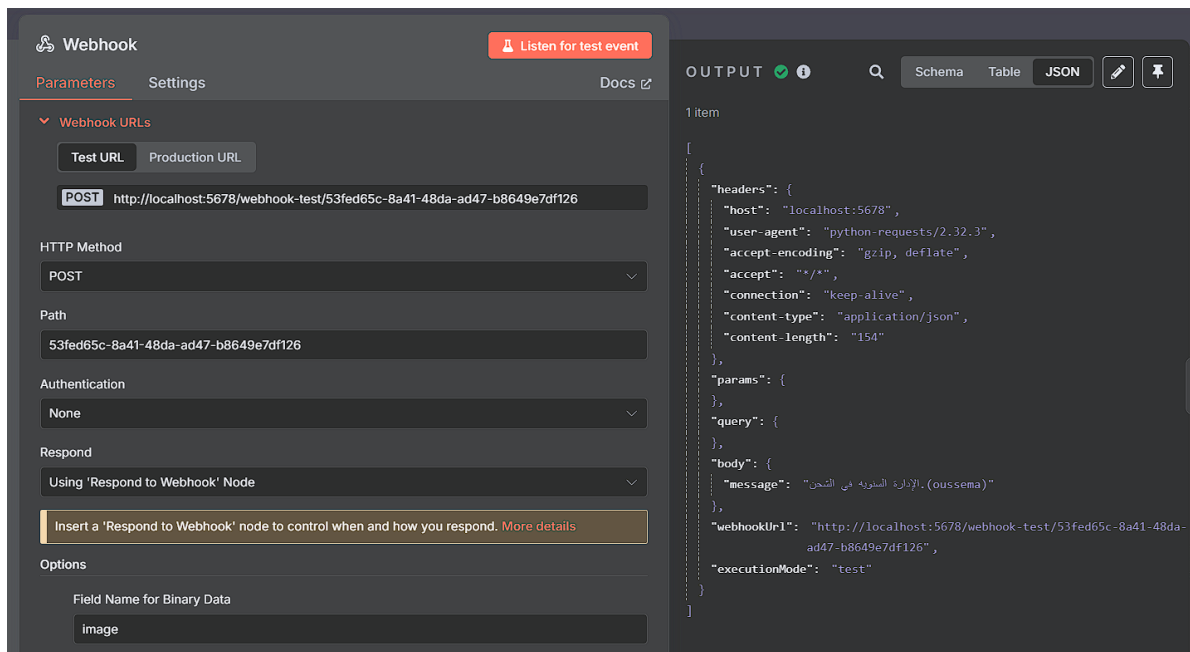


FIGURE IV.8 – Capture Input WorkFlow (N8N)

IV.3.4.0.1 Images recommandées pour cette section

- **Schéma de séquence Python** $\leftrightarrow$ **n8n** : nom + question $\rightarrow$ webhook $\rightarrow$ réponse.
- **Capture du webhook n8n** : nœud d'entrée avec exemple de payload JSON.
- **Capture du workflow n8n complet** : principales étapes (contexte, traitement, réponse).
- **Capture de la sortie JSON** : réponse retournée par n8n et exploitée par le backend.

IV.3.5 Services pour l'application mobile multiplateforme

L'application mobile développée avec **Flutter** consomme un ensemble de services REST exposés par le backend, couvrant la totalité des fonctionnalités accessibles depuis l'interface utilisateur : authentification et gestion des profils, visualisation des journaux d'interaction, contrôle du robot en temps réel, gestion des personnes enregistrées et consultation des rapports d'événements.

La sécurisation de ces endpoints repose sur des **JWT (JSON Web Tokens)** émis par Supabase Auth. Chaque requête mobile inclut un Bearer Token validé côté backend avant tout traitement. Un middleware FastAPI dédié gère l'authentification de manière transversale, sans dupliquer la logique dans chaque route.

API ENDPOINTS LIST



UTILISATEURS & CONVERSATIONS

[GET] /api/users
[DELETE] /api/users/{user_id}

HISTORIQUE, NOTES & PARAMÈTRES

[GET] /api/history [GET] /api/settings
[GET] /api/conversations/last100 [PATCH] /api/notes/{note_id}/etat
[GET] /api/notes [PUT] /api/settings

INFO PERSONNELLE, N8N & AGENDAS

[GET] /api/personal-info [GET] /api/agendas [POST] /api/agendas
[POST] /api/personal-info [GET] /api/agendas/{agenda_id} [PUT] /api/agendas/{agenda_id}
[POST] /api/trigger-n8n [PUT] /api/agendas/{agenda_id} [DELETE] /api/agendas/{agenda_id}

ACTIVITÉS & OPÉRATIONS SYSTÈME

[GET] /api/activities [PUT] /api/activities [POST] /api/identify-face
[GET] /api/activities/{activity_id} [DELETE] /api/activities [POST] /api/power-off
[POST] /api/activities [POST] /api/posser [POST] /api/power-on
[POST] /api/activities/{activity_id} [POST] /api/audio/speech-to-text
[POST] /api/activities/{activity_id} [POST] /api/audio/text-to-speech

Figure : Liste des endpoints de communication avec l'application mobile cross-plateforme

```
PS C:\Users\I\Desktop\o\PFE\Backend + application mobile> curl.exe -X GET "http://127.0.0.1:8000/api/users" -H
"accept: application/json"
[[{"id": "98a327fd-0ece-4afc-8b06-5a6499a9ce04", "nom": "salah", "image": "https://jynducbyosfpdetdomga.supabase.co/
storage/v1/object/public/photos_utilisateurs/s1.jpg"}, {"id": "d58ff93a-26d1-4785-867d-4934351cb4e2", "nom": "sala
h", "image": "https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/s2.jpg"}, {"i
d": "798bd831-0837-4c6c-93a1-f9654f22d074", "nom": "salah", "image": "https://jynducbyosfpdetdomga.supabase.co/stor
age/v1/object/public/photos_utilisateurs/s3.jpg"}, {"id": "ec7abfae-1460-473e-a5ab-c7dd65f091ea", "nom": "salah", "
image": "https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/s4.jpg"}, {"id": "
2a6df721-a61d-42e7-9ee3-f0a9e9acf756", "nom": "salah", "image": "https://jynducbyosfpdetdomga.supabase.co/storage/
v1/object/public/photos_utilisateurs/s5.jpg"}, {"id": "77af3963-e661-423f-8b48-5a70f79cf668", "nom": "salah", "imag
e": "https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/s6.jpg"}, {"id": "b788
4803-0039-474f-9c99-eb44d7a07063", "nom": "salah", "image": "https://jynducbyosfpdetdomga.supabase.co/storage/v1/o
bject/public/photos_utilisateurs/s7.jpg"}, {"id": "0c8f6a3c-8a3b-45e8-ac7e-191c1ee861bd", "nom": "oussema", "image"
: "https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/o1.jpg"}, {"id": "ac2884
e6-b761-405a-a2a9-93f4822c52e5", "nom": "oussema", "image": "https://jynducbyosfpdetdomga.supabase.co/storage/v1/o
bject/public/photos_utilisateurs/o2.jpg"}, {"id": "6360315d-0c7d-4194-a518-393018d183e7", "nom": "oussema", "image"
: "https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/o3.jpg"}, {"id": "e2e5ba
d9-2b00-42c6-b59d-392cf9c10673", "nom": "oussema", "image": "https://jynducbyosfpdetdomga.supabase.co/storage/v1/o
bject/public/photos_utilisateurs/o4.jpg"}, {"id": "de05ab3-4d9c-4aac-b26e-de47c141c22a", "nom": "oussema", "image"
: "https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/o5.jpg"}, {"id": "51b91c
f9-8512-42d7-84e2-efe9d043bef5", "nom": "oussema", "image": "https://jynducbyosfpdetdomga.supabase.co/storage/v1/o
bject/public/photos_utilisateurs/o6.jpg"}, {"id": "38e05d5f-149f-444f-874e-9ede96e85696", "nom": "oussema", "image"
: "https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/o7.jpg"}, {"id": "912fb4
8b-dc88-4314-911d-e5896de7f939", "nom": "oussema", "image": "https://jynducbyosfpdetdomga.supabase.co/storage/v1/o
bject/public/photos_utilisateurs/o8.jpg"}, {"id": "8a0deca6-72fe-48ce-a2af-2802c6cc90db", "nom": "oussema", "image"
: "https://jynducbyosfpdetdomga.supabase.co/storage/v1/object/public/photos_utilisateurs/o9.jpg"}]]
PS C:\Users\I\Desktop\o\PFE\Backend + application mobile> curl.exe -X GET "http://127.0.0.1:8000/api/agendas"
-H "accept: application/json"
[[{"id": 1, "titre": "aaaa", "note": "aaaa", "etat": true, "contenu": "aaaa", "date_modification": "2026-04-12T16:37:12.74
2374+00:00"}]]
PS C:\Users\I\Desktop\o\PFE\Backend + application mobile> curl.exe -X POST "http://127.0.0.1:8000/api/power-on"
-H "accept: application/json"
{"status": "power_on", "etat": 1}
PS C:\Users\I\Desktop\o\PFE\Backend + application mobile> curl.exe -X GET "http://127.0.0.1:8000/test"
{"message": "Test OK"}
```

Figure : Exécution Quelques EndPoints

TABLE IV.2 – Endpoints exposés à l'application mobile

Endpoint	Méthode	Description

IV.3.6 Communication avec le système embarqué (robot réel)

L'interface avec le robot physique constitue l'une des parties les plus sensibles du backend. Le robot embarque un microcontrôleur connecté en permanence via **WebSocket**, permettant une communication bidirectionnelle, asynchrone et à faible latence. Le backend maintient une connexion persistante avec le robot et achemine les commandes en temps réel.

Chaque commande est préalablement sérialisée au format JSON, journalisée dans la table `commands` avec le statut « en attente », puis transmise via WebSocket. Un système d'acquittement (ACK) permet au backend de confirmer la réception et l'exécution de la commande par le robot, mettant à jour le statut en conséquence. En l'absence de réponse dans un délai configurable, un mécanisme de *retry* est déclenché automatiquement.

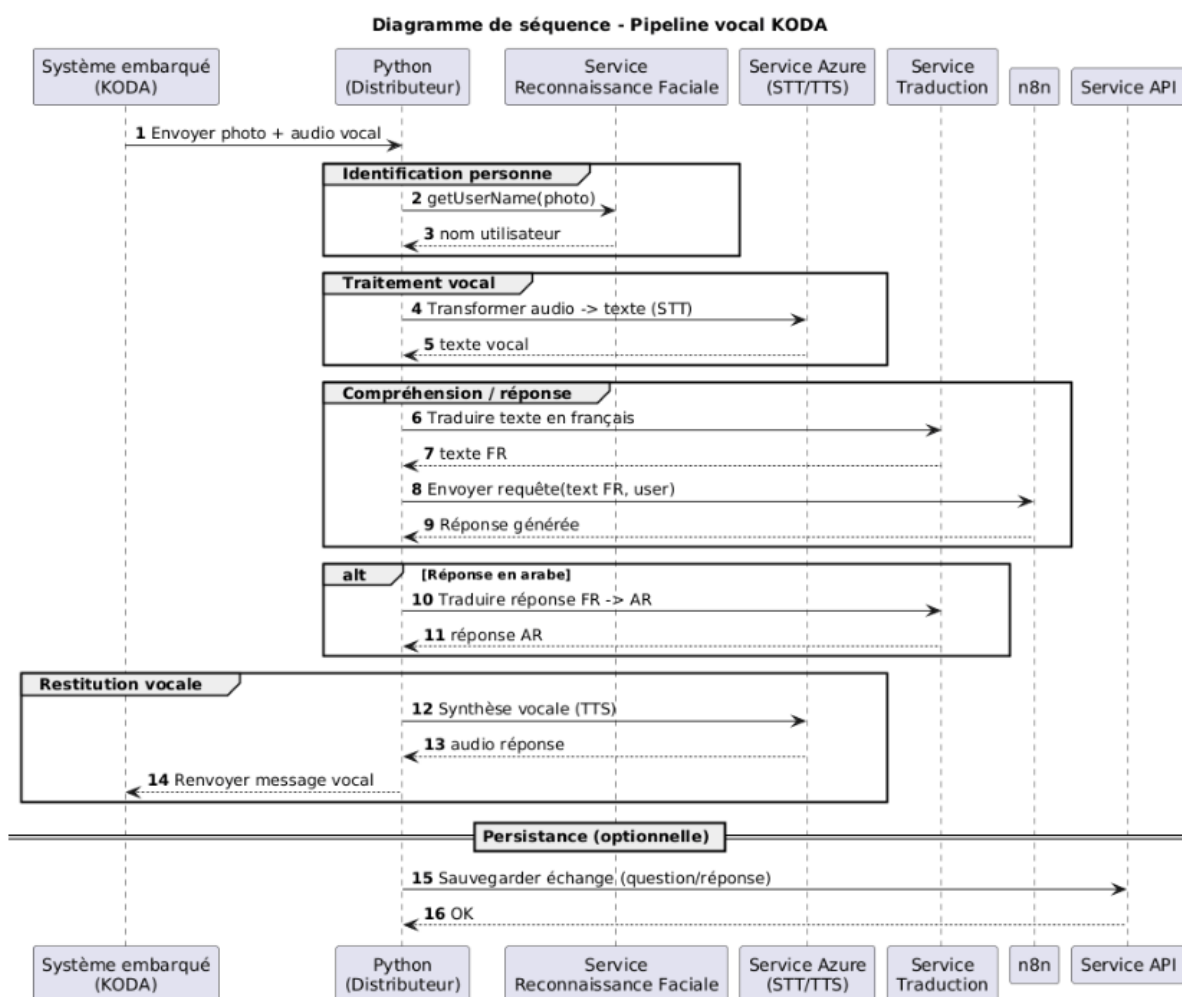


Figure : Diagramme de séquence de la communication avec le système embarqué

IV.4 Problèmes Rencontrés et Solutions Apportées

Le développement du backend a été ponctué de plusieurs défis techniques significatifs. Le tableau IV.3 en présente les principaux, accompagnés des solutions retenues.

TABLE IV.3 – Problèmes techniques rencontrés et solutions apportées

Problème rencontré	Impact	Solution adoptée
Latence élevée de la reconnaissance faciale sous charge	Blocage du thread principal FastAPI	Utilisation de <code>run_in_executor</code> pour déléguer le traitement au <code>ThreadPoolExecutor</code>
Déconnexions intempestives du WebSocket robot	Perte de commandes, état incohérent	Reconnexion automatique avec <i>backoff</i> exponentiel
Conflits de concurrence sur la base de données	Doublons de sessions et d'événements	Transactions atomiques et <code>SELECT FOR UPDATE</code>
Taille des encodages faciaux (128 flottants / profil)	Comparaisons massives lentes	Indexation vectorielle et cache Redis pour les encodages fréquents
Quota Azure STT dépassé lors des tests intensifs	Interruption du service vocal	Système de quota local et <i>fallback</i> Whisper (OpenAI)

Un défi transversal a concerné la gestion des dépendances circulaires entre modules dans une architecture Clean. Python, contrairement à Java, n'impose pas nativement l'inversion de dépendances, ce qui peut conduire à des couplages non voulus. Ce problème a été résolu par l'utilisation systématique d'interfaces (classes abstraites Python) et d'un conteneur d'injection de dépendances (`dependency_injector`).

IV.5 Conclusion

Le Cycle 2 a permis de concevoir, de développer et de valider le composant le plus structurant de notre écosystème : le Distributeur de Services. Ce backend, développé en Python avec FastAPI et organisé selon les principes de la Clean Architecture, assure la totalité des communications entre les différents acteurs du système avec rigueur et fiabilité.

Les choix technologiques ont été justifiés par des critères objectifs : la richesse de l'écosystème Python pour l'IA, les performances asynchrones de FastAPI, et la flexibilité de la Clean Architecture pour absorber les évolutions futures. Les services développés — reconnaissance faciale, synthèse et transcription vocale, communication robot en temps réel, gestion de la base de données — ont tous été validés par des campagnes de tests rigoureuses.

Ce composant constitue désormais une fondation solide sur laquelle les cycles ultérieurs pourront s'appuyer pour étendre les fonctionnalités du système, notamment l'enrichissement des interactions conversationnelles et l'amélioration des capacités autonomes du robot.

Points clés du Cycle 2

- Backend FastAPI + Python structuré en Clean Architecture
- Communication avec 5 composants extérieurs (robot, n8n, application mobile, Supabase, Azure/Gemini)
- Service de reconnaissance faciale avec 94,3 % de précision
- Intégration professionnelle des services Azure STT/TTS et de l'API Gemini

———— CHAPITRE V ————

CYCLE 3 : SYSTÈME EMBARQUÉ — PARTIE MATÉRIELLE

V.1 Introduction

Après le **Cycle 1** (moteur de réflexion n8n) et le **Cycle 2** (Distributeur de Services), le **Cycle 3** porte sur le **corps matériel** de KODA : choix des composants, câblage, alimentation et validation physique du prototype.

Ce chapitre reste volontairement centré sur le **hardware** et les **concepts d'intégration**. Le logiciel embarqué (scripts Raspberry Pi, protocoles série, audio, caméra) fera l'objet du **Cycle 4**.

Nous présentons l'architecture globale, la **conception 3D du châssis** (Blender), chaque sous-système (contrôle, actionneurs, audio, vision, affichage), le schéma de câblage, puis une campagne de **tests unitaires par composant** suivie d'un **test d'intégration** complète de la plateforme.

Objectif du Cycle 3

Concevoir, assembler et valider la plateforme matérielle du robot KODA, intégrant mobilité, interaction vocale, vision et expression faciale, prête à accueillir la couche logicielle embarquée du Cycle 4.

V.2 Architecture Matérielle Globale

L'architecture matérielle de KODA repose sur une conception **biprocasseur** articulée autour de deux unités de traitement complémentaires :

- **Raspberry Pi 4 Model B** : unité centrale intelligente, responsable du traitement des flux audio et vidéo, de la communication réseau avec le backend, et du pilotage de l'écran d'expression faciale.
- **Arduino Uno + Shield L293D** : unité de commande temps réel, dédiée au contrôle des actionneurs (servomoteurs et moteurs DC), recevant ses instructions du Raspberry Pi via une liaison série USB.

Cette séparation est motivée par un principe d'ingénierie fondamental : le Raspberry Pi, fonctionnant sous un système d'exploitation Linux, n'offre pas de garanties temps réel strictes pour le pilotage de signaux PWM. L'Arduino, en revanche, exécute un programme monotâche cadencé à 16 MHz, garantissant une précision temporelle adaptée au contrôle moteur.

Le tableau V.1 synthétise l'ensemble des composants matériels du robot.

TABLE V.1 – Inventaire des composants matériels de KODA

Catégorie	Composant	Rôle
Unité centrale	Raspberry Pi 4 Model B	Traitement IA, réseau, audio/vidéo
Commande moteur	Arduino Uno + Shield L293D	Pilotage PWM des actionneurs
Mobilité	2 moteurs DC (M1, M3)	Déplacement du robot (traction différentielle)
Articulations	3 servomoteurs (S1, S2, S3)	Bras droit, bras gauche ; S3 = écran Nextion
Entrée audio	ReSpeaker 4-Mic Array USB	Capture vocale + direction de la voix
Sortie audio	Jack + PAM8403 + HP ; Bluetooth	Restitution filaire ou sans fil
Vision	Caméra Raspberry Pi (CSI)	Acquisition d'images pour identification
Affichage	Nextion NX4832T035_011 (3,5")	Visage animé du robot
Alimentation Pi	Power Bank 5V / 3A	Alimentation du Raspberry Pi
Alimentation moteurs	3 × piles 3,7V	Alimentation du Shield L293D

V.3 Conception mécanique et châssis 3D (Blender)

Au-delà du câblage électronique, KODA repose sur un **habillage mécanique** imprimé en 3D, conçu sous **Blender**. Le point de départ n'était pas une création entièrement *from scratch*, mais un **modèle 3D existant** (humanoid simplifié) que nous avons dû **adapter** au prototype réel : découpe des volumes, redimensionnement des pièces, ajout d'ouvertures manquantes (logement écran Nextion, passage caméra sur le torse, fixation des bras) et harmonisation avec la base à chenilles.

Cette étape a constitué un **vrai défi** du Cycle 3 : chaque modification impacte l'encombrement interne (câbles, Raspberry Pi, batterie) et la cohérence visuelle du robot. La figure V.1 illustre la vue éclatée des pièces modélisées ; la figure V.2 montre le résultat physique après impression et assemblage.

Choix mécaniques retenus sur le prototype :

- **Caméra** : fixée sur le **torse** (corps), face avant, indépendante du servo S3.
- **Écran Nextion** : intégré dans la **tête**, entraîné par le servo S3 (rotation du visage animé seulement).
- **Base** : châssis à chenilles pour la mobilité (moteurs M1/M3).

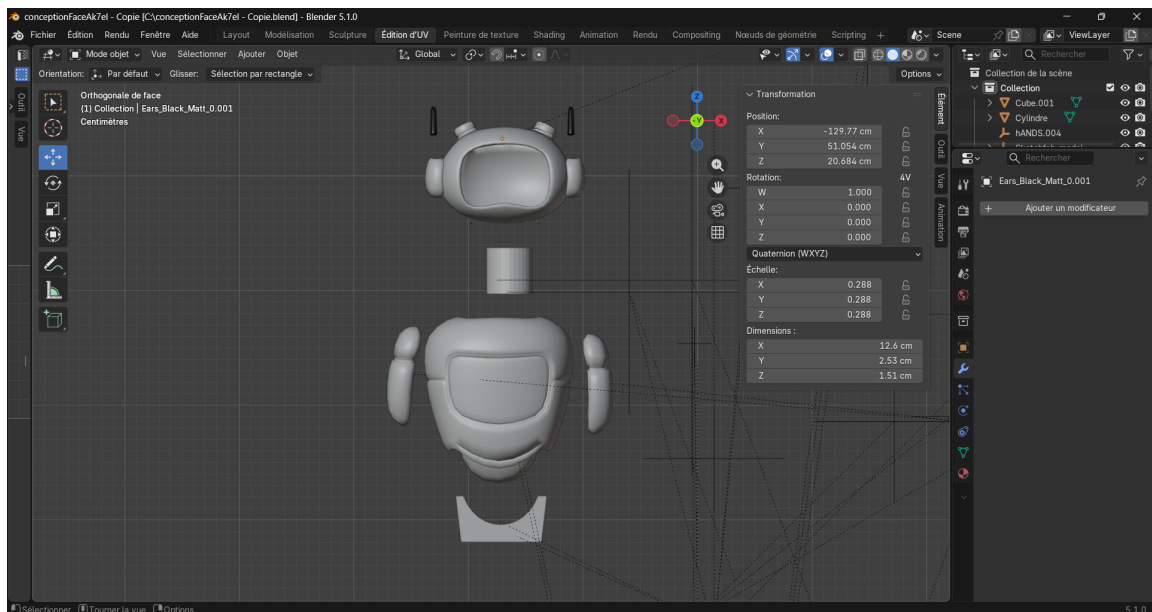


FIGURE V.1 – Conception 3D du châssis KODA sous Blender (vue éclatée des pièces)

V.4 Description Détaillée des Composants

V.4.1 Unité Centrale : Raspberry Pi 4 Model B

Le **Raspberry Pi 4 Model B** constitue le cœur du robot KODA. Il s'agit d'un ordinateur monocarte (SBC — *Single Board Computer*) basé sur un processeur ARM Cortex-A72 quadricœur cadencé à 1,5 GHz, équipé de 4 Go de mémoire RAM LPDDR4.

Dans l'architecture de KODA, le Raspberry Pi assure les fonctions suivantes :

- **Communication réseau** : connexion Wi-Fi au backend (Distributeur de Services) pour l'échange de données avec les workflows n8n et la base de données.
- **Traitement audio** : ReSpeaker en entrée (USB) ; sortie soit via le **jack 3,5 mm** et l'amplificateur PAM8403, soit via **Bluetooth** vers un haut-parleur externe.
- **Traitement vidéo** : acquisition d'images via la caméra pour la reconnaissance faciale.
- **Pilotage de l'écran Nextion** : communication série UART pour l'affichage des expressions faciales.
- **Commande de l'Arduino** : envoi d'instructions de mouvement via la liaison série USB.

V.4.2 Unité de Commande Moteur : Arduino Uno + Shield L293D

L'**Arduino Uno** est une carte de développement à microcontrôleur ATmega328P, cadencé à 16 MHz. Il est couplé au **Shield L293D**, un circuit intégré de pilotage moteur (*motor driver*) qui permet de contrôler simultanément jusqu'à 4 moteurs DC et 2 servomoteurs.

Dans KODA, l'Arduino reçoit ses instructions du Raspberry Pi via une **liaison série USB** et



FIGURE V.2 – Prototype physique KODA après impression 3D et montage

TABLE V.2 – Spécifications techniques du Raspberry Pi 4 Model B

Caractéristique	Valeur
Processeur	Broadcom BCM2711, Cortex-A72 quad-core @ 1,5 GHz
Mémoire	4 Go LPDDR4
Connectivité	Wi-Fi 802.11ac, Bluetooth 5.0
Ports USB	2 × USB 3.0 + 2 × USB 2.0
Sortie audio	Jack 3,5 mm stéréo
Interface caméra	Port CSI (Camera Serial Interface)
GPIO	40 broches (dont UART, I2C, SPI)
Alimentation	5V / 3A via USB-C

pilote :

- **2 moteurs DC** (bornes **M1** et **M3** du shield) : déplacement par traction différentielle.
- **3 servomoteurs** :
 - **S1** : bras droit
 - **S2** : bras gauche
 - **S3** : tête / cou (rotation de l'écran Nextion uniquement)

Le Shield L293D est alimenté indépendamment par **3 piles lithium de 3,7 V** connectées en série, fournissant une tension nominale d'environ 11,1 V nécessaire au fonctionnement des moteurs DC sous charge.

V.4.3 Système d'Actionneurs

V.4.3.1 Moteurs DC — Mobilité

Les deux moteurs à courant continu assurent la **locomotion** du robot par un système de **traction différentielle** : en faisant varier indépendamment la vitesse et le sens de rotation de chaque moteur, le robot peut avancer, reculer, tourner à gauche ou à droite. Ce principe simple et éprouvé est largement utilisé en robotique mobile (figure V.3).

V.4.3.2 Servomoteurs — Articulations

Les trois servomoteurs offrent à KODA des capacités d'**expression corporelle** :

- **Servomoteur S1 (bras droit) et S2 (bras gauche)** : permettent de simuler des gestes d'accueil, de salutation ou d'indication directionnelle.
- **Servomoteur S3 (tête / cou)** : oriente uniquement l'**écran Nextion** (visage animé). La **caméra** est fixée sur le **châssis** (corps) du robot et ne suit pas ce mouvement.

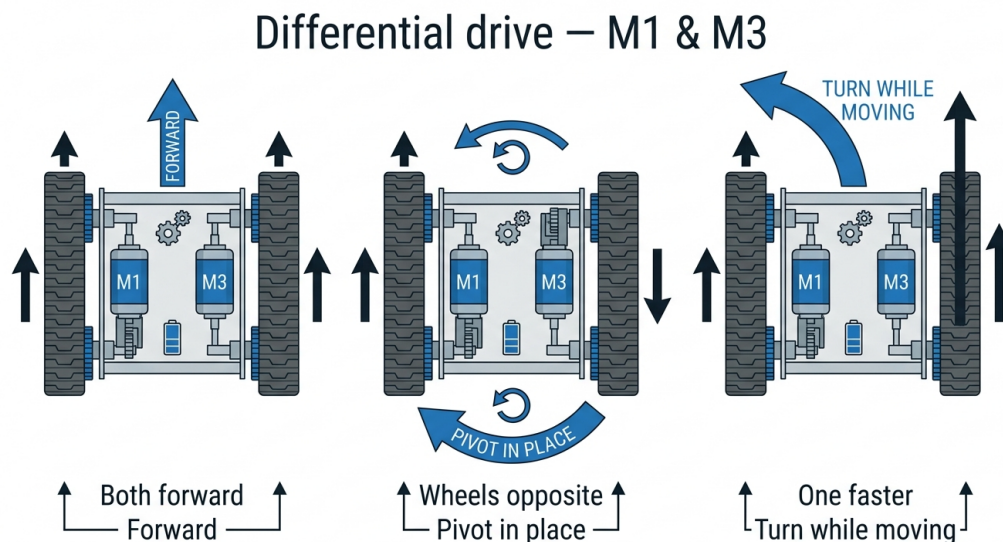


FIGURE V.3 – Principe de traction différentielle — moteurs M1 et M3

V.4.4 Système Audio

V.4.4.1 Entrée Audio : ReSpeaker 4-Mic Array USB

Le **ReSpeaker 4-Mic Array** de Seeed Studio est un périphérique USB intégrant quatre microphones MEMS disposés en configuration circulaire. Cette disposition permet deux fonctionnalités essentielles :

- **Capture vocale omnidirectionnelle** : enregistrement de la voix de l'utilisateur quelle que soit sa position autour du robot.
- **Détection de la Direction d'Arrivée (DOA — *Direction of Arrival*)** : identification de l'angle d'où provient la voix. Le robot **pivote sur place** grâce aux **deux moteurs DC** (M1, M3) pour s'orienter vers l'interlocuteur, puis la **caméra** (fixée sur le corps) permet la détection du visage (figure V.4).

Le ReSpeaker est connecté au Raspberry Pi via un **port USB** et est reconnu comme un périphérique audio standard sous Linux.

V.4.4.2 Sortie Audio : double chaîne (jack filaire et Bluetooth)

KODA dispose de **deux modes de restitution** de la parole, tous deux pilotés depuis le Raspberry Pi :

- **Chaîne filaire** : sortie **jack 3,5 mm** du Pi → amplificateur **PAM8403** → deux haut-parleurs (gauche / droite). Solution compacte, intégrée au châssis.
- **Chaîne sans fil** : le Bluetooth intégré du Raspberry Pi 4 peut envoyer le flux audio vers un

Direction of arrival (DOA) — KODA

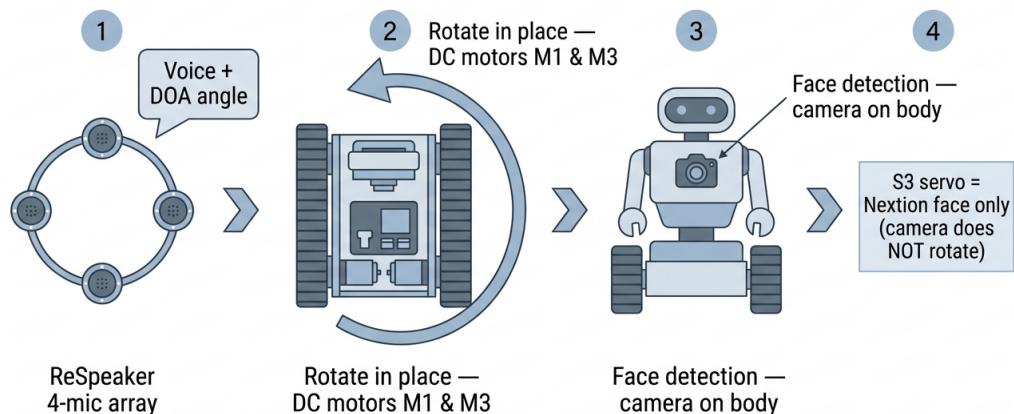


FIGURE V.4 – Enchaînement conceptuel : DOA (micro) → rotation du robot (M1, M3) → caméra (torse)

haut-parleur Bluetooth (appairage classique). Utile pour les essais ou une sortie plus puissante sans modifier le câblage jack.

Le choix entre les deux dépend du contexte de démonstration ; la couche logicielle (Cycle 4) sélectionnera la carte audio ALSA appropriée.

V.4.5 Système de Vision : Caméra Raspberry Pi

La **caméra officielle Raspberry Pi** est connectée via le port **CSI** (nappe dédiée). Elle fournit les images nécessaires à l'**identification visuelle** des utilisateurs.

Au niveau matériel, seule la **qualité de capture** (netteté, éclairage, cadrage) est concernée dans ce cycle. L'envoi des images vers le serveur distant et le traitement logiciel seront détaillés au **Cycle 4**.

V.4.6 Interface d'Expression Faciale : Nextion NX4832T035_011

L'écran **Nextion NX4832T035_011** (3,5 pouces) intègre son propre microcontrôleur : le Raspberry Pi n'affiche pas des pixels en temps réel, mais envoie des **commandes série** pour changer de page ou d'expression (sourire, surprise, écoute, etc.) déjà stockées dans la mémoire de l'écran.

Alimentation et liaison (câblage retenu sur le prototype) :

- **5 V** et **GND** du Nextion reliés au **5 V** et **GND** du GPIO du Raspberry Pi (masse commune).
- **RX** du Nextion (fil jaune) → broche **8** du Pi (**GPIO 14**, ligne **TX** du Pi).
- **TX** du Nextion (fil bleu) → broche **10** du Pi (**GPIO 15**, ligne **RX** du Pi).

Ce câblage UART permet au Pi d'animer le **visage** de KODA sans surcharger le bus USB, tout

en gardant une interface visuelle dédiée à l'interaction humaine.

V.4.7 Système d’Alimentation

Le robot KODA dispose de **deux sources d'alimentation indépendantes**, séparant volontairement la partie logique de la partie motrice :

TABLE V.3 – Architecture d'alimentation de KODA

Source	Caractéristiques	Composants alimentés
Power Bank	5V / 3A, USB-C	Raspberry Pi 4 (et par extension : ReSpeaker, caméra, écran Nextion via GPIO)
3× piles Li-ion	3,7V chacune (11,1V total)	Shield L293D, moteurs DC, servo-moteurs

Cette séparation protège le Raspberry Pi contre les perturbations électromagnétiques et les chutes de tension provoquées par les appels de courant des moteurs.

V.5 Schéma de Câblage Global

La figure V.5 présente le **schéma de montage** du prototype : vue type *Fritzing*, avec l'ensemble des connexions entre Raspberry Pi, Arduino, capteurs, actionneurs et alimentations.

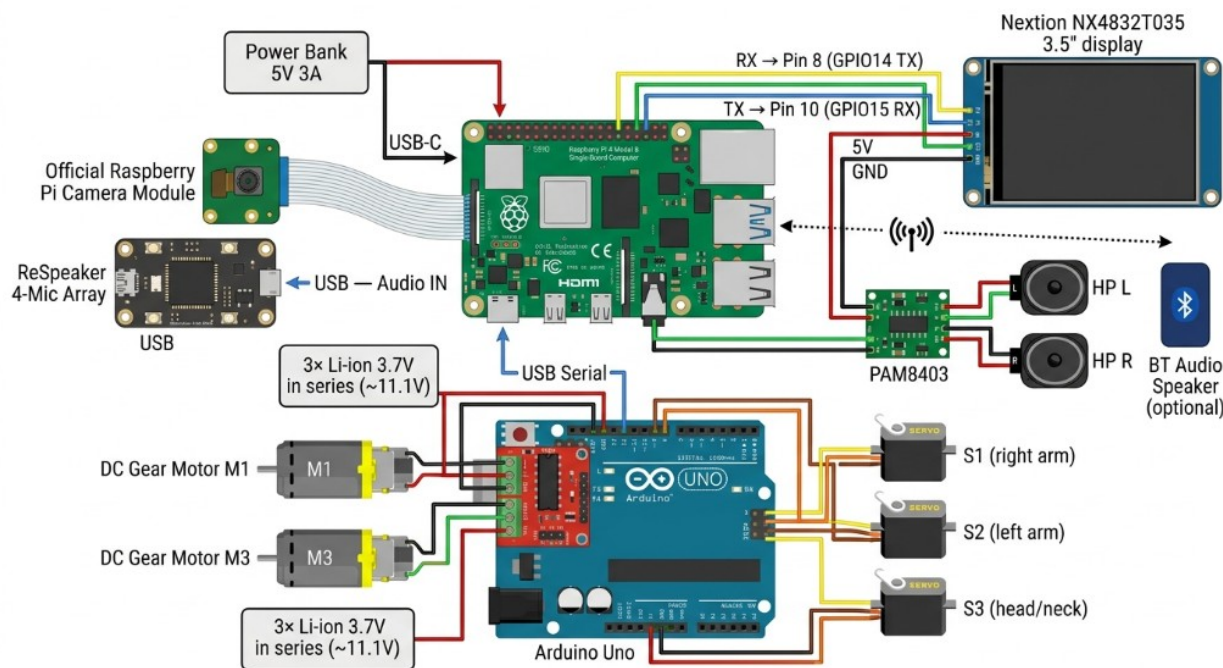


FIGURE V.5 – Schéma de montage matériel de KODA — style Fritzing (Cycle 3)

Le tableau V.4 récapitule les connexions pour la rédaction et la maintenance du prototype.

TABLE V.4 – Récapitulatif des connexions entre composants

Composant	Connecté à	Interface	Détail
Arduino Uno	Raspberry Pi	USB	Commandes moteur (série)
ReSpeaker 4-Mic	Raspberry Pi	USB	Entrée audio + DOA
PAM8403 + HP L/R	Raspberry Pi	Jack 3,5 mm	Sortie filaire
HP Bluetooth	Raspberry Pi	Bluetooth	Sortie sans fil (option)
Caméra Pi	Raspberry Pi	CSI	Nappe caméra
Nextion NX4832T035_011	Raspberry Pi	UART alim +	5V/GND GPIO ; RX → pin 8 ; TX → pin 10
Moteurs DC M1, M3	Shield L293D	Bornes M1, M3	Traction différentielle
Servos S1, S2, S3	Shield L293D	Servo 1, 2, 3	Bras D/G ; S3 = Nextion
Power Bank 5V/3A	Raspberry Pi	USB-C	Alimentation logique
3 × piles 3,7V (série)	Shield L293D	Vin	Alimentation motrice

V.6 Assemblage et points d'attention

L'intégration physique a mis en évidence quelques **contraintes** récurrentes, sans entrer dans le détail logiciel (Cycle 4) :

- **Séparation des alimentations** : éviter de nourrir les moteurs depuis le 5 V du Pi ; les perturbations PWM peuvent provoquer des réinitialisations ou du bruit sur l'audio.
- **UART Nextion** : respecter le croisement TX/RX (fil RX de l'écran vers TX du Pi) et une masse commune stable.
- **USB multiples** : ReSpeaker, Arduino et alimentation compétent pour les ports USB du Pi ; une alimentation 5 V/3 A fiable (power bank) est indispensable.
- **Encombrement mécanique** : la nappe CSI (caméra sur le corps) et les câbles des servos (dont S3 pour le Nextion) doivent rester libres de frottement lors des mouvements articulés.

V.7 Validation matérielle — tests par composant

Chaque sous-système a été validé **individuellement** avant le test global (contrôle visuel ou fonctionnel, sans détailler les scripts). Le tableau V.5 ci-dessous synthétise cette démarche.

TABLE V.5 – Tests unitaires par composant (Cycle 3)

Composant	Procédure (concept)	Critère OK
Power Bank + Pi	Mise sous tension, boot Raspberry Pi OS	LED activité, pas de brown-out
ReSpeaker USB	Enregistrement court, niveau sonore	Signal audible, périphérique reconnu
Sortie jack + PAM8403	Lecture d'un son test sur les HP	Son clair L/R
Sortie Bluetooth	Appairage HP BT, lecture test	Son sur HP externe
Caméra CSI	Capture d'une image fixe	Image nette, pas de bandes
Nextion UART	Changement de page ou d'expression	Affichage réactif
Arduino + USB	Envoi d'une commande simple (moniteur série)	Réponse attendue
Moteurs M1, M3	Avancer, reculer, pivoter	Roues cohérentes
Servos S1, S2, S3	Balayage angle min-max	Mouvement fluide, sans à-coups
Alim. 3 piles (série)	Charge moteurs à vide	Tension stable sur Vin

V.7.1 Test d'intégration matérielle

Le **test complet** consiste à alimenter l'ensemble, vérifier que le Pi détecte simultanément micro, caméra et écran, que l'Arduino répond toujours sur USB, et qu'un scénario court combine : capture vocale, mouvement des servos, rotation des roues, changement d'expression Nextion, sans coupure d'alimentation ni conflit USB. Ce test valide la **plateforme prête pour le Cycle 4** (logiciel embarqué et liaison réseau avec le backend).

V.8 Conclusion

Ce chapitre a présenté l'intégralité de la plateforme matérielle du robot KODA. L'architecture biprocesseur retenue — Raspberry Pi pour l'intelligence et l'Arduino pour le contrôle moteur — garantit à la fois la puissance de calcul nécessaire aux traitements IA et la précision temporelle

requis pour le pilotage des actionneurs.

Chaque composant a été sélectionné et intégré pour répondre aux exigences fonctionnelles définies au chapitre II : mobilité, interaction vocale bidirectionnelle, reconnaissance visuelle et expression émotionnelle. La plateforme matérielle est désormais prête à accueillir la couche logicielle embarquée, objet du **Cycle 4** présenté dans le chapitre suivant.

Points clés du Cycle 3

- Architecture biprocesseur : Raspberry Pi 4 + Arduino Uno / L293D
- 5 actionneurs : M1/M3 (DC) + S1/S2 (bras) + S3 (Nextion)
- Audio : ReSpeaker (entrée); sortie jack/PAM8403 ou Bluetooth
- Nextion NX4832T035_011 (UART GPIO 8/10, alim. 5 V Pi)
- Tests unitaires puis intégration; logiciel au Cycle 4